NVFP4 and Microscaled 4-Bit Floating-Point Formats for Large-Language-Model Training and Inference:

Mathematical Foundations, Empirical Evidence, Failure Modes, and Reception (September 2025 – October 2026)

Literature synthesis with original derivations and numerical experiments

27 October 2026

Abstract

Between September 2025 and October 2026 the 4-bit floating-point *microscaling* family moved from a hardware feature announcement to a format in which at least one publicly documented open-weight model was pretrained from scratch. This report synthesises the primary literature of that period on NVFP4 (16-element blocks, E4M3 block scales, an FP32 tensor scale) and its relatives MXFP4, UE5M3-scaled variants, "Four Over Six" adaptive scaling and HiFloat4, for both training and inference.

The report has four aims. First, it gives a self-contained mathematical treatment: the E2M1 grid, two-level scaling, mean-squared error under Gaussian and Laplace inputs (with an exact quadrature for block-absmax scaling), the effect of rounding the scale to E4M3 or to a power of two, stochastic rounding (unbiasedness, variance, headroom), random Hadamard transforms (a sub-Gaussian bound on block crest factors and a resulting bound on stochastic-rounding variance), gradient bias, accumulation in block-scaled GEMMs and a scaling-law view of the loss gap. Second, it collects published empirical results into tables with citations. Third, it reports original CPU simulations, all clearly marked as such, that check the mathematics and probe questions the literature leaves open. Fourth, it records failure modes, the reception of the format in the research community and in practitioner media, and open problems.

The main conclusions are as follows. (i) NVFP4 is, at the level of raw quantisation error, a modest improvement over MXFP4 (relative MSE about 0.0090 versus 0.0133 for Gaussian data in our simulation) that comes from the combination of the finer scale format and the smaller block, not from the element grid, which is identical; at equal storage the E4M3 scale accounts for the larger share (22% lower MSE at block size 32, against a further 12% from halving the block). (ii) Published pretraining results show relative loss gaps of roughly 1–2$\%$ against FP8 or BF16 baselines that can be partially closed by a late high-precision phase, and a careful 2026 comparison across recipes reports gaps that are small but strongly recipe- and horizon-dependent, with all-layer NVFP4 diverging in at least one setting. (iii) Many of the headline throughput and energy numbers are vendor figures and are not independently verified in the sources we could retrieve. (iv) Several design choices that are presented as essential in one paper (random Hadamard transforms, two-dimensional weight scaling) are contradicted or qualified in others, and our own simulations agree that rotation does not reduce error for NVFP4 with round-to-nearest. (v) Our audit of the flagship 12B downstream comparison shows that, given test-set sampling noise alone, most benchmark differences are statistically indistinguishable from zero, so the evidence is consistent with parity but cannot rank the precisions. (vi) A failure mode reported for a 120B-parameter run, zero weight gradients from low-norm channels in 16×16 weight tiles, is reproduced and predicted quantitatively by a closed-form zero-fraction formula, while total MSE hides it. (vii) On a small real model, weight MSE predicts perplexity within a format family but not across families. We flag where the evidence is thin.

Scope and honesty statement. We had no access to Blackwell hardware. All original experiments are float32 "fake-quantisation" simulations on a two-core CPU, at toy scale. No claim about wall-clock speed, energy or large-scale training is based on our own measurements. Every empirical statement is tagged with its provenance (Section 1.4).

Contents

Abstract i

1 Introduction 1
- 1.1 Why four bits . . . 1
- 1.2 What happened between September 2025 and October 2026 . . . 1
- 1.3 Contributions and structure of this report . . . 2
- 1.4 Evidence tags and conventions . . . 2

2 History and Context 4
- 2.1 From 32 bits to 8 bits . . . 4
- 2.2 Fine-grained scaling . . . 4
- 2.3 First attempts at 4-bit training . . . 5
- 2.4 NVFP4 and the Blackwell generation . . . 5
- 2.5 Timeline of the period . . . 5
- 2.6 Why a second look is warranted . . . 5

3 The Formats 7
- 3.1 Minifloat formats . . . 7
- 3.2 The E2M1 element grid . . . 7
- 3.3 Block-scaled formats . . . 8
  - 3.3.1 MXFP4 . . . 9
  - 3.3.2 NVFP4 . . . 9
  - 3.3.3 Two-dimensional (16×16) scaling . . . 9
- 3.4 Comparison table . . . 10
- 3.5 Software support . . . 10
- 3.6 Dequantisation to FP8 and mixed pipelines . . . 10

4 Quantisation Error of Block-Scaled FP4 12
- 4.1 Setting and metrics . . . 12
- 4.2 Round-to-nearest onto the E2M1 grid . . . 12
- 4.3 Absmax scaling: an exact expected-MSE formula . . . 13
  - 4.3.1 A decomposition: MSE $= \eta\,\mathbb{E}[A^2]/Q_{\max}^2$ . . . 13
- 4.4 Where in the grid the error lives . . . 15

4.5 Rounding the scale: E4M3 and power-of-two . . . 15
4.5.1 E4M3 block scales: a first-order penalty model . . . 15
4.5.2 Power-of-two scales (MXFP4) . . . 16
4.6 Block size and bits per element . . . 17
4.7 The dynamic range of the two-level scale . . . 18
4.8 INT4 versus FP4 elements . . . 19
4.9 Four Over Six: choosing the scale by error . . . 20
4.10 Summary of the chapter . . . 21

5 Stochastic Rounding and Gradient Bias 22
5.1 Stochastic rounding on the E2M1 grid . . . 22
5.2 Headroom: when is block-scaled SR actually unbiased? . . . 23
5.3 Why bias matters: a convergence bound . . . 23
5.4 RTN bias in practice: the dead zone . . . 24
5.5 SR for GEMM operands . . . 25
5.6 MS-EDEN: unbiased gradients without element-wise SR . . . 26
5.7 Is SR necessary? Quartet's reversal . . . 27
5.8 Summary . . . 27

6 Random Hadamard Transforms and Outliers 28
6.1 Definition and invariance . . . 28
6.2 What a rotation does to a block . . . 29
6.2.1 Gaussianisation and a crest-factor bound . . . 29
6.2.2 Rotation interacts with the shape of the grid . . . 29
6.3 Simulation: rotation versus input distribution . . . 30
6.4 Variance of stochastic-rounding products under rotation . . . 31
6.5 Rotation in the weight-gradient GEMM . . . 33
6.6 Costs . . . 33
6.7 Summary . . . 34

7 Block-Scaled GEMM: Arithmetic and Accumulation 35
7.1 Structure of a block-scaled dot product . . . 35
7.2 Accumulation error: a sequential-sum model . . . 35
7.3 Systematic gain of the quantisers . . . 36
7.4 From tensor error to GEMM error to layer error . . . 37
7.5 Summary . . . 37

8 Training Recipes 39
8.1 The NVIDIA recipe . . . 39
8.1.1 Setting . . . 39
8.1.2 Components . . . 39
8.1.3 Reported results . . . 40

8.1.4 Why the evidence is thinner than it looks . . . 41
8.2 Competing and refining recipes . . . 41
8.2.1 Quartet and Quartet II . . . 41
8.2.2 TetraJet-v2 . . . 42
8.2.3 Four Over Six in pretraining . . . 42
8.2.4 "What Matters for NVFP4 Training?" . . . 42
8.2.5 Transposition-invariant 2D scaling, full-stack FP4, CHON . . . 44
8.2.6 AMD MXFP4 on native FP4 hardware . . . 44
8.2.7 Graphcore: UE5M3 scales and a simpler recipe . . . 44
8.3 Comparison across recipes . . . 45
8.4 Points of disagreement . . . 45

9 Loss Gap, Scaling Laws and the Economics of 4-Bit Training 47
9.1 A second-order model of the loss gap . . . 47
9.2 Why the gap grows late in training . . . 48
9.3 Scaling-law formulations . . . 48
9.3.1 Quartet's parameter and data efficiencies . . . 48
9.3.2 An empirical slope from NVIDIA's MXFP4 versus NVFP4 comparison . . . 49
9.4 Economics: when does the gap eat the speed-up? . . . 50
9.5 Summary . . . 51

10 Inference: Post-Training Quantisation, QAT and Distillation 52
10.1 What PTQ to NVFP4 has to do . . . 52
10.2 Reported PTQ accuracy of NVFP4 . . . 52
10.2.1 Large reasoning models (vendor) . . . 52
10.2.2 Open-source models of many sizes (Red Hat) . . . 53
10.2.3 Controlled comparison of PTQ methods: MR-GPTQ . . . 53
10.2.4 Four Over Six in PTQ . . . 54
10.3 Why PTQ degrades on reasoning models: QAT versus QAD . . . 55
10.3.1 The method . . . 55
10.3.2 Results . . . 55
10.3.3 Reading the QAT/QAD evidence . . . 56
10.4 PTQ in the production recipe of Nemotron 3 Super . . . 56
10.5 What PTQ cannot fix . . . 57
10.6 Summary . . . 58

11 Hardware, Kernels and Systems 59
11.1 Peak throughput and memory . . . 59
11.1.1 Published peak numbers . . . 59
11.1.2 Memory footprint . . . 59
11.2 Kernel-level constraints from the Transformer Engine . . . 60

11.3 A roofline model of the quantisation overhead . . . 60
11.4 Reported speed-ups . . . 61
11.5 Software ecosystem . . . 62
11.5.1 Frameworks . . . 62
11.5.2 Other hardware . . . 63
11.6 Energy . . . 63
11.7 Summary . . . 63

12 An Audit of the Empirical Evidence 64
12.1 Putting loss gaps into common units . . . 64
12.2 Sampling noise in the downstream benchmarks of the 12B comparison . . . 64
12.3 A ledger of the main claims . . . 66
12.4 Recurrent weaknesses . . . 66
12.5 Summary . . . 67

13 Our Numerical Experiments 69
13.1 What fake-quantisation experiments can show . . . 69
13.2 Scoreboard: what we expected and what we found . . . 69
13.3 Real weights and perplexity . . . 70
13.3.1 Design . . . 70
13.3.2 Weight error . . . 70
13.3.3 Perplexity . . . 71
13.3.4 Activation quantisation and rotation . . . 72
13.4 Toy-scale training under simulated recipes . . . 72
13.4.1 Design . . . 72
13.4.2 Results . . . 73
13.4.3 Limitations of the toy experiment . . . 74

14 Alternatives and Competing Formats 75
14.1 Iso-bit comparison of 4-bit block formats . . . 75
14.2 Integer formats with fine-grained scales . . . 76
14.2.1 The INT-versus-FP study . . . 76
14.2.2 INT4 in production models . . . 76
14.3 Power-of-two scales: MXFP4 . . . 77
14.4 Wider-range scales: UE5M3 . . . 77
14.5 HiFloat4: hierarchical scaling . . . 78
14.6 Eight-bit baselines . . . 79
14.7 Weight-only and rotation-based 4-bit methods . . . 79
14.8 Summary . . . 79

15 Strengths, Weaknesses and Failure Modes 80
15.1 Advantages . . . 80

15.2 Disadvantages . . . 80
15.3 Catalogue of failure modes . . . 81
15.4 Failure mode I: low-norm channels inside two-dimensional tiles . . . 81
15.5 Failure mode II: forward noise and the late-training gap . . . 83
15.6 Failure mode III: instabilities that are not numerical . . . 83
15.7 Which failures are most likely to bite in practice . . . 83

16 Reception, Perception and Controversies 84
16.1 Sources and how much they weigh . . . 84
16.2 Press and newsletter reception . . . 84
16.3 Practitioner reception: the inference side . . . 85
16.4 A case of disappointment: GLM-5.2 . . . 85
16.5 Controversies in the research literature . . . 86
16.6 What the community does not seem to be arguing about . . . 87
16.7 Summary . . . 87

17 Case Studies 88
17.1 Summary comparison . . . 88
17.2 Nemotron 3 Super . . . 88
17.2.1 Pretraining . . . 88
17.2.2 Findings that go beyond the original paper . . . 88
17.2.3 Accuracy of the deployed checkpoints . . . 89
17.3 DeepSeek-V4 . . . 89
17.4 gpt-oss . . . 90
17.5 Kimi K2 Thinking . . . 90
17.6 NVFP4 beyond text: video, KV cache, attention . . . 90
17.7 Lessons across the cases . . . 91

18 Open Problems and Outlook 92
18.1 Open problems . . . 92
18.1.1 Independent replication at scale . . . 92
18.1.2 Removing the exemptions . . . 92
18.1.3 A theory of the late-training gap . . . 92
18.1.4 Scale-format design . . . 93
18.1.5 When to rotate . . . 93
18.1.6 Low-precision optimiser states, communication and attention . . . 93
18.1.7 Reinforcement learning with FP4 rollouts . . . 93
18.1.8 Joint scaling laws . . . 93
18.1.9 Evaluation methodology . . . 94
18.1.10 Hardware co-design . . . 94
18.2 Below four bits? . . . 94

18.3 Outlook . . . 94

19 Conclusions **96**
19.1 What is established . . . 96
19.2 What is plausible but thinly supported . . . 97
19.3 What remains open . . . 97
19.4 Limitations of this report . . . 97
19.5 Closing assessment . . . 98

A Supplementary Derivations **99**
A.1 Dynamic range of the block-scaled formats . . . 99
A.2 Overflow-free tensor scale with the Four Over Six rule . . . 100
A.3 The gain of a rounding quantiser . . . 100
A.4 Stationary excess loss of SGD on a quadratic . . . 100
A.5 Monte Carlo uncertainty of the synthetic experiments . . . 102
A.6 The tile-zero probability with a single large channel . . . 102
A.7 Variance of the quantisation-error estimate in a GEMM output . . . 102
A.8 Notation reference . . . 103

B Simulation Code **104**
B.1 Quantiser library (`fp4lib.py`) . . . 104
B.2 Quantised linear layer (`qlinear.py`) . . . 108
B.3 Experiment scripts . . . 109
B.3.1 Exp. 2: analytic MSE and the scale-range anomaly (`exp2_analytic_and_anomaly.py`) . . . 109
B.3.2 Exp. 4: stochastic rounding in GEMM (`exp4_sr_gemm.py`) . . . 111
B.3.3 Exp. 5b: SR dot-product variance with and without RHT (`exp5b_sr_variance_rht.py`) . . . 113
B.3.4 Exp. 6: real weights and perplexity (`exp6_real_weights.py`) . . . 113
B.3.5 Exp. 7: toy training (`exp7_toy_train.py`) . . . 115
B.3.6 Exp. 12: low-norm channels in 2D tiles (`exp12_lowchannel.py`) . . . 117
B.3.7 Exp. 14: SGD stationary loss and Monte Carlo errors (`exp14_ou.py`) . . . 118
B.4 Other files . . . 119

C Supplementary Result Tables **120**
C.1 Catalogue of experiments . . . 120
C.2 Signal-to-noise form of the format comparison . . . 121
C.3 Block-size sweeps for the remaining distributions . . . 121
C.4 Real-weight results . . . 121

D Glossary **123**

Chapter 1

Introduction

1.1 Why four bits

Training and serving large language models is limited by three resources: arithmetic throughput, memory capacity and memory bandwidth. Each halving of the number of bits per number roughly halves the memory footprint and the bandwidth cost of moving a tensor, and on hardware that provides native low-precision matrix units it can roughly double the peak arithmetic rate. The industry has moved down this ladder in steps: FP32 to FP16/BF16, then FP8 [1], and, with the Blackwell generation of NVIDIA GPUs, to 4-bit floating-point formats executed natively in the tensor cores.

Four bits is not a number that can represent a neural-network tensor on its own. A signed 4-bit floating-point value with two exponent bits and one mantissa bit (E2M1) takes only sixteen codes, of which fifteen are distinct values: $\{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$. Its dynamic range is a factor 12 between the smallest and largest non-zero magnitude. A trained tensor spans many orders of magnitude. The microscaling idea is to share a higher-precision scale factor among a small *block* of elements, so that the 4-bit code only has to describe the shape of the block and the scale describes its magnitude. The two formats that matter in practice differ in the block size and in the scale:

- **MXFP4** (Open Compute Project): blocks of 32 elements, scale in E8M0 (a pure power of two), 4.25 bits per element [2].
- **NVFP4** (NVIDIA): blocks of 16 elements, scale in FP8 E4M3, plus one FP32 scale per tensor, 4.5 bits per element plus a negligible tensor-level overhead [3, 4].

1.2 What happened between September 2025 and October 2026

The period covered here begins with the publication of NVIDIA's technical report on pretraining a 12-billion-parameter hybrid Mamba–Transformer on 10 trillion tokens in NVFP4 [3]. In the thirteen months that followed, the literature moved quickly:

- Academic groups produced competing training recipes (Quartet II, TetraJet-v2, Four Over Six, transposition-invariant block quantisation, full-stack FP4) and a comparative study at up to 1 trillion tokens [5–10].
- Critical analyses questioned whether the format's design choices are optimal. They examined integer versus floating-point elements, the role of rotations, and the limited range of the E4M3 scale [11–14].
- A frontier-scale open model, NVIDIA Nemotron 3 Super (about 120B total and 12.7B active

parameters), reported being pretrained in NVFP4 for the entire 25-trillion-token run [15].

- Other labs took different routes: DeepSeek-V4 uses FP4 (MXFP4) quantisation-aware training late in post-training for expert weights and one attention path rather than FP4 pretraining [16]; OpenAI's gpt-oss releases ship MXFP4 expert weights for inference [17]; Moonshot's Kimi K2 Thinking chose INT4 quantisation-aware training [18]; Huawei proposed HiFloat4 for its own accelerators [19].
- Practitioners reported mixed experience with NVFP4 post-training quantisation: strong recovery on large models in some write-ups, and anecdotal degradation on long-horizon agentic tasks in others [20, 21].

1.3 Contributions and structure of this report

This report is a literature synthesis with an original mathematical and computational component. We make the following contributions.

1. A unified notation and a rigorous derivation of the error properties of E2M1 block quantisation under absmax scaling (Chapters 3–4), including an exact quadrature for the expected MSE of block-absmax scaling for any symmetric density, validated against Monte Carlo (Section 4.3).
2. A first-order model of the penalty from rounding block scales to E4M3, a derivation of the dynamic-range limits of the two-level scale, and a prediction of the outlier magnitude at which blocks start to flush to zero, checked against simulation (Sections 4.5 and 4.7).
3. A sub-Gaussian bound for the block crest factor after a random Hadamard transform and a derived bound for the variance of stochastic-rounding dot products that explains the observed benefit of rotation for stochastic rounding but not for round-to-nearest (Chapters 5–6).
4. A systematic collection of reported numbers with sources (Chapters 10, 11, 12 and 17), separating peer-reviewed or arXiv results from vendor blogs and anecdotal material, with a statistical audit of the headline benchmark comparison and a ledger of claims.
5. Original simulation results (Chapter 13) on synthetic data, real small-model weights and a toy language-model training run.
6. An account of competing formats, controversies, failure modes (including a quantitative model of channel erasure in two-dimensional weight tiles) and open problems (Chapters 14–18).

1.4 Evidence tags and conventions

Because the subject is surrounded by marketing, we attach a provenance tag to claims that carry numbers or causal assertions:

Tag	Meaning
[S]	Reported in a primary source we retrieved and read (a paper, technical report, or model card). We paraphrase and cite; we have not reproduced the experiment.
[D]	Derived by us in this report (proof or calculation, shown in the text or in Appendix A).
[E]	Result of an experiment we ran (CPU simulation; code in Appendix B). These are simulations at small scale, not hardware measurements.
[V]	A vendor or promotional claim (company blog, press material). Not independently verified.
[U]	Uncertain: secondary reporting, anecdote, forum or newsletter content, or a claim whose source we could only partly check.

Notation. For a block $\boldsymbol{x} \in \mathbb{R}^G$ we write $\mathrm{amax}(\boldsymbol{x}) = \max_i |x_i|$. The FP4 (E2M1) grid is $\mathcal{G} = \{0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$ with $Q_{\max} = 6$. RTN is round-to-nearest-even onto $\mathcal{G}$, SR is stochastic rounding onto $\mathcal{G}$. Relative MSE is $\mathrm{MSE}(\hat{\boldsymbol{x}}) = \mathbb{E}\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|^2 / \mathbb{E}\|\boldsymbol{x}\|^2$ and $\mathrm{QSNR} = -10 \log_{10} \mathrm{MSE}$ in dB. Unless stated, "G" denotes the block size.

What we did not do. We did not run any code on Blackwell hardware; we did not verify any throughput, latency or energy figure; and we did not train models beyond a toy size. Where we cite a number from a vendor blog we say so. All dates in the text are publication dates of the cited versions. "Today" is 27 October 2026 and works released after our retrieval may be missing.

Chapter 2

History and Context

2.1 From 32 bits to 8 bits

Deep-learning training began in FP32. Mixed-precision training with FP16 storage, FP32 master weights and loss scaling [22] cut memory and raised throughput on tensor-core GPUs; the bfloat16 format later removed the need for loss scaling by keeping FP32's exponent range with a shorter mantissa. These are 16-bit formats with a *per-tensor* or no explicit scale.

The first jump to 8 bits was the FP8 interchange proposal by NVIDIA, Arm and Intel, which defined two encodings: E4M3 (four exponent bits, three mantissa bits; no infinities, a single NaN pattern, maximum finite value 448) and E5M2 (IEEE-like, five exponent bits) [1]. The authors showed that training with the hyper-parameters unchanged from a 16-bit baseline matched quality across CNNs, RNNs and Transformers, including language models up to 175B parameters.[S] The scaling granularity was still per tensor, with scale factors chosen from running statistics, which requires care because activation outliers consume the narrow dynamic range of an 8-bit format.

2.2 Fine-grained scaling

The step that made sub-8-bit formats plausible was moving the scale from the tensor to a small group of elements. DeepSeek-V3's FP8 training framework quantised activations in 1×128 tiles and weights in 128×128 blocks, each with its own scale, and reported a relative loss error against BF16 consistently below 0.25% at roughly one trillion tokens on two model scales.[S] The same report observed that on the H800 tensor cores the accumulation of FP8 products retained only about 14 bits of mantissa, with a maximum relative error of nearly 2% for $K = 4096$ in their test, and worked around it by periodically promoting partial sums to CUDA cores.[S] The report described its fine-grained scheme as "highly consistent" with the idea of microscaling formats and noted that Blackwell tensor cores had announced support for smaller scaling granularity.[S]

Microscaling was standardised by the Open Compute Project. The MX specification, with authors from AMD, Arm, Intel, Meta, Microsoft, NVIDIA and Qualcomm, defines a format by three parameters: block size k, the scale's data format and the element's data format, and instantiates MXINT8, MXFP8, MXFP6 and MXFP4 [2]. The shared scale is E8M0 and the block size is 32.[S] Blackwell's tensor cores execute MXFP8, MXFP6 and MXFP4 natively [23]. Mishra et al. showed that MXFP8 with the E4M3 element type matches BF16 pretraining for models up to 8B parameters and up to 15T tokens, provided the E8M0 scale is converted by *rounding up*, so that no scaled element exceeds the element format's maximum.[S] That detail—round the power-of-two scale up—returns in Chapter 4, where it is the difference between saturation and loss of precision in MXFP4.

2.3 First attempts at 4-bit training

Four-bit training predates Blackwell. Xi et al. trained Transformers with 4-bit integers; Chmiel et al. proposed accurate 4-bit matrix multiplications at standard formats [24, 25]. These used per-tensor or per-token INT4 and special handling of gradients (for example logarithmic unbiased quantisation of the backward pass).

Once FP4 hardware was announced, several preprint groups simulated FP4 on older GPUs. Wang et al. (January 2025) proposed a differentiable quantisation estimator for the weights and outlier clamping and compensation for activations, with vector-wise scaling, and trained up to 13B parameters on up to 100B tokens, reporting accuracy comparable to BF16 and FP8 with a mixed-precision scheme[26].[S] Tseng et al. (February 2025) used MXFP4 GEMMs in the backward pass of GPT models up to 6.7B parameters, combining stochastic rounding with the random Hadamard transform to bound the variance of stochastic rounding, and estimated a speed-up of more than $1.3\times$ over FP8 and $1.7\times$ over BF16 in the backward pass[27].[S] Zhou et al. and Chen et al. explored FP4 and MXFP4 pretraining (the latter for vision Transformers with an oscillation-reduction method)[28, 29]. Castro et al. introduced Quartet (May 2025), a scaling-law-guided MXFP4 recipe with a Blackwell kernel implementation[30]. Chmiel et al. (May 2025) trained a 7B model on 256 Intel Gaudi2 accelerators on datasets up to 200B tokens predominantly in FP4, concluding that the NVFP4 block layout (16 elements, E4M3 scale) gave the best results among the options they compared, using stochastic rounding for backward and update passes and round-to-nearest in the forward pass, and identified an empirical threshold for effective quantised training when the gradient norm falls below about $\sqrt{3}$ times the quantisation noise[31].[S]

2.4 NVFP4 and the Blackwell generation

Blackwell GPUs were announced in 2024 and first shipped in volume in 2025. NVIDIA introduced the name **NVFP4** in a June 2025 technical blog on inference, describing it as a 4-bit format with 16-element blocks, E4M3 block scales and a second-level FP32 scale, and presenting post-training-quantisation results for DeepSeek-R1-0528 and other models[4].[V] An August 2025 follow-on blog announced NVFP4 *training* and claimed a $7\times$ GEMM speed-up on GB300 relative to Hopper[32].[V] The technical report that backs the training claim was posted in late September 2025[3].[S]

The Blackwell Ultra generation raises dense NVFP4 throughput to 15 PFLOPS per GPU versus 10 PFLOPS for the original Blackwell, according to NVIDIA[33].[V] The MLPerf Training v5.1 round (November 2025) included a Llama 3.1 405B result using NVFP4 on 2,560 GPUs in 18.79 minutes and on 5,120 GPUs in 10 minutes, with the final iterations run in FP8; NVIDIA's accompanying blog attributes a factor-of-three improvement to the combination of NVFP4 and software gains, so the NVFP4 contribution alone is not isolated[34].[V]

2.5 Timeline of the period

Figure 2.1 places the principal works of this report on a calendar. Month placement follows the arXiv identifier or the blog's date and is approximate for items without a date in our notes.

2.6 Why a second look is warranted

The NVIDIA report is a vendor document with limited third-party replication at the time of writing. Its central evidence is one 12B model run with one data blend, one random seed per configuration, and an

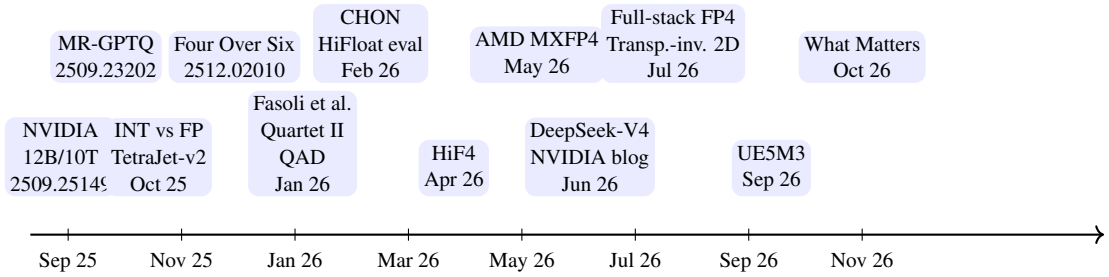


Figure 2.1: Timeline of principal sources (month from arXiv identifier or blog date). Nemotron 3 Super (2026) is omitted for lack of a firm date in our notes.

FP8 baseline whose own recipe is not the DeepSeek-V3 recipe. The research community responded in three ways: attempts to reproduce and compare (Chapter 8 and Chapter 12), attempts to find the cheapest sufficient set of tricks, and attempts to question the format itself (Chapter 14). The next chapters set up the mathematics needed to evaluate those responses.

Chapter 3

The Formats

This chapter defines precisely the number formats used in the rest of the report. Definitions follow the NVIDIA technical report for NVFP4 [3] (its appendix specifies the quantisation procedure), the OCP microscaling paper for MX formats [2], and the FP8 paper for E4M3 [1]. Where we state a bit-level fact not taken from a paper we derive it from the format definition.[D]

3.1 Minifloat formats

A *minifloat* with e exponent bits, m mantissa bits and a sign bit represents, for exponent field $E \in \{0, \dots, 2^e - 1\}$ and mantissa field $M \in \{0, \dots, 2^m - 1\}$ and bias b,

$$v = \begin{cases} (-1)^S \, 2^{E-b} \, (1 + M \, 2^{-m}) & E > 0 \text{ (normal)}, \\ (-1)^S \, 2^{1-b} \, (M \, 2^{-m}) & E = 0 \text{ (subnormal, including zero)}. \end{cases} \tag{3.1}$$

Special values (infinities, NaN) are format dependent. The formats used below are:

Format	Sign	e	m	Bias b	Max finite	Min normal / subnormal	Specials
FP4 E2M1	1	2	1	1	6	1 / 0.5	none
FP8 E4M3 (“FN”)	1	4	3	7	448	2^{-6} / 2^{-9}	one NaN pattern, no Inf
UE5M3	0	5	3	15	$1.75 \cdot 2^{16} = 114688$	2^{-14} / 2^{-17}	(depends on convention)
E8M0	0	8	0	127	2^{127}	2^{-127}	one NaN pattern

Table 3.1: Minifloat formats used in this report.[D] The UE5M3 maximum assumes the E4M3-style convention that the top mantissa pattern of the top exponent is reserved; conventions in the sources may differ and we use the format only qualitatively.

For E4M3 the maximum finite value is $448 = 1.75 \cdot 2^8$ because the exponent field 1111 with mantissa 111 is reserved for NaN; this is the “FN” (finite, NaN) convention of Micikevicius et al. [1]. The smallest positive subnormal is $2^{1-7} \cdot 2^{-3} = 2^{-9}$ and the smallest normal is 2^{-6}. The E8M0 format has no sign and no mantissa: it represents powers of two 2^{E-127}.

3.2 The E2M1 element grid

With $e = 2$, $m = 1$, $b = 1$ the sixteen codes (sign bit, then E, then M) decode as in Table 3.2. Because $E = 0$ is the subnormal exponent, the value $0.5 = 2^0 \cdot (1 \cdot 2^{-1})$ is a subnormal and the positive grid is

Magnitude bits EM	000	001	010	011	100	101	110	111
Value	0	0.5	1	1.5	2	3	4	6
Spacing to next	0.5	0.5	0.5	0.5	1	1	2	—

Table 3.2: The non-negative E2M1 grid.[D]

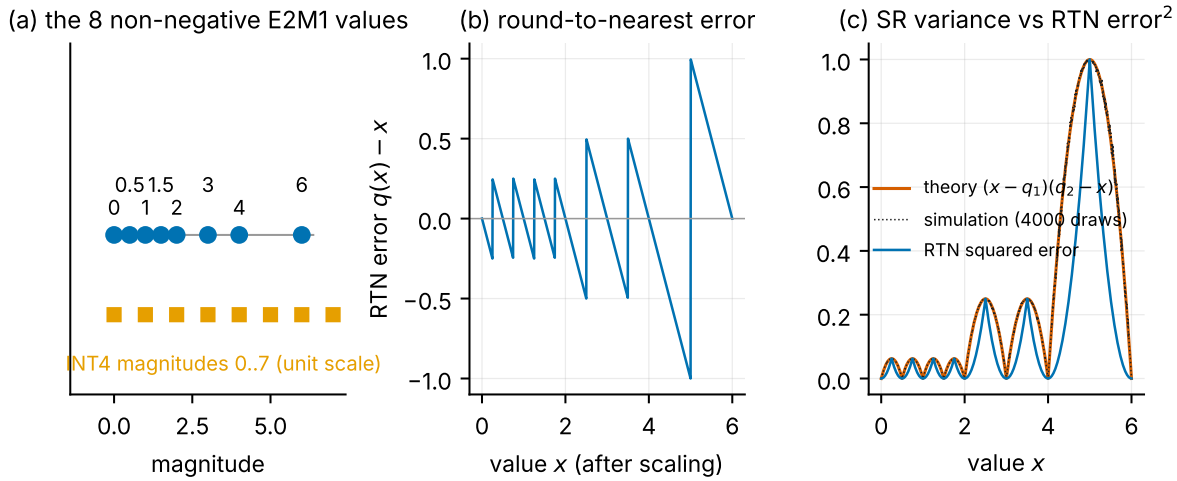


Figure 3.1: (a) The non-negative E2M1 values (top) against the INT4 magnitudes scaled to the same maximum (bottom). (b) Round-to-nearest error $Q(x) - x$ after scaling so the block maximum is 6. (c) Variance of stochastic rounding, $(x - q_1)(q_2 - x)$, from theory and from 4000 simulated draws per point, and the squared RTN error.[E]

$\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$. The format has no infinity or NaN: all sixteen codes are numbers, and "-0" is the sixteenth code.

Three structural facts will be used repeatedly.

1. The spacing is 0.5 on $[0, 2]$, 1 on $[2, 4]$ and 2 on $[4, 6]$. So the grid is *uniform on* $[0, 2]$ and then geometric: it is the union of three binades with 2 points each ($\{1, 1.5\}$, $\{2, 3\}$, $\{4, 6\}$) plus the subnormal pair $\{0, 0.5\}$ continuing the spacing of the first binade.
2. The ratio of largest to smallest non-zero magnitude is $6/0.5 = 12$ (21.6 dB). A symmetric INT4 grid $\{-7, \ldots, 7\}$ has ratio 7, and the grid points are uniformly spaced.
3. The round-to-nearest decision boundaries are the midpoints $0.25, 0.75, 1.25, 1.75, 2.5, 3.5, 5$. Ties are resolved to the code with an even mantissa bit (round-to-nearest-even): $0.25 \to 0$, $0.75 \to 1$, $1.25 \to 1$, $1.75 \to 2$, $2.5 \to 2$, $3.5 \to 4$, $5 \to 4$. Our reference implementation reproduces this and was unit-tested on all midpoints.[E]

Figure 3.1 shows the grid, the quantisation error of round-to-nearest as a function of the scaled value, and the variance of stochastic rounding (derived in Chapter 5).

3.3 Block-scaled formats

A block-scaled format represents a vector $\boldsymbol{x} \in \mathbb{R}^n$, partitioned into n/G contiguous blocks of G elements, by 4-bit codes q_i and one scale $\hat{s}_b$ per block, with reconstruction $\hat{x}_i = \hat{s}_{b(i)}\, q_i$. The cost in bits per element is $4 + \beta/G$ where β is the number of scale bits.

3.3.1 MXFP4

OCP MXFP4: $G = 32$, scale E8M0, $\beta = 8$, so 4.25 bits per element [2].[S] The scale is a power of two, $\hat{s}_b = 2^{k_b}$. The OCP reference conversion chooses $k_b = \lfloor \log_2 \operatorname{amax}(\boldsymbol{x}_b) \rfloor - e^{\text{elem}}_{\max}$, with $e^{\text{elem}}_{\max} = 2$ for E2M1 (because $6 = 1.5 \cdot 2^2$). Elements with $|x_i|/\hat{s}_b > 6$ are saturated to ± 6. We call this variant *floor*. Mishra et al. recommend rounding the scale *up*: $k_b = \lceil \log_2(\operatorname{amax}(\boldsymbol{x}_b)/6) \rceil$ *(ceil)*, which guarantees no saturation [23].[S] We study both in Section 4.5. Alternative choices (round $\log_2$ to nearest) also exist; we include them in our simulations under the name *near*.

3.3.2 NVFP4

NVFP4: $G = 16$, block scale E4M3 ($\beta = 8$), so 4.5 bits per element, plus one FP32 per-tensor scale (a negligible $32/n$ bits per element) [3, 4].[S] The quantisation procedure of the NVIDIA report's appendix is two-level. For a tensor $\boldsymbol{X}$ with global absolute maximum $\operatorname{amax}(\boldsymbol{X})$ and block b:

$$s_{\text{enc}} = \frac{6 \cdot 448}{\operatorname{amax}(\boldsymbol{X})}, \qquad s_{\text{dec}} = \frac{1}{s_{\text{enc}}}, \tag{3.2}$$

$$s_{\text{dec},b} = \frac{\operatorname{amax}(\boldsymbol{X}_b)}{6}, \tag{3.3}$$

$$\hat{s}_b = \text{e4m3}(s_{\text{dec},b} \cdot s_{\text{enc}}) \quad \text{(stored, round to nearest)}, \tag{3.4}$$

$$s_{\text{enc},b} = \frac{1}{\hat{s}_b \cdot s_{\text{dec}}}, \qquad q_i = \text{RTN}(x_i \, s_{\text{enc},b}) \,, \tag{3.5}$$

$$\hat{x}_i = q_i \cdot \hat{s}_b \cdot s_{\text{dec}}. \tag{3.6}$$

The tensor-level factor s_{enc} maps the tensor's largest absolute value to 6×448, so that the *stored* block scale of the block containing the global maximum is exactly 448 (the largest E4M3 value) and every other block's stored scale lies in $(0, 448]$. Equation (3.5) divides by the *rounded* scale so that the element codes are consistent with what the hardware will multiply back. Our reference implementation of (3.2)–(3.6) uses an E4M3 rounding function that we verified bit-identical to `torch.float8_e4m3fn` on a dense set of inputs.[E]

Because $\hat{s}_b \neq s_{\text{dec},b} s_{\text{enc}}$ in general, the block's largest element is mapped to $6\,(s_{\text{dec},b} s_{\text{enc}})/\hat{s}_b$, which is slightly above or below 6. If it is above 6 the element saturates at 6; we quantify this in Section 4.5.

Why two levels. E4M3 has only $2^4 - 1$ usable exponent values, so its positive range is 2^{-9} to 448 (a ratio of $2^{17.8} \approx 229{,}000$ including subnormals, and $2^{14.8} \approx 28{,}700$ for normals only). Real tensors, especially gradients, can have block magnitudes spanning more than that. The FP32 tensor scale removes the common magnitude so that the block scales only have to describe the *relative* magnitude of blocks. This design also explains the weakness analysed in Section 4.7: relative block magnitudes beyond 2^{15}–2^{18} are not representable.[D]

3.3.3 Two-dimensional (16×16) scaling

In the NVIDIA training recipe, weights use 16×16 tiles that share one scale, while activations and gradients use 1×16 blocks [3].[S] A tile's scale is replicated for each of its sixteen rows when passed to the tensor cores. Because $\boldsymbol{W}$ and $\boldsymbol{W}^\top$ share tile structure, the forward quantisation of a weight is identical to the quantisation used in backpropagation through the transposed weight, which is the motivation given for the scheme (Chapter 8). The cost is quantisation error: a tile's maximum is taken over 256 rather than 16 elements, which our simulations quantify (Chapter 13).

3.4 Comparison table

Table 3.3 compares the formats. Peak-throughput rows are vendor figures.

	MXFP4	NVFP4	FP8 (E4M3) per-block	INT4 (typical)
Element	E2M1	E2M1	E4M3	4-bit int
Block size G	32	16 (1D) or 16×16 (2D)	32 (MXFP8)	16–128
Scale format	E8M0	E4M3 + FP32 tensor	E8M0	FP16/FP32/E4M3
Bits per element	4.25	4.5	8.25	$4 + 16/G$
Compression vs FP16	3.76×	3.56×	1.94×	depends
Compression vs FP8 (8 bit)	1.88×	1.78×	—	—
Scale rounding	power of two	$\leq 6.25\%$ rel. error	power of two	—

Table 3.3: Format comparison; compression ratios are derived from bits per element, ignoring the FP32 tensor scale.[D]

NVIDIA's inference blog states the memory reduction as about $3.5\times$ relative to FP16 and about $1.8\times$ relative to FP8, which matches the bit count above.[V] Peak dense tensor-core throughput for NVFP4 on Blackwell Ultra is stated as 15 PFLOPS versus 10 PFLOPS on Blackwell[33][V]; we do not make claims about delivered throughput.

3.5 Software support

The Transformer Engine documentation describes an NVFP4 recipe object (`NVFP4BlockScaling`) with switches to disable the random Hadamard transform, disable two-dimensional weight quantisation and disable stochastic rounding, and notes that NVFP4 training is supported on SM100/SM103 (data-centre Blackwell) and only for a specific (TN) GEMM layout[35].[S] In inference, NVFP4 support exists in vendor toolchains and, according to a 2026 secondary report, in llama.cpp; that report carries a September 2026 correction regarding model size, which we flag as an example of the unreliability of such secondary material[36].[U]

3.6 Dequantisation to FP8 and mixed pipelines

Two properties of the formats simplify conversion. First, every E2M1 value is exactly representable in E4M3 (and in FP16, BF16 and FP32). Second, if a block of 32 elements with a power-of-two scale is held inside a larger FP8 block with its own scale, then FP4→FP8 dequantisation is lossless whenever the ratio of the two scales is a representable shift that does not push any element out of FP8's range. DeepSeek-V4 relies on this property: its FP4 expert weights use 1×32 tiles within the 128×128 blocks used for FP8 weights, and the report states that dequantisation of FP4 to FP8 is lossless under a scale-ratio condition, so FP4 weights can be used with FP8 compute paths[16].[S] The same report says that on current hardware FP4×FP8 matrix multiplications have the same peak FLOPs as FP8×FP8, so the benefit there is memory and bandwidth, not peak arithmetic.[S]

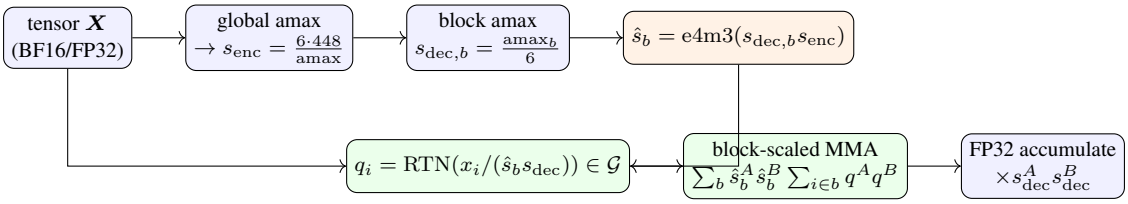


Figure 3.2: NVFP4 quantisation and block-scaled matrix multiplication (schematic; follows the procedure of the NVIDIA report's appendix and the description of the hardware accumulation in the report).[S]

Chapter 4

Quantisation Error of Block-Scaled FP4

This chapter derives how much error the formats of Chapter 3 introduce when they quantise a tensor with round-to-nearest, as a function of the input distribution, block size and scale format. Stochastic rounding is treated in Chapter 5 and rotations in Chapter 6. Throughout, "[E]" results come from our float32 CPU simulations with 2^{21} samples per distribution unless stated otherwise (code in Appendix B); standard errors of the reported relative MSEs are of order 10^{-5} or below, so we report four or five digits only where they are stable.

4.1 Setting and metrics

Let $\boldsymbol{x} \in \mathbb{R}^G$ be one block with i.i.d. entries of a symmetric density p with unit variance. A quantiser $\mathcal{Q}$ returns $\hat{\boldsymbol{x}}$. The *relative MSE* is $\mathrm{MSE} = \mathbb{E}\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|^2 / \mathbb{E}\|\boldsymbol{x}\|^2$ and $\mathrm{QSNR} = -10 \log_{10} \mathrm{MSE}$. These are the metrics used in several of the papers we cite [5, 11] and they are convenient because for unit variance data MSE equals the per-element mean squared error.

Two properties of MSE matter for how to read it. It is an average dominated by large elements (squared error is not weighted by importance to the model), and it is computed on the distribution of values, not on the distribution of their downstream effect. We return to this in Chapter 9 where the connection to loss is made through a second-order expansion. The *crest factor* of a block is $\kappa = \mathrm{amax}(\boldsymbol{x}) / \|\boldsymbol{x}\|_2 \cdot \sqrt{G}$, i.e. the ratio of absolute maximum to root-mean-square; always $1 \le \kappa \le \sqrt{G}$, with $\kappa = \sqrt{G}$ for a block with a single non-zero entry. For Gaussian entries κ is about 2–3 at the block sizes of interest.

4.2 Round-to-nearest onto the E2M1 grid

Let $Q : \mathbb{R} \to \mathcal{G}$ denote round-to-nearest-even onto the E2M1 grid (saturating at ± 6), and $e(z) = z - Q(z)$ its error. From Table 3.2 the error function is a sawtooth: on the interval between neighbouring grid points $g_j < g_{j+1}$ it is linear with slope 1, jumps at the midpoint, and has amplitude $\Delta_j/2$, where $\Delta_j = g_{j+1} - g_j \in \{0.5, 1, 2\}$ (Figure 3.1b).

If z is uniformly distributed within an interval of width Δ_j then $\mathbb{E}[e^2] = \Delta_j^2/12$. The "high-resolution" (bin-wise uniform) approximation to the MSE of a scaled variable z with density p_z is therefore

$$\mathbb{E}[e(z)^2] \;\approx\; \sum_j P_j \, \frac{\Delta_j^2}{12}, \qquad P_j = \mathbb{P}\big(z \in \text{interval around } g_j\big), \tag{4.1}$$

with the lowest bin (around zero) of width 0.5 and the top bin $[5, 6]$ of width 1 (the cell around 6 is

Gaussian					
G	FP4 quad.	FP4 MC	FP4/INT4	INT4 quad.	INT4 MC
4	0.00471	0.00471	1.495	0.00315	0.00315
8	0.00701	0.00701	1.362	0.00515	0.00515
16	0.00886	0.00887	1.221	0.00726	0.00725
32	0.01022	0.01022	1.086	0.00941	0.00941
64	0.01117	0.01118	0.965	0.01158	0.01157
128	0.01185	0.01186	0.862	0.01375	0.01375

Laplace (unit variance)					
G	FP4 quad.	FP4 MC	FP4/INT4	INT4 quad.	INT4 MC
4	0.00371	0.00371	1.017	0.00364	0.00365
8	0.00601	0.00601	0.915	0.00656	0.00657
16	0.00832	0.00832	0.812	0.01024	0.01025
32	0.01051	0.01051	0.717	0.01467	0.01465
64	0.01259	0.01259	0.635	0.01982	0.01982
128	0.01463	0.01463	0.569	0.02569	0.02568

Table 4.1: Expected relative MSE with ideal (unquantised) absmax scale: quadrature of Equation (4.2) versus Monte Carlo (2^{22} samples), and ratio FP4/INT4 (from quadrature).[E]

truncated at the saturation point). This is the sense in which the relative precision of a floating-point grid is nearly constant: in the normal range, relative half-spacing $\Delta_j/(2g)$ lies between $2^{-(m+1)} \cdot \frac{1}{2} = 1/8$ and $2^{-(m+1)} = 1/4$ for $m = 1$. A two-bit exponent and one mantissa bit give a worst-case relative rounding error of 25% on the lower edge of each binade.[D]

Remark 4.1. Equation (4.1) is accurate only when the density of z varies slowly across a cell. For blocks of 16–32 Gaussian values scaled to the maximum, the density varies a lot across the coarse upper cells, and it is the *number* of elements in each cell, not the density, that matters. We therefore use (4.1) only for intuition and rely on the exact calculation of Section 4.3 for numbers.

4.3 Absmax scaling: an exact expected-MSE formula

With an unquantised scale $s = \operatorname{amax}(\boldsymbol{x})/6$ the block maximum maps to ± 6, which is a grid point, so the maximum element is reproduced exactly. This observation turns the expected MSE into a one-dimensional integral.

Proposition 4.2 (Expected MSE of absmax-scaled E2M1)**.** *Let $X_1, \dots, X_G$ be i.i.d. with a symmetric density, let $h(u) = 2p(u)$ and $H(a) = \int_0^a h$ be the density and distribution function of $|X|$, and let $A = \max_i |X_i|$. With scale $s = A/6$ and round-to-nearest, the expected squared error per element is*

$$\mathbb{E}\Big[\tfrac{1}{G}\|\hat{\boldsymbol{X}} - \boldsymbol{X}\|^2\Big] = \frac{G-1}{36}\int_0^\infty a^2\, h(a)\, H(a)^{G-2}\, K(a)\, da, \qquad K(a) = \int_0^a e\Big(\frac{6u}{a}\Big)^2 h(u)\, du. \quad (4.2)$$

The same formula holds for any grid with maximum $Q_{\max}$ (replace 6 by $Q_{\max}$ and e by that grid's error), in particular for INT4 with $Q_{\max} = 7$.

Proof. The density of A is $f_A(a) = G\,h(a)H(a)^{G-1}$. Conditionally on $A = a$ one entry has magnitude a and the other $G-1$ are i.i.d. with magnitude density $h(u)/H(a)$ on $[0, a]$. The maximum entry is scaled to 6 and has zero error. For each other entry the squared error is $s^2 e(6u/a)^2 = (a/6)^2 e(6u/a)^2$, with conditional expectation $(a/6)^2 K(a)/H(a)$. Hence the conditional expected squared error of the block is $(G-1)(a/6)^2 K(a)/H(a)$. Multiplying by $f_A(a)$, integrating and dividing by G gives (4.2). □

Evaluating (4.2) by numerical quadrature on a fine grid reproduces Monte Carlo to three or four significant digits for both Gaussian and Laplace inputs (Table 4.1, Figure 4.1). Because the formula is exact up to quadrature error, agreement validates both the formula and our quantiser implementation.[E]

4.3.1 A decomposition: MSE $= \eta\, \mathbb{E}[A^2]/Q_{\max}^2$

Write the per-element MSE in the form $\mathrm{MSE} = \eta\, \mathbb{E}[A^2]/Q_{\max}^2$, where η is the mean squared error in the normalised units in which the block maximum equals $Q_{\max}$. For a uniform grid (INT4, spacing one

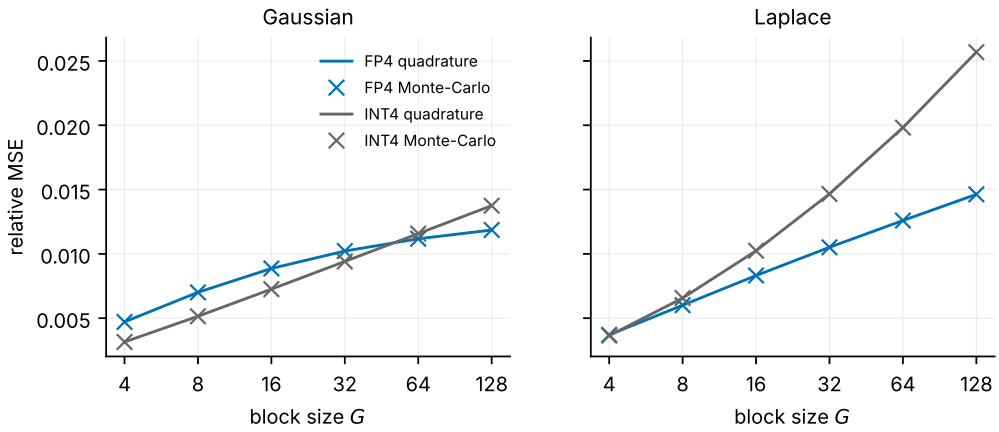


Figure 4.1: Quadrature (lines) against Monte Carlo (markers) for the expected relative MSE of absmax-scaled FP4 and INT4 blocks versus block size G.[E]

G	$\mathbb{E}[A^2]$	η_{FP4}	η_{INT4}	$\eta_{\mathrm{FP4}}/\eta_{\mathrm{INT4}}$
4	2.472	0.0686	0.0625	1.097
8	3.461	0.0729	0.0729	1.000
16	4.558	0.0700	0.0780	0.898
32	5.725	0.0643	0.0805	0.798
64	6.940	0.0580	0.0817	0.710
128	8.196	0.0521	0.0822	0.634

Table 4.2: Gaussian data: normalised error $\eta = \mathrm{MSE}\, Q_{\max}^2/\mathbb{E}[A^2]$ for FP4 ($Q_{\max} = 6$) and INT4 ($Q_{\max} = 7$) with ideal absmax scales. $\mathbb{E}[A^2]$ is the second moment of the block maximum from Monte Carlo.[E]

unit of the normalised scale), error within a cell is uniform with variance $1/12$ to a good approximation, except for the maximum entry which is exact, so $\eta_{\mathrm{INT4}} \approx \frac{G-1}{12G}$. Table 4.2 confirms this to within 1% for $G \geq 8$ using Monte Carlo values of $\mathbb{E}[A^2]$ (e.g. $G = 8$: 0.0729 against $\frac{7}{96} = 0.0729$). This gives the closed form

$$\mathrm{MSE}_{\mathrm{INT4}}(G) \;\approx\; \frac{G-1}{588\,G}\,\mathbb{E}[A_G^2] \quad (\sigma = 1) \qquad \text{[D][E]} \tag{4.3}$$

where $588 = 12 \cdot 49$.

For FP4 the normalised error η_{FP4} is not constant but *decreases* with G (from 0.069 at $G = 4$ to 0.052 at $G = 128$): as G grows, the block maximum becomes larger relative to the bulk, so most elements land in the finer low-magnitude cells where the floating-point grid is relatively more precise. The INT4 grid has no such adaptation. Both forces are visible in the last column of Table 4.2: the ratio $\eta_{\mathrm{FP4}}/\eta_{\mathrm{INT4}}$ falls from 1.10 to 0.63. The MSE ratio is this multiplied by $(Q_{\max}^{\mathrm{FP4}}/Q_{\max}^{\mathrm{INT4}})^{-2} = 49/36 = 1.36$ in favour of INT4, because INT4's largest level is 7 against 6, so for the same block maximum the INT4 step is $6/7$ as large:

$$\frac{\mathrm{MSE}_{\mathrm{FP4}}}{\mathrm{MSE}_{\mathrm{INT4}}} = \frac{49}{36} \cdot \frac{\eta_{\mathrm{FP4}}}{\eta_{\mathrm{INT4}}}. \tag{4.4}$$

This ratio crosses one between $G = 32$ and $G = 64$ for Gaussian data (quadrature: 1.086 at $G = 32$, 0.965 at $G = 64$), and is below one for all $G \geq 8$ on Laplace data (Table 4.1).[E] The crossover location is a statement about a particular distribution (Gaussian) and an idealised scale; it moves with the tail weight, as discussed in Section 4.8.

Scaled magnitude $\lvert z\rvert$	share of elements	share of squared error
zero bin $\lvert z\rvert < 0.25$	0.068	0.022
$[0.25, 2)$	0.432	0.137
$[2, 4)$	0.307	0.351
$[4, 6]$	0.193	0.490

Table 4.3: Share of elements and of total squared error by scaled magnitude, Gaussian input, $G = 16$, ideal scale, round-to-nearest.[E] Total relative MSE 0.00888. The block maximum (scaled to 6) is in the last row but contributes no error.

4.4 Where in the grid the error lives

A natural question is which grid cells contribute most to the error of NVFP4-like quantisation. Table 4.3 decomposes the squared error of Gaussian data, $G = 16$, ideal scale, by the scaled magnitude $z = x/s$ of each element.

Nineteen percent of the elements lie in $[4, 6]$—where the grid spacing is 2—and account for 49% of the squared error, while the 43% of elements in $[0.25, 2)$ contribute 14%. This is the key to understanding a family of recent improvements: the coarse top cell of the E2M1 grid is where most of the loss occurs. Section 4.9 shows that Four Over Six exploits exactly this.

4.5 Rounding the scale: E4M3 and power-of-two

So far the scale was exact. In NVFP4 it is rounded to E4M3 (3.4); in MXFP4 to a power of two.

4.5.1 E4M3 block scales: a first-order penalty model

Let the stored scale be $\hat{s} = s(1 + \delta)$ with $s = A/6$. With three explicit mantissa bits, adjacent normal E4M3 values with exponent E are 2^{E-3} apart, so for a scale of mantissa $m \in [1, 2)$ the relative half-spacing is $h(m) = 2^{-4}/m \in (2^{-5}, 2^{-4}]$ and $|\delta| \le h(m)$.

The block maximum is quantised as $q = \mathrm{RTN}(\mathrm{amax}\,/\hat{s}) = \mathrm{RTN}(6/(1 + \delta))$. Since $|\delta| \le 1/16$, $6/(1 + \delta) \in [5.65, 6.4]$: it rounds, or saturates, to 6 in all cases (the nearest-neighbour boundary below 6 is 5). The reconstruction is $6\hat{s} = A(1 + \delta)$, so the maximum entry now carries a relative error exactly δ, i.e. a squared error $A^2\delta^2$. The other entries see a grid whose scale is off by $1 + \delta$; since δ has zero mean, the first-order effect on their expected error vanishes. This yields the model

$$\Delta\mathrm{MSE}_{\mathrm{E4M3}} \;\approx\; \frac{1}{G}\mathbb{E}\big[A^2\,\delta^2\big] \;\approx\; \frac{\mathbb{E}[A^2]}{G}\,\mathbb{E}[\delta^2], \qquad \mathbb{E}[\delta^2] = \mathbb{E}\Big[\frac{h(m)^2}{3}\Big]. \tag{4.5}$$

If the mantissa of the scale is log-uniformly distributed on $[1, 2)$ (as is the case for the leading digits of quantities spread over many binades), then $\mathbb{E}[1/m^2] = \frac{3}{8\ln 2}$ and

$$\mathbb{E}[\delta^2] = \frac{2^{-8}}{3} \cdot \frac{3}{8\ln 2} = \frac{2^{-8}}{8\ln 2} \approx 7.04 \times 10^{-4}, \tag{4.6}$$

a root-mean-square relative scale error of 2.65%.[D] The same quantity is directly measured as the relative error of the block-maximum element: it is 6.9×10^{-4} (Gaussian) to 7.0×10^{-4} (all other distributions, Table 4.6) for NVFP4 with RTN.[E]

Table 4.4 compares the model with the measured excess MSE of NVFP4 over the same block size with an ideal FP32 scale. The model is accurate to a few percent at $G \le 8$, within 12% at $G = 16$

G	$\mathbb{E}[A^2]$	predicted ΔMSE	measured ΔMSE	ratio
4	2.472	4.35e-04	4.25e-04	0.98
8	3.461	3.05e-04	2.89e-04	0.95
16	4.558	2.01e-04	1.76e-04	0.88
32	5.725	1.26e-04	1.01e-04	0.80
64	6.940	7.64e-05	5.37e-05	0.70
128	8.196	4.51e-05	3.40e-05	0.75

Table 4.4: Gaussian data: excess relative MSE of NVFP4 (E4M3 scale) over ideal-scale FP4, model (4.5) versus simulation.[D][E]

and over-predicts by 20–30% for $G \geq 32$; we did not isolate the second-order effects responsible. The practical content is that the E4M3 scale costs about 2% of the MSE at $G = 16$ and that this penalty falls as $1/G$ times growth of $\mathbb{E}[A^2]$.[E]

4.5.2 Power-of-two scales (MXFP4)

For E8M0 the scale is $\hat{s} = 2^k$. Write $\rho = \hat{s}/s$ for the ratio of stored to ideal scale.

Round up (ceil). $\hat{s} = 2^{\lceil \log_2 s \rceil} \geq s$, so $\rho = 2^u$ with $u = \lceil \log_2 s \rceil - \log_2 s \in [0, 1)$ and the block maximum maps to $6/\rho \in (3, 6]$. Nothing saturates, but the grid is coarser than necessary by ρ. If u were uniform, $\mathbb{E}[\rho^2] = \int_0^1 4^u \, du = \frac{3}{2 \ln 2} = 2.16$. In the high-resolution regime the MSE scales as ρ^2, so this is an upper estimate of the penalty relative to an ideal scale. The worst case, found by Mishra et al., is $A/\hat{s}$ barely above 3: the top binade of the grid ($\pm 4, \pm 6$) is then never used and only $\log_2(3/0.5) = 2.58$ of the $\log_2(6/0.5) = 3.58$ binades are exercised, i.e. the block loses about one bit of dynamic range [23].[S][D]

Round down (floor, the OCP reference). $\hat{s} = 2^{\lfloor \log_2 A \rfloor - 2}$ gives $A/\hat{s} \in [4, 8)$. Writing $f = \log_2 A - \lfloor \log_2 A \rfloor$, the block maximum exceeds 6 when $2^f \cdot 4 > 6$, i.e. $f > \log_2 1.5 \approx 0.585$: if f were uniform, 41.5% of blocks saturate their largest element, by up to $2\hat{s}$ (up to 25% of its value).[D]

Round log to nearest. $\hat{s} = 2^{\mathrm{round}(\log_2 s)}$ splits the difference: $\rho \in [2^{-1/2}, 2^{1/2}]$, so both saturation (when $\rho < 1$) and coarse grids (when $\rho > 1$) occur.

Table 4.5 reports the outcome on six synthetic distributions and Table 4.6 the errors at the block maximum. A few observations follow.

1. **NVFP4 beats MXFP4 by a wide margin in MSE.** On Gaussian data NVFP4 ($G = 16$) has relative MSE 0.00904, MXFP4 0.01329 (ceil) or 0.01320 (floor): a 32% reduction (1.67 dB). On Laplace data the gap is wider (0.0086 vs 0.0175 for ceil; 0.0159 for floor).[E]
2. **Part of the gap is scale format, part is block size.** NVFP4 at $G = 32$ (still with an E4M3 scale) has MSE 0.01032, so moving from E8M0 to E4M3 at fixed $G = 32$ cuts the MSE from 0.0133 to 0.0103 (22%), and halving the block size cuts it further to 0.0090 (12%). At equal 4.25 bits per element, E4M3 scales at $G = 32$ are about 1.1 dB better than E8M0 scales.[E]
3. **Floor and ceil differ little in MSE but a lot at the top.** On Gaussian data the MSE of "floor" is within 1% of "ceil", yet the error of the block-maximum element is 50% larger with floor (0.018 vs 0.012): saturation is concentrated precisely on the largest values. The outlier-channel distribution

Format	Gauss.	Laplace	t_5	t_3	Row-scaled	Outl. chan.
Excess kurtosis	0.0	3.0	4.9	63.0	84.0	294.7
FP4, G=16, ideal FP32 scale	0.00886	0.00832	0.00827	0.00793	0.00889	0.00605
FP4, G=32, ideal FP32 scale	0.01021	0.01051	0.01041	0.01132	0.01023	0.01144
INT4, G=16, FP32 scale	0.00726	0.01024	0.01014	0.01287	0.00728	0.00894
INT4, G=32, FP32 scale	0.00941	0.01469	0.01500	0.02107	0.00939	0.01776
NVINT4 (G=16, E4M3 scale)	0.00744	0.01052	0.01041	0.01323	0.00748	0.00958
MXINT4 (G=32, E8M0 up)	0.02075	0.03179	0.03296	0.04334	0.02038	0.02548
MXFP4 floor (OCP rule)	0.01320	0.01587	0.01587	0.02019	0.01320	0.02783
MXFP4 ceil (round scale up)	0.01329	0.01753	0.01750	0.02280	0.01369	0.02554
MXFP4 nearest-log scale	0.01474	0.01742	0.01748	0.02231	0.01438	0.03257
NVFP4 (G=16, E4M3)	0.00904	0.00860	0.00854	0.00830	0.00907	0.00668
NVFP4 G=32 (E4M3)	0.01032	0.01070	0.01060	0.01160	0.01038	0.01206
NVFP4 + Four Over Six (MSE)	0.00756	0.00761	0.00752	0.00763	0.00757	0.00634
NVFP4 with SR	0.01892	0.01827	0.01819	0.01803	0.01880	0.01755
NVFP4 with SR, 16/17 headroom	0.02355	0.02481	0.02442	0.02667	0.02347	0.03168

Table 4.5: Relative MSE of 4-bit block formats on synthetic data, round-to-nearest unless noted (2^{21} samples). "Row-scaled": Gaussian rows of length 1024 with log-normal row scale ($\sigma_{\log} = 1$); "outlier channels": $1/128$ of the columns have $30\times$ the standard deviation. Block size is 16 along each row unless the label gives 32.[E]

	Gaussian		Laplace	
Format	rel. err. of block max	zeroed frac.	rel. err. of block max	zeroed frac.
MXFP4 floor (OCP rule)	0.01838	0.088	0.01693	0.165
MXFP4 ceil	0.01216	0.107	0.01092	0.211
MXFP4 nearest-log	0.02624	0.083	0.02311	0.156
NVFP4 (G=16)	0.00069	0.068	0.00070	0.128
NVFP4 + Four Over Six	0.00068	0.081	0.00068	0.144
NVFP4 SR	0.00384	0.068	0.00385	0.125
NVFP4 SR, 16/17 headroom	0.01764	0.072	0.01749	0.132

Table 4.6: Relative squared error of the block-maximum element, and fraction of non-zero inputs quantised to zero.[E]

reverses the MSE ranking (0.0278 floor vs 0.0255 ceil). The "nearest" rule is the worst of the three on Gaussian data (0.0147).[E]

4. **The block-maximum error of NVFP4 is tiny** (7×10^{-4}), consistent with (4.5).[E]

Does MXFP4 depend on the absolute scale of the data? Because E8M0 scales are quantised on a logarithmic grid with spacing of one octave, the position of $\log_2 A$ within an octave could in principle modulate the error. We scaled Gaussian data by $2^{k/12}$, $k = 0, \ldots, 12$, and recomputed the MSE (2^{20} samples each). Over one octave the ceil rule varies between 0.01336 and 0.01415 ($\pm 3\%$ about its mean), the floor rule between 0.01311 and 0.01445, and the nearest rule between 0.0140 and 0.0166; NVFP4 is flat at 0.00903–0.00906.[E] The modulation is therefore small for Gaussian data at $G = 32$ because $\log_2 A$ itself fluctuates by more than a tenth of an octave from block to block, which averages the phase; it could be larger for data whose block maxima are nearly deterministic (for instance post-normalisation activations with bounded range) and we did not test this.[U]

4.6 Block size and bits per element

Table 4.7 sweeps the block size from 4 to 128 for Gaussian and Laplace data. Figure 4.2 plots them.

The following consequences are useful. The MSE of every format increases with G, as $\mathbb{E}[A_G^2]$ grows.

Gaussian

G	FP4 ideal	INT4 ideal	NVFP4 (E4M3)	MXFP4 ceil	MXFP4 floor
4	0.00471	0.00315	0.00513	0.01155	0.01537
8	0.00701	0.00514	0.00730	0.01212	0.01454
16	0.00886	0.00725	0.00904	0.01272	0.01385
32	0.01024	0.00938	0.01034	0.01330	0.01322
64	0.01120	0.01153	0.01125	0.01414	0.01278
128	0.01187	0.01369	0.01190	0.01557	0.01268

Laplace

G	FP4 ideal	INT4 ideal	NVFP4 (E4M3)	MXFP4 ceil	MXFP4 floor
4	0.00371	0.00365	0.00421	0.01214	0.01609
8	0.00601	0.00657	0.00640	0.01342	0.01569
16	0.00832	0.01027	0.00860	0.01519	0.01559
32	0.01053	0.01472	0.01071	0.01758	0.01589
64	0.01261	0.01989	0.01272	0.02073	0.01676
128	0.01466	0.02582	0.01472	0.02425	0.01846

Table 4.7: Relative MSE versus block size G (Monte Carlo). "ideal": FP32 scale; NVFP4: E4M3 scale with FP32 tensor scale; MXFP4: E8M0 scale, ceil or floor rule.[E]

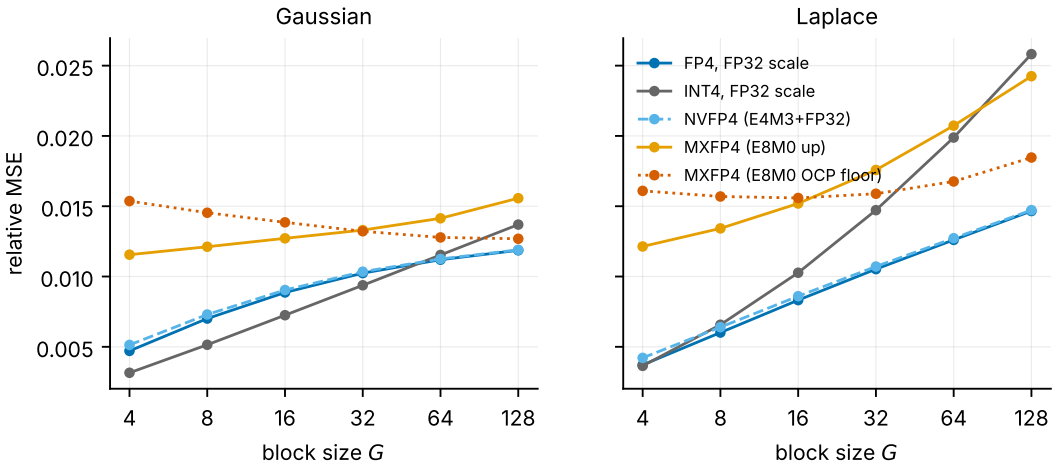


Figure 4.2: Relative MSE versus block size for ideal-scale FP4 and INT4, E4M3-scaled FP4, and E8M0-scaled FP4 (ceil and OCP floor).[E]

The E4M3 penalty relative to an ideal scale vanishes as G grows (Table 4.4). Since the scale costs $8/G$ bits per element, the choice between $G = 16$ and $G = 32$ is a choice between 4.5 and 4.25 bits: the extra 0.25 bits reduce the Gaussian MSE by 12% (0.58 dB). Under the classical high-rate rule of 6.02 dB per bit the same 0.25 bits would buy 1.5 dB, so the finer scale is a less efficient use of bits than a longer mantissa, although mantissa bits are not available without changing the tensor-core datapath.[D] This heuristic comparison ignores that the high-rate rule applies to uniform quantisers.

4.7 The dynamic range of the two-level scale

The NVFP4 block scale lives in E4M3 and is expressed relative to the tensor maximum. A block with maximum A_b in a tensor with global maximum A_x has, from (3.2)–(3.4), the value presented to the E4M3 rounder

$$v_b = s_{\mathrm{dec},b}\, s_{\mathrm{enc}} = \frac{A_b}{6} \cdot \frac{6 \cdot 448}{A_x} = 448\, \frac{A_b}{A_x}. \tag{4.7}$$

Three regimes follow from the E4M3 format: for $v_b \geq 2^{-6}$ (normal range) the relative scale error is bounded by 2^{-4}; for $2^{-10} < v_b < 2^{-6}$ the scale is subnormal and has absolute spacing 2^{-9} so its relative error grows as $2^{-10}/v_b$; and for $v_b \leq 2^{-10}$ the scale rounds to zero and the whole block is flushed. In terms of block magnitude relative to the largest block in the tensor the boundaries are

$$\frac{A_b}{A_x} \approx \frac{2^{-6}}{448} = 3.5 \times 10^{-5} \quad \text{(loss of precision)}, \qquad \frac{2^{-10}}{448} = 2.2 \times 10^{-6} \quad \text{(flush)}. \tag{4.8}$$

These are -89 dB and -113 dB. An E8M0 scale spans $2^{\pm 127}$ by comparison, and UE5M3 extends the E4M3 lower limit by 2^{-8}.[D]

G	blocks flushed	energy flushed (prediction)	measured rest-of-tensor MSE	MSE without outlier
4	0.887	0.739	0.744	0.0047
8	0.788	0.656	0.663	0.0070
16	0.621	0.517	0.528	0.0088
32	0.387	0.322	0.336	0.0102
64	0.150	0.125	0.142	0.0111

Table 4.8: One outlier of size $10^6\sigma$ in a Gaussian tensor: predicted flushed-energy fraction from (4.7) versus measured MSE of the remaining tensor (NVFP4, E4M3 scales).[D][E]

	$G = 4$			$G = 16$		
A/σ	ideal	E4M3	UE5M3	ideal	E4M3	UE5M3
10^2	0.0047	0.0051	0.0051	0.0088	0.0090	0.0090
10^3	0.0047	0.0051	0.0051	0.0088	0.0090	0.0090
10^4	0.0047	0.0051	0.0051	0.0088	0.0090	0.0090
10^5	0.0047	0.0086	0.0051	0.0088	0.0098	0.0090
10^6	0.0047	0.7440	0.0051	0.0088	0.5280	0.0090

Table 4.9: Relative MSE of the rest of the tensor with one outlier of magnitude $A\sigma$, E4M3 versus UE5M3 block scales.[E]

One outlier. Take a tensor of i.i.d. $\mathcal{N}(0, \sigma^2)$ entries with a single entry replaced by $A\sigma$. The outlier sets $A_x = A\sigma$ and other blocks have maxima near $\mathbb{E}[A_G]\sigma$, where $\mathbb{E}[A_G] \approx 1.47$ ($G = 4$), 2.08 ($G = 16$), 2.60 ($G = 64$). Precision is lost when $448\mathbb{E}[A_G]/A \lesssim 2^{-6}$, i.e. $A \gtrsim 28{,}700\,\mathbb{E}[A_G]$: 4×10^4 to 7.5×10^4 over this range of G. Blocks flush when $A_b/(A\sigma) \leq 2^{-10}/448$, i.e. when $A_b \leq A\sigma/458{,}752$; for $A = 10^6$ this is $A_b \leq 2.18\sigma$, which a fraction $\mathbb{P}(|X| < 2.18\sigma)^G$ of G-blocks satisfy.[D]

Table 4.8 compares the energy fraction in flushed blocks, computed this way from simulated blocks, to the measured relative MSE of the rest of the tensor from the actual quantiser. For $A = 10^6\sigma$ the prediction ($0.74, 0.52, 0.13$ for $G = 4, 16, 64$) lies within 0.02 of the measurement ($0.74, 0.53, 0.14$); at $A = 10^5\sigma$ no block is flushed but the subnormal scales raise the MSE of $G = 4$ by about 80% (0.0086 against 0.0047, Table 4.9), as predicted from the loss-of-precision threshold. The UE5M3 scale is immune up to the largest tested outlier. Full table in Table 4.9 and Figure 4.3.[E]

This is a worst-case scenario in magnitude: no trained tensor is known to us to contain a single entry 10^5 times the standard deviation of the rest. The relevant empirical question is how large the *block-to-block* dynamic range of real tensors is, which is the subject of the literature on the "finer is not better" anomaly [13] and the Graphcore UE5M3 report [14]. The former reports, for microscaling formats, that reducing the block size below some value increases rather than decreases error for certain LLM tensors, and traces this to the limited range of the FP8 scale; the latter proposes an unsigned E5M3 scale and reports stable pretraining.[S] (See Chapter 14 for details and our caveats.) Our calculation shows that the cause is plausible: smaller G lowers $\mathbb{E}[A_G]$, so more blocks fall below the loss-of-precision threshold.

4.8 INT4 versus FP4 elements

Chen et al. compare integer and floating-point microscaling formats and find that the better element type depends on the crest factor of the block: INT wins when the crest factor is small and FP when it is large, so that with a Hadamard rotation (which lowers the crest factor) NVINT4 can beat NVFP4 [11].[S] Our results are in line with that and give a mechanism. For INT4 the normalised error is nearly constant

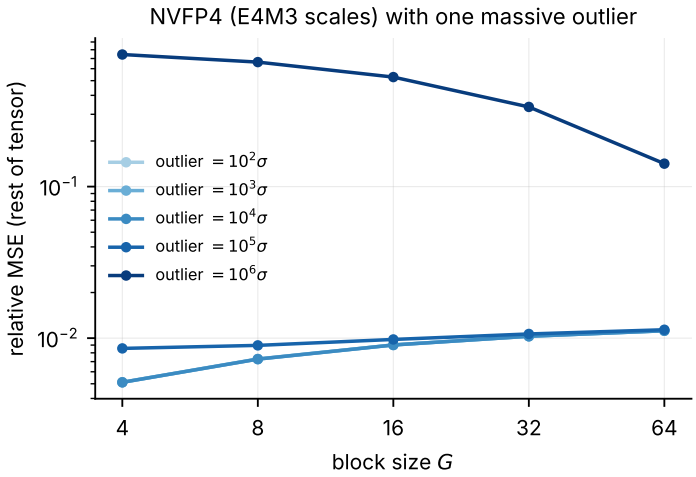


Figure 4.3: Rest-of-tensor relative MSE for NVFP4 (E4M3 scale) with a single massive outlier, versus block size.[E]

($\eta \approx \frac{G-1}{12G}$) so $\mathrm{MSE}_{\mathrm{INT4}}$ is determined by $\mathbb{E}[A^2]$, i.e. by the crest factor: (4.3). For FP4 η *decreases* with G, because heavier tails push the bulk of the block into the finer lower cells; so FP4's error depends on the crest factor less strongly than INT4's.

1. On Gaussian data with ideal scales INT4 beats FP4 at $G \leq 32$ and loses at $G \geq 64$ (quadrature crossover between 32 and 64, Table 4.1).[E]
2. On Laplace data FP4 is at least as good as INT4 for $G \geq 8$ and the gap widens with G (at $G = 32$: 0.0105 against 0.0147).[E]
3. In practice: NVINT4 (INT4 with E4M3 scale, $G = 16$) has MSE 0.00744 against 0.00904 for NVFP4 on Gaussian data, but 0.0105 against 0.0086 on Laplace, 0.0132 against 0.0083 on Student-t_3, and 0.0096 against 0.0067 on the outlier-channel distribution.[E]

Which of these synthetic distributions resembles a real tensor is an empirical matter. Weight matrices of trained transformers are often close to Gaussian within a block but contain structured outliers; activations are heavy-tailed. Our real-weight experiment (Chapter 13) shows that on SmolLM2-135M weights NVINT4 also has lower MSE than NVFP4 (0.00794 against 0.00896), i.e. the weights behave closer to the Gaussian case.[E]

4.9 Four Over Six: choosing the scale by error

Table 4.3 shows that nearly half of the squared error comes from the $[4, 6]$ cell. Cook et al. ("Four Over Six", 4/6) observe that scaling a block so that its maximum maps to 4 rather than 6 changes which cells are used. With maximum 4 the grid $\{0, 0.5, 1, 1.5, 2, 3, 4\}$ has spacing 0.5 up to 2 and 1 above, so the relative spacing near the block maximum is 0.25 (spacing 1 over maximum 4) instead of 0.33 ($2/6$) and the lowest cells are coarser (relative spacing 0.125 instead of 0.083) [7]. For each block they quantise with both candidate scales and keep the one with lower error (MSE for post-training quantisation, MAE for pretraining in their implementation), at the cost of a per-block selection. Because the "4" option uses a scale $1.5\times$ larger, the tensor scale uses 256 instead of 448 so that the largest stored block scale ($256 \cdot 1.5 = 384$) stays within E4M3's 448.[S][D]

In our simulation the per-block MSE selection lowers the Gaussian relative MSE of NVFP4 from 0.00904 to 0.00756 (16%) and from 0.00860 to 0.00761 on Laplace data (Table 4.5), matching the

values 9.0×10^{-3} and 7.6×10^{-3} that the Quartet II paper reports for RTN and RTN+4/6 on Gaussian data.[E][S] With 4/6 the fraction of non-zero inputs that are quantised to zero rises from 6.8% to 8.1% (Gaussian), because the coarser low cells swallow more small values. 4/6 does not remove the scale-range problem: with the tensor scale based on 256 it flushes sooner, and in the one-outlier test with $A = 10^6\sigma$ its MSE at $G = 16$ is 0.78 against 0.53 for plain E4M3 scales (Table 4.9 data, $G = 16$).[E]

4.10 Summary of the chapter

- The E2M1 grid has about $12\times$ dynamic range, uniform spacing up to 2, and a coarse top cell that accounts for about half of the squared error after absmax scaling (Gaussian, $G = 16$).[E]
- For absmax scaling the expected MSE is a one-dimensional integral (Equation (4.2)); INT4's MSE is $\approx \frac{G-1}{588G}\mathbb{E}[A_G^2]$.[D]
- At $G = 16$, E4M3 scale rounding costs about 2% of MSE and is explained by a first-order model; power-of-two scales cost about 30% at $G = 32$.[D][E]
- The E4M3 range is $2^{14.8}$ (normals) to $2^{17.8}$ (with subnormals) relative to the largest block; outliers beyond $\sim 4 \times 10^4\sigma$ lose scale precision and beyond $\sim 10^6\sigma$ flush blocks.[D][E]
- Which element type is best (INT4 or FP4) depends on the distribution and block size; for Gaussian blocks of 16 INT4 is better in MSE.[E]

Chapter 5

Stochastic Rounding and Gradient Bias

Round-to-nearest minimises the error of a single quantisation but is biased in the sense that $\mathbb{E}[\mathrm{RTN}(x)] = \mathrm{RTN}(x) \neq x$ for a fixed x, with systematic errors that can be correlated across steps. Stochastic rounding (SR) trades extra variance for unbiasedness. This chapter derives the properties of SR on the E2M1 grid, shows how bias in gradients enters an SGD convergence bound, analyses the headroom required for unbiased SR with block scales, and reviews the more recent MS-EDEN estimator of Quartet II.

5.1 Stochastic rounding on the E2M1 grid

For a scaled value $z \in [-6, 6]$ with neighbouring grid points $g_j \leq z \leq g_{j+1}$ (spacing $\Delta = g_{j+1} - g_j$), stochastic rounding returns

$$\mathrm{SR}(z) = \begin{cases} g_{j+1} & \text{with probability } p = \dfrac{z - g_j}{\Delta}, \\ g_j & \text{with probability } 1 - p. \end{cases} \tag{5.1}$$

Proposition 5.1 (Unbiasedness and variance of SR)**.** *For $z \in [g_j, g_{j+1}]$: $\mathbb{E}[\mathrm{SR}(z)] = z$ and*

$$\mathrm{Var}[\mathrm{SR}(z)] = (z - g_j)(g_{j+1} - z) \leq \frac{\Delta^2}{4}. \tag{5.2}$$

If z is uniformly distributed on the cell, the average variance is $\Delta^2/6$, twice the average squared error $\Delta^2/12$ of round-to-nearest.

Proof. $\mathbb{E}[\mathrm{SR}(z)] = g_j(1-p) + g_{j+1}p = g_j + p\Delta = z$. The variance of a two-point distribution on $\{g_j, g_{j+1}\}$ with success probability p is $p(1-p)\Delta^2 = (z-g_j)(g_{j+1}-z)$, maximal at the midpoint with value $\Delta^2/4$. With $u = (z - g_j)/\Delta \sim U[0, 1]$, $\mathbb{E}[u(1-u)]\Delta^2 = \Delta^2/6$, while the squared RTN error is $\min(u, 1-u)^2\Delta^2$ with mean $\Delta^2/12$. □

We verified the variance formula by simulation (4000 draws at each of 1201 inputs on $[0, 6]$: the maximum deviation between empirical and theoretical variance is 0.04 and the maximum absolute bias 0.043 in grid units; both are of the size expected from Monte-Carlo noise, since the standard error of the mean at the widest cell is $2\sqrt{0.25/4000} = 0.016$ and the maximum is taken over 1201 points) and the factor of two: the variance averaged over a uniform input on the grid divided by the average RTN squared error is 1.9999.[E] In an NVFP4 quantiser the factor is not exactly two because the block maximum is on the grid (zero variance) and E4M3 scale rounding adds error to both: SR gives relative MSE 0.0189 against 0.0090 for RTN ($2.09\times$) on Gaussian data (Table 4.5).[E]

SR draws averaged n	no headroom	16/17 headroom
1	0.0516	0.0581
4	0.0263	0.0285
16	0.0146	0.0143
64	0.0101	0.0071
256	0.0082	0.0036
1024	0.0077	0.0018
single RTN product	0.0365	

Table 5.1: Relative Frobenius error of the average of n independent SR-quantised NVFP4 products $\boldsymbol{AB}$ (64×1024 by 1024×64). Without headroom, the error stops improving near 0.0077 because clipping biases the estimator; with $16/17$ headroom it continues to fall as $1/\sqrt{n}$ (down to 0.0018 at $n = 1024$).[E]

Because the SR error is pure variance, it *averages out* over independent draws, while RTN's does not. Table 5.1 shows a fixed matrix product $\boldsymbol{AB}$ ($K = 1024$) estimated by averaging n independent SR-quantised products: the relative error decreases as $1/\sqrt{n}$ towards zero provided the quantiser does not clip, and plateaus if it does.

5.2 Headroom: when is block-scaled SR actually unbiased?

Proposition 5.1 requires $|z| \le 6$. In a block-scaled format $z_i = x_i/\hat{s}_b$ and, with the scale chosen as $A_b/6$ and rounded to E4M3, the largest element can exceed 6 when the scale rounds down. It is then clipped to 6 and $\mathbb{E}[\hat{x}] = 6\hat{s} < x$: the estimator is biased on exactly the largest elements. A scale rounded to E4M3 satisfies $\hat{s} \ge \frac{16}{17}v$ where v is the value presented for rounding (the worst case is a tie at $v = 2^E(1 + 2^{-4})$ rounding down to 2^E; for three mantissa bits the largest downward relative change is $1/17$). Hence presenting the scale as $v = \frac{17}{16} \cdot A_b/6$, i.e. dividing by $6 \cdot \frac{16}{17}$, guarantees $\hat{s} \ge A_b/6$ and no clipping. This is the factor $16/17$ in the Quartet II description of unbiased NVFP4 stochastic rounding, which for MXFP4 is the factor $3/4$ already proposed by Tseng et al. [27].[S][D]

The price of the headroom is a coarser grid: the scale grows by a factor between $17/16$ and about $(17/16)^2$, and the block maximum no longer lies on a grid point so it is rounded stochastically. In our simulation the relative MSE of Gaussian NVFP4 with SR rises from 0.0189 to 0.0236 ($+25\%$) and the error of the block-maximum element from 0.0038 to 0.0176 (Table 4.6). The value 23.5×10^{-3} for SR in Table 1 of the Quartet II paper [5] agrees with our 0.0236 (and with 0.0231 in a second run with 2^{20} samples), which suggests that their SR includes this headroom.[E][S]

5.3 Why bias matters: a convergence bound

Suppose a gradient estimator g_t satisfies $\mathbb{E}[g_t \mid x_t] = \nabla f(x_t) + \beta_t$ with $\|\beta_t\| \le \beta$ and $\mathbb{E}\|g_t - \mathbb{E}[g_t \mid x_t]\|^2 \le \sigma^2$, and f is L-smooth and bounded below by f^*.

Proposition 5.2 (SGD with biased gradients)**.** *SGD with constant step $\eta \le 1/(4L)$ satisfies*

$$\frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}\|\nabla f(x_t)\|^2 \;\le\; \frac{4\,(f(x_0) - f^*)}{\eta T} + 3\beta^2 + 2L\eta\sigma^2. \tag{5.3}$$

Proof. By L-smoothness, $f(x_{t+1}) \le f(x_t) - \eta\langle\nabla f(x_t), g_t\rangle + \frac{L\eta^2}{2}\|g_t\|^2$. Taking conditional expectations, $\mathbb{E}\langle\nabla f, g_t\rangle = \|\nabla f\|^2 + \langle\nabla f, \beta_t\rangle \ge \frac{1}{2}\|\nabla f\|^2 - \frac{1}{2}\|\beta_t\|^2$, and $\mathbb{E}\|g_t\|^2 \le 2\|\nabla f\|^2 + 2\|\beta_t\|^2 + \sigma^2$.

Gradient noise (grid units)	true mean	RTN mean	SR mean
0.0	0.120	0.000	0.119
0.05	0.120	0.019	0.119
0.3	0.120	0.120	0.119

Table 5.2: Dead-zone toy: average over 15 small gradient components of the time-averaged quantised value. The true mean is 0.120.[E]

Therefore $\mathbb{E}f(x_{t+1}) \le \mathbb{E}f(x_t) - \eta(\frac{1}{2} - L\eta)\mathbb{E}\|\nabla f\|^2 + \eta(\frac{1}{2} + L\eta)\beta^2 + \frac{L\eta^2\sigma^2}{2}$. With $\eta \le 1/(4L)$, $\frac{1}{2} - L\eta \ge \frac{1}{4}$ and $\frac{1}{2} + L\eta \le \frac{3}{4}$. Summing over t and rearranging gives (5.3). □

The bound separates three effects. The first term vanishes with T. The third, from variance, is proportional to the learning rate and disappears when the learning rate is decayed. The middle term does *not* depend on η or T: bias creates an irreducible floor on the gradient norm that cannot be removed by training longer or by annealing. Quantisation variance σ_q^2 adds to σ^2 and is controlled by the learning rate; quantisation bias adds to β and is not. This is the formal reason that every FP4 training recipe in the literature uses stochastic rounding (or a theoretically unbiased substitute) for gradient tensors [3, 5, 27, 31].[D][S] It is also consistent with the empirical threshold found by Chmiel et al.: quantised training becomes less effective when the gradient norm falls to about $\sqrt{3}$ times the quantisation noise, i.e. when the $2L\eta\sigma_q^2$ term is no longer small relative to $\|\nabla f\|^2$.[S][U] (We have not verified how exactly the two constants relate; this is an interpretation.)

Remark 5.3 (Bound is worst-case). The bound controls the worst case over bias vectors of norm β. Real RTN bias is not adversarial: it is a deterministic function of the input and, for zero-mean symmetric inputs, tends to be uncorrelated with the gradient direction. The empirical question is how large β is in practice, and the literature disagrees on how much it matters (Section 5.7).

5.4 RTN bias in practice: the dead zone

Round-to-nearest maps every scaled value with $|z| < 0.25$ to zero. In a block whose maximum is much larger than most entries—gradients are typically heavy-tailed—a large fraction of the block is mapped to zero. Even if the zeroed components are consistent across steps, they are *never* applied.

We built a minimal example (Table 5.2). A block of 16 gradient values has one entry fixed at the grid maximum 6 (so the scale is 1) and fifteen small entries between 0.05 and 0.2, i.e. all inside the zero cell. We add zero-mean Gaussian noise of standard deviation ν (in grid units) to the small entries at each of 20,000 steps and average the quantised values. With no noise, RTN outputs exactly 0 for all small entries, while SR's average is 0.119 against the true mean 0.120. As the noise grows to the width of the cell, RTN regains the mean through "dithering" (0.1196 at $\nu = 0.3$): the noise itself randomises the rounding.[E]

This yields two cautionary points. First, the benefit of SR over RTN depends on the signal-to-noise ratio of the gradient. When minibatch noise is large relative to the cell width, RTN is effectively dithered and its bias is small. When the noise is small—late in training, at low learning rates, with large batches—RTN's bias is not hidden. Second, the effect is localised: it matters for components that are small *relative to the block maximum*, which depends on the block size and on outliers. The Nemotron 3 Super report observed a related phenomenon in a real run: the fraction of weight-gradient entries that are exactly zero reached 7% of parameters, about three times higher with NVFP4 than with BF16, and it attributes the excess to underflow in the quantised gradient path [15].[S] (Their remedies were limited to keeping sensitive layers in higher precision; we found no evidence that zero gradients caused measurable

downstream loss, and the MXFP8 healing phase gave no downstream gain in that report.)[S]

5.5 SR for GEMM operands

In a quantised GEMM $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$ both operands are quantised. Let $\hat{\boldsymbol{A}} = \mathcal{Q}(\boldsymbol{A})$, $\hat{\boldsymbol{B}} = \mathcal{Q}(\boldsymbol{B})$ with independent randomness. If each quantiser is unbiased conditionally on its input and the scales are deterministic functions of the input (not of the random draw), then $\mathbb{E}[\hat{\boldsymbol{A}}\hat{\boldsymbol{B}}] = \mathbb{E}[\hat{\boldsymbol{A}}]\mathbb{E}[\hat{\boldsymbol{B}}] = \boldsymbol{A}\boldsymbol{B}$, so the product is unbiased.

Remark 5.4 (Data-dependent selection breaks unbiasedness). If the quantiser chooses among candidates using the realised random rounding (for instance Four Over Six selecting the lower-error scale after rounding), the selection is correlated with the draw and the estimator is no longer unbiased. The Quartet II authors make this point to explain why combining Four Over Six with SR is biased; our own SR+4/6 numbers (relative MSE 0.0175 in their Table 1, lower than 0.0235 for plain SR) reflect a real variance reduction, but one that comes with bias [5].[S][D]

Variance of an SR dot product. Let only $\boldsymbol{x}$ be stochastically rounded and let $\boldsymbol{y}$ be fixed. Then $\mathbb{E}[\hat{\boldsymbol{x}}^\top \boldsymbol{y}] = \boldsymbol{x}^\top \boldsymbol{y}$ and, by independence across elements and Proposition 5.1,

$$\operatorname{Var}[\hat{\boldsymbol{x}}^\top \boldsymbol{y}] = \sum_i y_i^2 \operatorname{Var}[\hat{x}_i] \le \sum_b \hat{s}_b^2 \, \|\boldsymbol{y}_b\|^2, \tag{5.4}$$

since within block b the maximum spacing of the E2M1 grid is 2, so $\operatorname{Var}[\hat{x}_i] \le (2\hat{s}_b)^2/4 = \hat{s}_b^2$. With $\hat{s}_b \approx A_b/6$,

$$\operatorname{Var}[\hat{\boldsymbol{x}}^\top \boldsymbol{y}] \lesssim \frac{1}{36} \sum_b A_b^2 \, \|\boldsymbol{y}_b\|^2. \tag{5.5}$$

The variance is controlled by the block maxima: one outlier in $\boldsymbol{x}_b$ raises the noise on every element of the block, and the noise is multiplied by the block's share of $\boldsymbol{y}$. Rotations that reduce A_b relative to $\|\boldsymbol{x}_b\|$ therefore directly reduce the variance of SR products, which is the argument of Tseng et al. [27] and is made quantitative in Chapter 6.[D]

Error of a quantised GEMM. With independent noises on $\boldsymbol{A}$ and $\boldsymbol{B}$ having relative MSE ε_A and ε_B, a first-order expansion for random inner products gives

$$\frac{\|\hat{\boldsymbol{C}} - \boldsymbol{C}\|_F}{\|\boldsymbol{C}\|_F} \approx \sqrt{\varepsilon_A + \varepsilon_B} \tag{5.6}$$

independent of the inner dimension K. To see this, write $\hat{\boldsymbol{C}} - \boldsymbol{C} = \delta\boldsymbol{A}\,\boldsymbol{B} + \boldsymbol{A}\,\delta\boldsymbol{B} + \delta\boldsymbol{A}\,\delta\boldsymbol{B}$ and take $\boldsymbol{A} \in \mathbb{R}^{M\times K}$, $\boldsymbol{B} \in \mathbb{R}^{K\times N}$ with independent zero-mean unit-variance entries, and error matrices whose entries are uncorrelated with the operands and have variances ε_A, ε_B. Each entry of $\delta\boldsymbol{A}\,\boldsymbol{B}$ is a sum of K terms of variance ε_A, so $\mathbb{E}\|\delta\boldsymbol{A}\,\boldsymbol{B}\|_F^2 = MNK\varepsilon_A$, whereas $\mathbb{E}\|\boldsymbol{C}\|_F^2 = MNK$. The same holds for $\boldsymbol{A}\,\delta\boldsymbol{B}$, the cross term has zero mean and the last term is second order. Quantisation errors are *not* exactly uncorrelated with the data, so this is a heuristic derivation.[D] We verify (5.6) in Table 5.3: for Gaussian operands the measured error is flat in $K \in [64, 16384]$ and matches the prediction to within 1.5% in all but one entry (4% for SR at $K = 64$).[E] Predicted values are $\sqrt{2 \times 0.00905} = 0.135$ (RTN), $\sqrt{2 \times 0.0189} = 0.194$ (SR).

K	NVFP4 RTN	pred.	NVFP4 SR	pred.	MXFP4 RTN	INT4 G=32
64	0.1350	0.1354	0.2006	0.1939	0.1623	0.1366
256	0.1339	0.1343	0.1954	0.1956	0.1631	0.1368
1024	0.1352	0.1346	0.1941	0.1945	0.1618	0.1374
4096	0.1339	0.1346	0.1938	0.1945	0.1619	0.1370
16384	0.1361	0.1346	0.1959	0.1944	0.1632	0.1382

Table 5.3: Relative Frobenius error of a $128 \times K$ by $K \times 128$ Gaussian GEMM with both operands quantised, versus inner dimension K; prediction $\sqrt{\varepsilon_A + \varepsilon_B}$ from (5.6).[E]

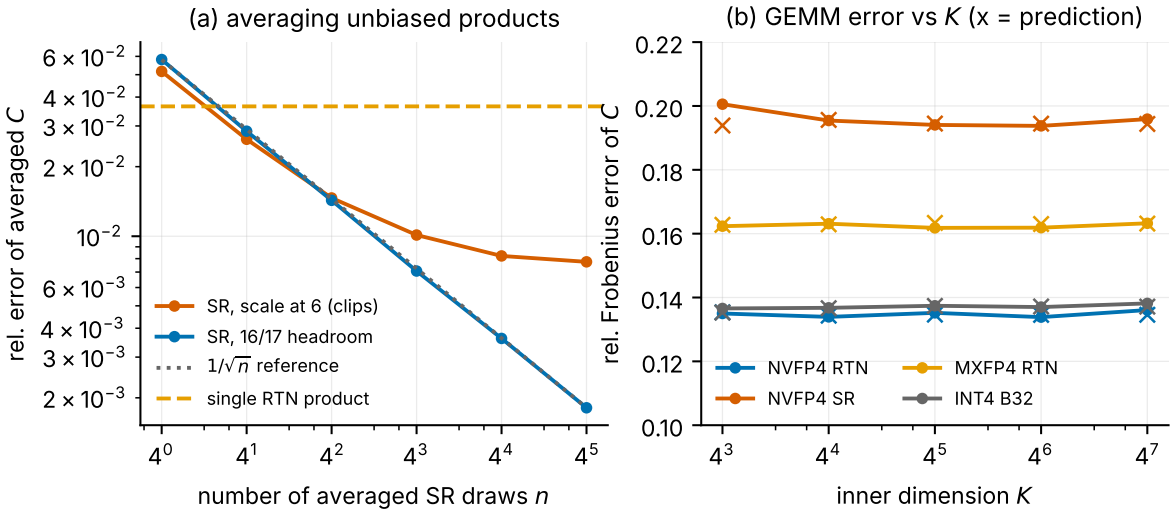


Figure 5.1: (a) Averaging independent SR-quantised products converges at rate $1/\sqrt{n}$ if no clipping occurs (16/17 headroom) and stalls with clipping bias otherwise. The dashed line is the error of a single RTN product. (b) GEMM error versus K; crosses are the first-order prediction.[E]

5.6 MS-EDEN: unbiased gradients without element-wise SR

Element-wise SR doubles the quantisation error (Table 4.5). Quartet II replaces it by MS-EDEN, a block-scale variant of the EDEN estimator of Vargaftik et al. [37]. The construction, as described in the Quartet II paper, is:

1. Apply a random Hadamard transform to each chunk of 128 elements of the vector $\boldsymbol{x}$.
2. Quantise the rotated vector with round-to-nearest (this has the low error of RTN).
3. Compute for each group of 16 elements a correction $S = \langle \boldsymbol{x}^{\mathrm{RHT}}, \boldsymbol{x}^{\mathrm{RHT}} \rangle / \langle \boldsymbol{x}^{\mathrm{RHT}}, \boldsymbol{x}^{\mathrm{RTN}} \rangle$ that makes the quantised vector aligned in magnitude with the original, and fold it into the E4M3 block scale by *stochastically* rounding $S \cdot \hat{s}_b$ to E4M3.

Because the stochastic rounding of the scale is unbiased, the correction is represented in expectation, and the construction is unbiased up to the asymptotic guarantee of EDEN (stated in the limit of large dimension). Their reported relative MSE on Gaussian data is 9.4×10^{-3} for MS-EDEN against 23.5×10^{-3} for SR, a factor of 2.5, compared to 9.0×10^{-3} for RTN [5].[S] They report better gradient estimation across the major matrix multiplications, validate end-to-end training up to 1.9B parameters on 38B tokens, and provide Blackwell kernels with up to $4.2\times$ speed-up over BF16.[S] We have not reimplemented MS-EDEN. The headline error claim is a statement about Gaussian data; its generalisation to the heavy-tailed tensors discussed in Chapter 12 is an open empirical question (Chapter 18).

5.7 Is SR necessary? Quartet's reversal

The NVIDIA recipe applies SR to gradient tensors and states that doing so is essential for convergence of the 12B model, while applying SR to forward-pass tensors is detrimental because it amplifies error [3].[S] The Quartet paper originally concluded that SR in the backward pass was superior and, in its latest version, rejects that conclusion. Its footnote explains that an implementation bug in the first draft caused large degradation for RTN backward quantisation on longer runs; after the fix, RTN backward quantisation (with a Hadamard transform, for MXFP4) consistently outperformed the alternatives, although the gap with SR decreased with training duration [30].[S] In the isolated-method comparison of that paper's Table 2 (MXFP4, Hadamard-transformed operands), the fitted data-efficiency eff_D is 0.93 for RTN against 0.88 for SR, with a magnitude misalignment $1 - \mathbb{E}[1/S]$ of 9.3×10^{-3} for RTN against 0 for SR.[S]

This is a genuine disagreement about regime, not a contradiction. The two groups use different formats (MXFP4 with $G = 32$ versus NVFP4 with $G = 16$), different scales (scaling-law fits at 25–400 tokens per parameter plus a 7B-parameter run, versus 12B parameters for 10T tokens, i.e. about 830 tokens per parameter) and different treatment of Hadamard rotations in the backward pass. Our bias bound (Proposition 5.2) says that bias eventually matters at long horizons or low learning rates; the empirical report that the gap narrows with training duration is in the opposite direction of what that argument would suggest if bias accumulated, so either the RTN bias is small after rotation (rotation makes the RTN error approximately independent of the signal) or the tested horizons were too short. We have no data to separate these. The comparison paper of 2026 (Chapter 8) reports that rankings only settle after more than 150B tokens, which is a reason to treat conclusions from shorter runs with caution [10].[S][U]

5.8 Summary

- SR is unbiased with variance $(z - g_j)(g_{j+1} - z)$; on average it doubles the squared error of RTN.[D][E]
- Unbiasedness of block-scaled SR requires a headroom factor ($16/17$ for E4M3 scales) so that no element is clipped; without it the estimator has a clipping-bias floor that prevents averaging from converging.[D][E]
- Bias adds an irreducible term $3\beta^2$ to the SGD gradient-norm bound; variance adds a term proportional to the learning rate.[D]
- The need for SR on gradients is established in NVIDIA's 12B run but contested in the Quartet line of work; MS-EDEN offers unbiasedness with RTN-like error on Gaussian data.[S]

Chapter 6

Random Hadamard Transforms and Outliers

Rotations are the standard remedy for outliers in 4-bit quantisation: incoherence processing in weight-only quantisation (QuIP#), rotated LLMs (QuaRot), and the NVFP4 training recipe all rely on them [3, 38, 39]. This chapter analyses what a random Hadamard transform (RHT) does to the blocks of a block-scaled format, derives a bound on the post-rotation crest factor and on the variance of stochastic-rounding dot products, and reports our simulations of when rotation helps FP4 and when it does not. The picture that emerges is more nuanced than "rotations reduce outliers, hence improve quantisation": for NVFP4 with round-to-nearest the rotation is neutral or harmful on most input distributions we tested.

6.1 Definition and invariance

Let $\boldsymbol{H}_d \in \{\pm 1\}^{d\times d}$ be the Sylvester Hadamard matrix (d a power of two), $\tilde{\boldsymbol{H}}_d = \boldsymbol{H}_d/\sqrt{d}$ its orthogonal normalisation (symmetric, $\tilde{\boldsymbol{H}}_d^2 = \boldsymbol{I}$), and $\boldsymbol{D} = \mathrm{diag}(\epsilon_1, \dots, \epsilon_d)$ with i.i.d. Rademacher signs ϵ_i. The random Hadamard transform is $\boldsymbol{x} \mapsto \tilde{\boldsymbol{H}}_d \boldsymbol{D}\boldsymbol{x}$. For a longer vector, apply it independently to consecutive tiles of d entries (block-diagonal $\boldsymbol{R} = \mathrm{blockdiag}(\tilde{\boldsymbol{H}}_d\boldsymbol{D}, \dots)$), usually with one sign vector shared by all tiles.

Since $\boldsymbol{R}$ is orthogonal, $\langle \boldsymbol{R}\boldsymbol{x}, \boldsymbol{R}\boldsymbol{y}\rangle = \langle \boldsymbol{x}, \boldsymbol{y}\rangle$. A GEMM $\boldsymbol{C} = \boldsymbol{A}\boldsymbol{B}$ with contraction dimension K is unchanged if $\boldsymbol{A}$ and $\boldsymbol{B}^\top$ are both rotated along K: $\boldsymbol{C} = (\boldsymbol{A}\boldsymbol{R}^\top)(\boldsymbol{R}\boldsymbol{B})$. The point of rotating is that the rotated operands, not the originals, are quantised.

Where the contraction dimension lies. In a linear layer $\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{W}^\top$ the three GEMMs contract over different axes: Fprop over input features, Dgrad over output features and Wgrad over tokens (batch times sequence). Block scales are applied along the contraction axis. Consequently the block structure of the *same tensor* is different in the three GEMMs: for activations $\boldsymbol{X}$, blocks run along features in Fprop and along tokens in Wgrad. The NVIDIA recipe applies the RHT only in Wgrad, where blocks run along the token dimension, and reasons that rotation along a dimension that the weight also participates in (Fprop, Dgrad) would not be undone consistently after quantising the weight [3].[S] It also reports that at smaller scales the transform showed no measurable benefit for Fprop and Dgrad, and for the 12B model it reports $d = 16$ to be better than $d = 4$ and similar to $d = 128$, with a single random sign vector shared by all layers.[S]

A/σ	crest before	crest after	NVFP4 none	NVFP4 RHT_{16}	NVINT4 none	NVINT4 RHT_{16}
1	2.08	2.12	0.0090	0.0090	0.0073	0.0075
3	2.50	2.06	0.0081	0.0091	0.0102	0.0071
5	3.18	1.91	0.0069	0.0094	0.0159	0.0061
10	3.73	1.59	0.0076	0.0101	0.0222	0.0042
30	3.97	1.23	0.0081	0.0140	0.0148	0.0025
100	4.00	1.07	0.0015	0.0079	0.0015	0.0018

Table 6.1: One-spike blocks: a 16-element block with one entry of magnitude $A\sigma$ and fifteen $\mathcal{N}(0,1)$ entries, with and without a 16-point random Hadamard transform (2×10^5 blocks per row; relative MSE of the block). "Crest" is $\sqrt{16}\,\|\cdot\|_\infty/\|\cdot\|_2$.[E]

6.2 What a rotation does to a block

6.2.1 Gaussianisation and a crest-factor bound

Proposition 6.1 (Crest factor after RHT)**.** *Let $\boldsymbol{x} \in \mathbb{R}^d$ be fixed and $\boldsymbol{y} = \tilde{\boldsymbol{H}}_d \boldsymbol{D}\boldsymbol{x}$ with random signs $\boldsymbol{D}$. Then for every $\delta \in (0,1)$, with probability at least $1-\delta$,*

$$\|\boldsymbol{y}\|_\infty \;\le\; \|\boldsymbol{x}\|_2 \sqrt{\frac{2\ln(2d/\delta)}{d}}. \tag{6.1}$$

Proof. Coordinate j is $y_j = \sum_i \epsilon_i \tilde{H}_{ji} x_i$ with $|\tilde{H}_{ji}| = d^{-1/2}$, a Rademacher sum with coefficients $c_i = \tilde{H}_{ji} x_i$, $\sum_i c_i^2 = \|\boldsymbol{x}\|^2/d$. Hoeffding's inequality gives $\mathbb{P}(|y_j| \ge t) \le 2\exp(-t^2 d/(2\|\boldsymbol{x}\|^2))$. A union bound over d coordinates and $t = \|\boldsymbol{x}\|\sqrt{2\ln(2d/\delta)/d}$ gives the claim. □

Since $\|\boldsymbol{y}\|_2 = \|\boldsymbol{x}\|_2$, the crest factor $\kappa = \sqrt{d}\|\boldsymbol{y}\|_\infty/\|\boldsymbol{y}\|_2$ obeys $\kappa(\boldsymbol{y}) \le \sqrt{2\ln(2d/\delta)}$ w.h.p.; without rotation $\kappa(\boldsymbol{x})$ can be as large as $\sqrt{d}$ (for a vector with a single non-zero entry, which the rotation turns into a vector of entries $\pm\|\boldsymbol{x}\|/\sqrt{d}$, crest factor exactly 1). The bound is *sub-logarithmic in the dimension*: it replaces $\sqrt{d}$ by $\sqrt{2\ln(2d/\delta)}$. For $d = 16$, $\delta = 0.01$: $\sqrt{2\ln 3200} = 4.0$, equal to the trivial worst case $\sqrt{16} = 4$. For $d = 32$: 4.2 against 5.7; for $d = 128$: 4.5 against 11.3.[D]

Remark 6.2 (Implication for block size 16). At $G = 16$ the maximum crest factor is 4, a Gaussian block has typical crest factor about 2.1 (from $\mathbb{E}[A_{16}] = 2.08$ for unit variance), and the guarantee of Proposition 6.1 is vacuous. There is little room for rotation to help a 16-element block unless the block is close to the worst case. For $G = 32$ and $G = 128$ there is more room, which is one reason why rotations help MXFP4 and long-group INT4 more than NVFP4.[D]

6.2.2 Rotation interacts with the shape of the grid

The E2M1 grid is non-uniform and, as Section 4.9 showed, the highest cell is coarse. Which block shapes does the FP4 grid represent well, and which does a rotation produce? Table 6.1 shows one block of 16 values in which one entry has magnitude A and the remaining fifteen are standard normal, quantised with NVFP4 (a) as is and (b) after a 16-point RHT.

Without rotation a spike block is almost ideal for FP4: the spike is mapped to 6 and reproduced up to the E4M3 scale error, and the fifteen small entries are nearly all flushed to zero, an error of at most $15/(A^2+15)$ of the block energy (0.0015 at $A = 100$). With rotation the block becomes *flat*: every output has magnitude $\approx A/4 \pm \sqrt{15}/4$, so after scaling nearly every element lies in $[5,6]$ and rounds to 6. This collapses the fine structure: the grid has no level between 4 and 6, so the reconstruction is a constant equal to (roughly) the block maximum, which exceeds the block mean by several percent, and

NVFP4 ($G = 16$)

Input	kurt.	none	d=16	d=32	d=64	d=128	d=256
Gaussian	0.0	0.00904	0.00905	0.00905	0.00907	0.00905	0.00905
Laplace	3.0	0.00861	0.00923	0.00914	0.00909	0.00908	0.00906
t_5	5.3	0.00852	0.00920	0.00914	0.00911	0.00908	0.00906
outlier channels	166.6	0.00646	0.01314	0.01339	0.01251	0.01253	0.01252
massive token rows	629.6	0.00901	0.00904	0.00895	0.00903	0.00909	0.00894
sparse spikes	882.5	0.00590	0.01139	0.01232	0.01238	0.01163	0.01087

NVINT4 ($G = 16$, **E4M3 scale**)

Input	kurt.	none	d=16	d=32	d=64	d=128	d=256
Gaussian	0.0	0.00745	0.00745	0.00746	0.00743	0.00743	0.00746
Laplace	3.0	0.01052	0.00693	0.00717	0.00731	0.00737	0.00741
t_5	5.3	0.01037	0.00697	0.00715	0.00728	0.00734	0.00738
outlier channels	166.6	0.00962	0.00259	0.00277	0.00306	0.00305	0.00305
massive token rows	629.6	0.00740	0.00755	0.00730	0.00758	0.00756	0.00757
sparse spikes	882.5	0.00676	0.00342	0.00355	0.00374	0.00403	0.00447

Table 6.2: Relative MSE under tiled RHT of size d (d none = no rotation); 2048×1024 tensors.[E]

the relative error is of the order of $((\max - \text{mean})/\text{mean})^2 \approx 0.4$–$1\%$ rather than 0.15% (at $A = 30$ the NVFP4 error rises from 0.0081 to 0.0140; at $A = 100$ from 0.0015 to 0.0079).[E][D] An INT4 grid with uniform spacing behaves the opposite way: its error is proportional to $\mathbb{E}[A^2]/Q^2_{\max}$, which the rotation reduces strongly (NVINT4 at $A = 30$: $0.0148 \to 0.0025$).[E]

This is a mechanistic explanation of a pattern that other groups have reported. Egiazarian et al. find that Hadamard rotations *hurt* NVFP4 round-to-nearest quantisation while they improve MXFP4 and INT4 formats, and propose block-wise rotations together with GPTQ-style error compensation (MR-GPTQ) [12]; Chen et al. find that with Hadamard rotation integer formats (NVINT4) can outperform NVFP4 [11].[S]

6.3 Simulation: rotation versus input distribution

We rotated 2048×1024 tensors along the last axis with tiles of $d \in \{16, 32, 64, 128, 256\}$ and one shared sign vector, then quantised with blocks along the same axis. Tables 6.2 and 6.3 give the relative MSE. The input distributions are Gaussian; Laplace; Student-t_5; "outlier channels" ($1/64$ of the columns scaled by 30); "massive token rows" ($1/256$ of the rows scaled by 50); and "sparse spikes" (0.2% of entries scaled by 40, i.i.d.).

The results are consistent across formats. First, on Gaussian inputs the rotation changes nothing (as it must: a rotated Gaussian block is distributed as the original). Second, for NVFP4 the rotation is neutral or harmful for every heavy-tailed distribution: Laplace $0.00861 \to 0.00923$ at $d = 16$ ($+7\%$, and $+5\%$ at $d = 256$), outlier channels $0.0065 \to 0.0131$ ($2\times$), sparse spikes $0.0059 \to 0.0114$. Third, NVINT4, INT4 and MXFP4 all gain substantially on the same inputs; NVINT4 reaches 0.0026 on outlier channels against 0.0065 for NVFP4 without rotation. Fourth, massive token rows have no effect for any format, because the scale of a row is absorbed by the block scales along the row; a rotation along the *token* axis (as in Wgrad) would behave differently, and we examine it in Section 6.5. Fifth, larger tiles beyond $d = 16$ give small further gains for formats that benefit, and sometimes lose (NVINT4 on sparse spikes: $0.0034 \to 0.0045$ from $d = 16$ to 256), so tile size and block size are best matched. When the tile exceeds the quantisation block, rotation also mixes energy across blocks, which removes some of the block-to-block magnitude variation that block scaling exploits.[E][U]

MXFP4 ($G = 32$, **E8M0 ceil**)

Input	kurt.	none	d=16	d=32	d=64	d=128	d=256
Gaussian	0.0	0.01332	0.01333	0.01332	0.01332	0.01331	0.01331
Laplace	3.0	0.01756	0.01341	0.01323	0.01328	0.01331	0.01328
t_5	5.3	0.01747	0.01335	0.01316	0.01323	0.01327	0.01328
outlier channels	166.6	0.02601	0.01236	0.01088	0.01086	0.01086	0.01091
massive token rows	629.6	0.01368	0.01366	0.01353	0.01364	0.01404	0.01405
sparse spikes	882.5	0.01969	0.01261	0.01133	0.01159	0.01131	0.01162

INT4 ($G = 32$, **ideal FP32 scale**)

Input	kurt.	none	d=16	d=32	d=64	d=128	d=256
Gaussian	0.0	0.00942	0.00941	0.00942	0.00939	0.00940	0.00942
Laplace	3.0	0.01466	0.00916	0.00893	0.00917	0.00928	0.00934
t_5	5.3	0.01496	0.00915	0.00894	0.00913	0.00925	0.00931
outlier channels	166.6	0.01805	0.00472	0.00306	0.00340	0.00339	0.00339
massive token rows	629.6	0.00933	0.00929	0.00936	0.00945	0.00937	0.00960
sparse spikes	882.5	0.01136	0.00535	0.00411	0.00434	0.00469	0.00519

Table 6.3: As Table 6.2 for MXFP4 and INT4.[E]

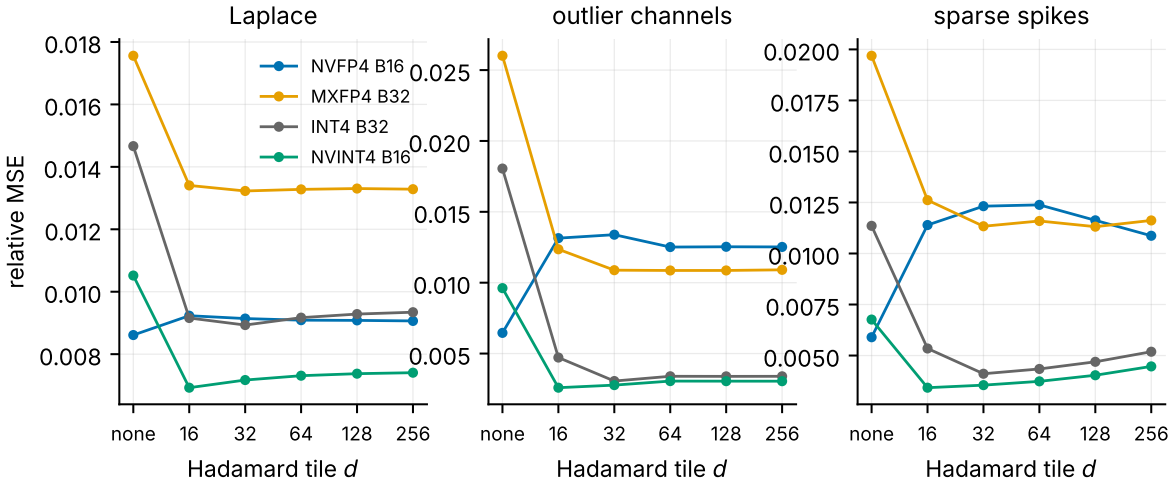


Figure 6.1: Relative MSE versus RHT tile size for four formats on three of the input distributions.[E]

6.4 Variance of stochastic-rounding products under rotation

Chapter 5 derived that the variance of a stochastic-rounding dot product is controlled by block maxima, $\mathrm{Var} \lesssim \frac{1}{36} \sum_b A_b^2 \|\boldsymbol{y}_b\|^2$. Rotation changes this quantity directly.

Corollary 6.3 (SR variance after RHT)**.** *Let $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, rotate both with the same tile-d RHT (tile = a union of whole quantisation blocks), and stochastically round $\boldsymbol{x}$ block-wise with ideal scales $A_b/6$. With probability at least $1 - \delta$ over the signs,*

$$\mathrm{Var}\left[\hat{\boldsymbol{x}}^\top \boldsymbol{y}\right] \;\le\; \frac{\ln(2d/\delta)}{18\,d}\,\|\boldsymbol{x}\|^2\|\boldsymbol{y}\|^2. \tag{6.2}$$

Proof. Within tile T, Proposition 6.1 gives $A_b^2 \le 2\ln(2d/\delta)\|\boldsymbol{x}_T\|^2/d$ for every block $b \subset T$. Substituting into (5.4), $\mathrm{Var} \le \frac{1}{36} \cdot \frac{2\ln(2d/\delta)}{d} \sum_T \|\boldsymbol{x}_T\|^2 \sum_{b\subset T} \|\boldsymbol{y}_b\|^2 = \frac{\ln(2d/\delta)}{18d} \sum_T \|\boldsymbol{x}_T\|^2\|\boldsymbol{y}_T\|^2$ and $\sum_T \|\boldsymbol{x}_T\|^2\|\boldsymbol{y}_T\|^2 \le \|\boldsymbol{x}\|^2\|\boldsymbol{y}\|^2$. □

Without rotation the same argument gives only $\mathrm{Var} \le \frac{1}{36}\|\boldsymbol{x}\|^2\|\boldsymbol{y}\|^2$ (worst case $A_b = \|\boldsymbol{x}_b\|$). The ratio of the two bounds is $2\ln(2d/\delta)/d$: it equals about 1.0 for $d = 16$, 0.55 for $d = 32$, 0.30 for $d = 64$

Input	Format	none	d=16	d=32	d=64	d=128
no outliers	NVFP4	0.0055	0.0057	0.0057	0.0059	0.0056
no outliers	MXFP4	0.0066	0.0062	0.0057	0.0060	0.0055
$1/256$ of entries at 20σ	NVFP4	0.0343	0.0134	0.0094	0.0084	0.0074
$1/256$ of entries at 20σ	MXFP4	0.0448	0.0126	0.0090	0.0085	0.0066
$1/64$ at 20σ	NVFP4	0.0292	0.0113	0.0079	0.0091	0.0070
$1/64$ at 20σ	MXFP4	0.0317	0.0103	0.0076	0.0080	0.0065
$1/16$ at 20σ	NVFP4	0.0215	0.0096	0.0085	0.0077	0.0064
$1/16$ at 20σ	MXFP4	0.0239	0.0080	0.0078	0.0078	0.0062
$1/256$ at 100σ	NVFP4	0.0491	0.0180	0.0156	0.0139	0.0092
$1/256$ at 100σ	MXFP4	0.0975	0.0234	0.0221	0.0127	0.0099

Table 6.4: Relative standard deviation of an SR-quantised dot product $\langle \hat{\boldsymbol{x}}, \hat{\boldsymbol{y}} \rangle$ versus RHT tile d ($n = 4096$, $y = x + 0.3\,\text{noise}$, 400 draws).[E]

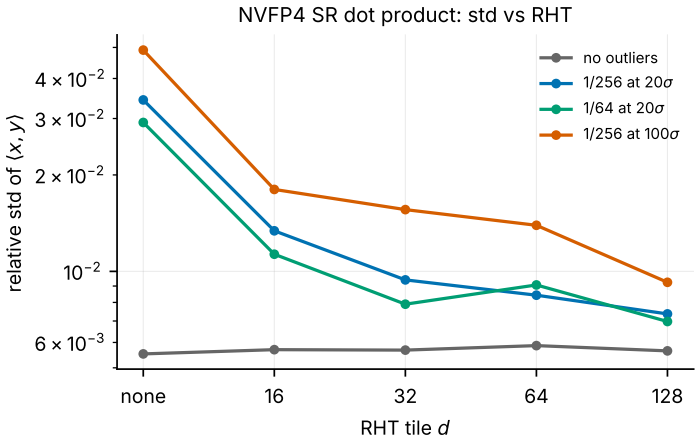


Figure 6.2: Standard deviation of an SR dot product versus RHT tile size, for sparse outliers of 20σ (densities $1/256$ and $1/64$) and 100σ; the flat line is the no-outlier case.[E]

and 0.16 for $d = 128$ at $\delta = 0.01$. This is the sense in which a larger rotation tile helps SR, and the sense in which a tile equal to the block size ($d = 16$ for NVFP4) offers no *guarantee*.[D] For $\boldsymbol{y} = \boldsymbol{x}$ the bound gives a relative standard deviation of at most $\sqrt{\ln(2d/\delta)/(18d)}$, i.e. 0.17 for $d = 16$ and 0.066 for $d = 128$. These numbers are an order of magnitude above what we measure (0.013 and 0.007 on spiky data, 0.0055 without outliers): the bound uses the largest cell spacing for every element, a union bound over coordinates and a worst case over inputs, so it is informative about the *scaling* with d and not about absolute levels.[D][E]

Simulation. Table 6.4 reports the relative standard deviation of $\langle \hat{\boldsymbol{x}}, \hat{\boldsymbol{y}} \rangle$ (both vectors rounded stochastically with headroom, $n = 4096$, 400 draws, fresh random signs each draw) for vectors with sparse outliers. Without outliers the rotation has no effect (relative standard deviation 0.0055–0.0059 for NVFP4 for every d). With $1/256$ of the entries at 20σ, rotation reduces the standard deviation of NVFP4 from 0.0343 to 0.0134 ($d = 16$) and 0.0074 ($d = 128$): variance reductions of $6.6\times$ and about $22\times$. At 100σ the reduction is from 0.0491 to 0.0092. Measured relative biases are of order 10^{-3} or smaller; the largest are for the unrotated 100σ case (0.0037 NVFP4, 0.0063 MXFP4), which are within about 1.5 standard errors of zero (standard error $= \text{std}/\sqrt{400}$). MXFP4 behaves the same way with its $3/4$ headroom. The $d = 16$ tile gives a $6.6\times$ variance reduction despite the vacuous bound: spike blocks have crest factor near the worst case 4, and rotation lowers it to about 2.[E]

Gradient	Rounding of $\boldsymbol{G}$	no RHT	RHT d=16	RHT d=128
Gaussian	RTN	0.135	0.134	0.134
	SR (single draw)	0.180	0.180	0.180
	SR (mean of 40)	0.098	0.098	0.098
token-spiky	RTN	0.110	0.134	0.147
	SR (single draw)	0.191	0.198	0.195
	SR (mean of 40)	0.098	0.098	0.098
Student-$t_{2.5}$	RTN	0.128	0.137	0.135
	SR (single draw)	0.190	0.182	0.181
	SR (mean of 40)	0.098	0.098	0.098

Table 6.5: Relative Frobenius error of a quantised weight-gradient GEMM, with and without RHT along the token axis. "Mean of 40" averages 40 independent SR draws with fresh signs; it plateaus at $\approx 0.098 \approx \sqrt{0.0090}$ because the RTN-quantised $\boldsymbol{X}$ contributes a deterministic error that does not average out.[E]

6.5 Rotation in the weight-gradient GEMM

The NVIDIA report found the rotation of Wgrad inputs important at 12B scale [3].[S] We tested the weight-gradient GEMM $\boldsymbol{dW} = \boldsymbol{G}^\top \boldsymbol{X}$ with inner dimension $T = 4096$ tokens, 256×256 output, quantising both operands along T with blocks of 16: the gradient $\boldsymbol{G}$ is rounded either by RTN or by SR (with headroom), the activation $\boldsymbol{X}$ always by RTN. Gradients are Gaussian, "token-spiky" (every 97th token scaled by 60) or Student-$t_{2.5}$. We report $\|\boldsymbol{d\hat{W}} - \boldsymbol{dW}\|_F / \|\boldsymbol{dW}\|_F$ in Table 6.5.

We found *no* benefit of the rotation in this metric: for Gaussian gradients the numbers are identical to three decimals; for token-spiky gradients RTN error rises from 0.110 to 0.134 ($d = 16$) and 0.147 ($d = 128$); SR errors change by less than 4%. This does not contradict the variance results above: Table 6.5 measures the Frobenius error of a matrix whose energy is dominated by the large tokens (the "spikes" are signal), while Table 6.4 measures the standard deviation of a scalar product relative to its mean. It also does not contradict the NVIDIA result, which concerns the stability of a long training run, not the instantaneous error of one GEMM. It does show that a one-step error metric is not a reliable predictor of whether Wgrad rotation helps, and that the claimed benefit should be confirmed by training experiments rather than inferred from MSE. We return to this point when discussing the toy training runs of Chapter 13.[E][U]

6.6 Costs

A dense $d \times d$ Hadamard transform applied by a GEMM costs about $2d$ FLOPs per element (for $d = 16$, 32 FLOPs per element), while a fast Walsh–Hadamard transform costs $\log_2 d$ additions per element; with tensor cores the dense 16×16 form is cheap relative to the contraction $2K$ FLOPs per output element when $K \gg d$, but it adds memory traffic and a kernel pass unless fused. In the NVIDIA recipe the transform is applied to Wgrad inputs only. Quartet II applies RHT over chunks of 128 elements for MS-EDEN [5].[S] A large share of the practical complexity of FP4 training kernels comes from fusing such transforms with quantisation and transposition; we quote no timings because we did not measure them.

6.7 Summary

- An RHT makes every coordinate of a rotated tile sub-Gaussian; the crest factor after rotation is $O(\sqrt{\ln d})$ w.h.p. against $\sqrt{d}$ before. At $G = 16$ the guarantee is vacuous.[D]
- The variance of an SR dot product is bounded by $\frac{\ln(2d/\delta)}{18d}\|\boldsymbol{x}\|^2\|\boldsymbol{y}\|^2$ after a tile-d rotation; measured variance reductions of 6.6× ($d = 16$) to about 22× ($d = 128$) on spiky data.[D][E]
- For round-to-nearest NVFP4 the rotation is neutral on Gaussian data and increases MSE on heavy-tailed data, because it turns spike-dominated blocks (cheap for FP4) into flat blocks that collapse onto the coarse top cell. INT4-based formats benefit strongly.[E][D]
- The benefit of rotation in a Wgrad GEMM cannot be established from one-step error alone; it is an empirical question about training stability.[E][U]

Chapter 7

Block-Scaled GEMM: Arithmetic and Accumulation

Chapters 4–6 treated quantisation of a tensor. A tensor-core GEMM adds a second error source, the arithmetic of the dot product itself. This chapter works out what a block-scaled FP4 matrix-multiply-accumulate computes exactly, where rounding can enter, and how much error each source contributes.

7.1 Structure of a block-scaled dot product

Let $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^K$ be one row of the left operand and one column of the right operand, each stored as FP4 codes q_i^a, q_i^b with a scale per block of G elements. Their dot product is

$$\hat{y} = s_{\text{ten}}^a \, s_{\text{ten}}^b \sum_{b=1}^{K/G} \hat{s}_b^a \, \hat{s}_b^b \sum_{i \in b} q_i^a \, q_i^b. \tag{7.1}$$

The inner sum is a dot product of E2M1 codes. Writing each E2M1 value as $\alpha/2$ with $\alpha \in \{0, 1, 2, 3, 4, 6, 8, 12\}$, a product is $\alpha\alpha'/4$ with $\alpha\alpha' \le 144$, and the sum of $G = 16$ products is an integer multiple of $\frac{1}{4}$ of magnitude at most $2304/4 = 576$, which needs 12 bits plus sign. The inner sum can therefore be computed *exactly* in fixed-point arithmetic; no floating-point rounding is needed within a block of 16 (or 32).[D] The block scales multiply as E4M3×E4M3, a product of two 4-bit significands ($1.xxx \times 1.yyy$, at most 8 bits) with an exponent sum, again exactly representable in a small floating-point or fixed-point format. The tensor-level scales $s_{\text{ten}} = 1/s_{\text{enc}}$ are FP32 factors that can be applied after the accumulation (in the epilogue).

The only inexact step is therefore the accumulation *across* blocks, with $n = K/G$ partial sums. In the NVIDIA report the tensor cores accumulate in FP32 and produce BF16 or FP32 outputs [3].[S] The internal accumulator width of the Blackwell block-scaled instructions is not stated in the sources we read; for FP8 on Hopper, DeepSeek-AI et al. [40] measured that the accumulation retained about 14 bits of the mantissa product, with a maximum relative error of nearly 2% at $K = 4096$ in their test.[S] Whether an analogous truncation exists for FP4 is something we cannot determine from the literature we retrieved.[U]

7.2 Accumulation error: a sequential-sum model

Suppose the partial sums are accumulated sequentially in a format with p explicit mantissa bits (unit roundoff $u = 2^{-(p+1)}$). For n terms $t_1, \ldots, t_n$ the standard worst-case bound is $|\hat{S} - S| \le (n -$

K	FP32 acc.	FP16 acc.	BF16 acc.
256	2.3e-07	2.1e-03	1.7e-02
1024	4.0e-07	3.7e-03	2.9e-02
4096	8.2e-07	7.2e-03	5.3e-02
16384	2.0e-06	1.4e-02	1.1e-01
65536	3.3e-06	3.2e-02	2.0e-01

Table 7.1: Median relative error of a sequentially accumulated sum of K zero-mean random products, by accumulator format (each partial sum rounded to the format).[E]

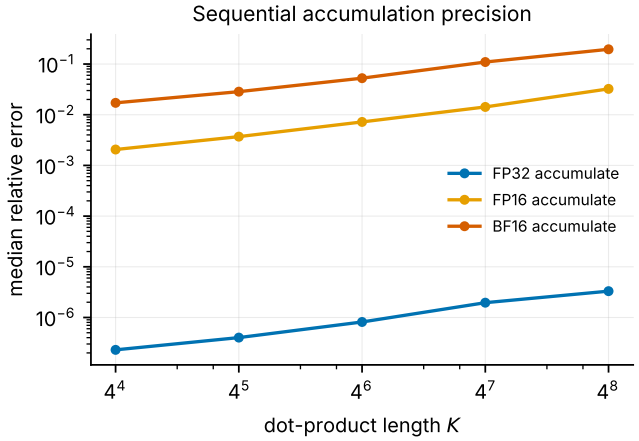


Figure 7.1: Median relative error of sequential accumulation versus number of terms.[E]

$1)u\sum_i|t_i| + O(u^2)$, and for random-sign terms with random rounding errors the typical error scales as $u\sqrt{n}\,\max_k|S_k|$ where S_k is the k-th partial sum.

We tested sequential accumulation of K terms in FP32, FP16 and BF16 (each addition rounded to the accumulator format): products of independent Gaussian factors ($\sigma = 0.5$ and 0.5 with mean 0.1 on the second factor, so each product is zero-mean), 256 independent dot products, median relative error against a float64 sum (Table 7.1, Figure 7.1).[E]

Three conclusions follow. (i) With an FP32 accumulator the error ($\leq 3.3 \times 10^{-6}$ at $K = 65536$) is negligible against the quantisation error of FP4 operands (≈ 0.13, Table 5.3): five orders of magnitude. (ii) With an FP16 or BF16 accumulator the error grows roughly as $\sqrt{K}$ and at $K = 65536$ reaches 0.03 (FP16) and 0.20 (BF16): comparable to the entire quantisation error. (iii) In a block-scaled GEMM the number of accumulated terms is $n = K/G$, not K: a contraction of $K = 65536$ with $G = 16$ accumulates 4096 partial sums, and the relevant rows of Table 7.1 are those with $K \approx n$ (for $n = 4096$: FP16 7.2×10^{-3}, BF16 5.3×10^{-2}). Whenever the accumulator has fewer than about 24 bits, accumulation can compete with quantisation error, and Wgrad GEMMs (contraction over tokens, tens of thousands) are the most exposed. A lossy accumulator is therefore a plausible and *unquantified* source of loss in FP4 training on hardware where it exists; the simulations in this report, and any fake-quantisation study in FP32, cannot detect it.[E][D][U]

7.3 Systematic gain of the quantisers

Beyond random error, a quantiser can systematically shrink or inflate a vector. Table 7.2 reports the projection of $\hat{\boldsymbol{x}}$ onto $\boldsymbol{x}$ ("gain", $\langle\hat{\boldsymbol{x}}, \boldsymbol{x}\rangle/\|\boldsymbol{x}\|^2$) and the energy ratio $\|\hat{\boldsymbol{x}}\|^2/\|\boldsymbol{x}\|^2$ for 2^{21} samples.

For a quantiser whose error is orthogonal to its output, $\|\hat{\boldsymbol{x}}\|^2 = \|\boldsymbol{x}\|^2 - \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$; for one whose

	Gaussian			Laplace		
Quantiser	$\langle \hat{x}, x\rangle / \langle x, x\rangle$	$\|\hat{x}\|^2/\|x\|^2$	rel. MSE	$\langle \hat{x}, x\rangle / \langle x, x\rangle$	$\|\hat{x}\|^2/\|x\|^2$	rel. MSE
NVFP4 RTN	0.9955	1.0001	0.00904	0.9963	1.0013	0.00857
NVFP4 + 4/6	0.9964	1.0004	0.00756	0.9971	1.0018	0.00759
MXFP4 ceil	0.9898	0.9930	0.01334	0.9909	0.9994	0.01755
INT4 G32	1.0006	1.0105	0.00941	1.0001	1.0149	0.01466
NVFP4 SR	0.9968	1.0124	0.01883	0.9951	1.0084	0.01832
NVFP4 SR 16/17	0.9997	1.0230	0.02355	0.9999	1.0245	0.02482

Table 7.2: Gain and energy ratio of block quantisers (Gaussian and Laplace data).[E]

error is independent of the input, $\|\hat{\boldsymbol{x}}\|^2 = \|\boldsymbol{x}\|^2 + \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2$. Round-to-nearest NVFP4 is between the two: gain 0.9955 (a 0.45% shrinkage of the signal component) with energy ratio 1.00007. MXFP4 (ceil) shrinks the signal by 1.0%. Stochastic rounding with $16/17$ headroom has gain 0.9997, essentially unbiased, with energy ratio 1.023: the extra energy is exactly the added variance. INT4 with $G = 32$ has gain 1.0006. These are small effects; the Quartet papers' "magnitude misalignment" is of the order of 10^{-2} and is defined for rotated vectors, so it is not directly comparable [30]. We conclude that, for the formats studied, systematic shrinkage of activations by RTN is at most $\sim 1\%$ and cannot by itself explain visible degradation; accumulated over many layers it could still matter, which we did not test.[E][U]

7.4 From tensor error to GEMM error to layer error

Equation (5.6) states $\varepsilon_{\mathrm{GEMM}} \approx \varepsilon_A + \varepsilon_B$ for the relative squared error of $\boldsymbol{AB}$. For a Fprop GEMM with RTN-quantised activations and weights of relative MSE 0.009 each, the relative error of the output is $\sqrt{0.018} = 13\%$, which is the figure measured in Table 5.3 (0.134). A 13% relative error in a single linear layer output looks large but must be read as noise relative to the signal's norm, spread across many output features. Two heuristics help.

Signal-to-noise through depth. Suppose layer ℓ contributes energy E_ℓ to the residual stream and adds independent noise of energy εE_ℓ. The relative variance of the accumulated noise is $\sum_\ell \varepsilon E_\ell / \sum_\ell E_\ell = \varepsilon$ if contributions add in energy: the relative error does not grow with depth in this idealised model, and is dominated by whichever layers carry most of the stream's energy. This is a qualitative argument; correlated contributions, normalisation layers and attention nonlinearities all change it, and we did not measure it.[D][U]

Sensitivity differs between layers. The empirical recipes keep the last layers in higher precision because they are sensitive (Chapter 8); a second-order model of the loss (Section 9.1) explains why: the loss increase from noise on a layer's output is weighted by the curvature of the loss with respect to that output, which is not uniform.

7.5 Summary

- Intra-block FP4 dot products are exact in fixed point; only accumulation across blocks and the output format introduce arithmetic error.[D]
- Accumulator precision of ≥ 24 bits makes arithmetic error negligible; a $\leq$ 16-bit mantissa accumulator would produce errors comparable to FP4 quantisation error at long contractions. The accumulator width of block-scaled FP4 instructions is not documented in the sources we read.[E][U]

- GEMM error is $\sqrt{\varepsilon_A + \varepsilon_B}$ independent of K for random operands (verified to 1.5%).[E]
- RTN shrinks signals by $\lesssim 1\%$; SR with headroom is unbiased to 3×10^{-4} in projection.[E]

Chapter 8

Training Recipes

This chapter describes the NVFP4 pretraining recipe of NVIDIA [3] in detail, reviews the evidence offered for it, and then surveys the recipes that appeared in the following year and the points on which they disagree. Numbers are those reported by the cited authors unless tagged otherwise.

8.1 The NVIDIA recipe

8.1.1 Setting

The report trains a 12-billion-parameter hybrid Mamba–Transformer (Nemotron-H family [41]) on 10 trillion tokens with a warmup-stable-decay (WSD) learning-rate schedule, with changes of the data blend during training, against an FP8 baseline. A second experiment compares formats on an 8B hybrid model trained for 1T tokens against a BF16 baseline. The authors state that the 10T-token run is the first public evidence of sustained 4-bit pretraining at multi-trillion-token scale.[S]

8.1.2 Components

Each linear layer has three GEMMs: forward (Fprop), activation-gradient (Dgrad) and weight-gradient (Wgrad). The recipe is:

1. **Selective precision.** All GEMMs in linear layers use NVFP4 inputs with BF16 or FP32 outputs, except the first two and the last eight blocks (each block is an FFN or a Mamba-2 layer with two linear layers), which stay in BF16, about 16% of linear layers. The report says training diverges if every linear layer is quantised, and that convergence remained stable with only the last four blocks in high precision in an ablation. It recommends keeping fewer than 15% of layers, mostly at the end of the network, in BF16 or MXFP8. Attention (softmax, QK and score–value batched GEMMs), embeddings, the output head, normalisation layers and non-linearities stay in BF16/FP32; master weights, weight gradients and optimiser state are FP32; tensor-parallel reductions are in BF16.[S]
2. **Random Hadamard transform on Wgrad inputs only**, tile 16×16, one random sign vector shared by all layers throughout training. The authors report $d = 16$ better than $d = 4$ and similar to $d = 128$ and no measurable benefit for Fprop or Dgrad at the scales they tested.[S]
3. **Two-dimensional weight scaling**: 16×16 tiles for weights, 1×16 blocks for activations and gradients, motivated by consistency between forward and backward quantisations of the same weight ("chain-rule violation" hypothesis); the report observes training is less sensitive to inconsistency in activations than weights.[S]

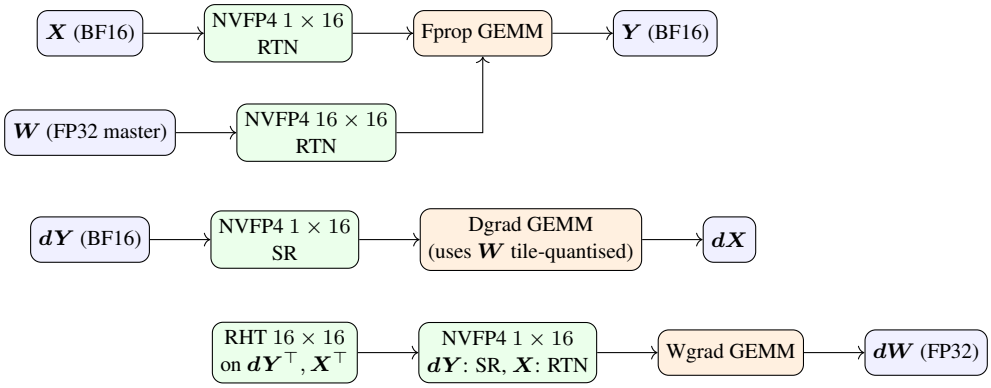


Figure 8.1: Compute flow of one NVFP4 linear layer in the NVIDIA recipe (schematic after Figure 5 of NVIDIA [3]). Green boxes quantise to NVFP4; orange boxes are tensor-core GEMMs with BF16/FP32 outputs.

Task	FP8	NVFP4	Task	FP8	NVFP4
General (avg.)	68.99	69.82	**Code (avg.)**	59.52	56.67
MMLU (5-shot)	77.36	76.57	HumanEval+	59.93	57.43
MMLU-Pro (5-shot)	62.62	62.58	MBPP+	59.11	55.91
AGIEval English CoT	67.01	70.31	**Commonsense (avg.)**	77.29	76.75
Math (avg.)	86.20	86.88	ARC Challenge	91.81	91.81
GSM8k CoT	89.08	92.27	HellaSwag	83.83	83.09
MATH	83.32	81.48	OpenBookQA	47.60	47.40
Multilingual (avg.)	77.93	80.24	PIQA	82.64	82.70
Global MMLU	74.00	74.94	Winogrande	80.58	78.77
MGSM	81.87	85.53			

Table 8.1: Downstream accuracy of the 12B hybrid model after 10T tokens of FP8 and NVFP4 pretraining (Table 2 of NVIDIA [3]; the group averages are as reported there).[S]

4. **Stochastic rounding for gradients**, round-to-nearest-even for weights and activations; SR on forward tensors is reported as detrimental, and SR on non-gradient backward tensors as unhelpful.[S]

8.1.3 Reported results

For the 12B model, the validation loss of NVFP4 tracks FP8 closely: the relative loss error stays below 1% during the stable phase and widens to slightly above 1.5% as the learning rate decays towards the end of training.[S] Downstream accuracies, evaluated in BF16, are close to those of the FP8 run on most benchmarks (Table 8.1). The authors single out coding, where NVFP4 is lower, and suspect noisy evaluation (MBPP+ drops on the final checkpoint).

Reading the table. Fourteen individual benchmark scores are listed: NVFP4 is higher on five (AGIEval, GSM8k, Global MMLU, MGSM and, by 0.06, PIQA), lower on eight and tied on one (ARC Challenge). Differences range from $+3.7$ (MGSM) to -3.2 (MBPP+) points. There is one run per configuration, so the table does not by itself show whether a difference exceeds run-to-run variability. For orientation, the 2026 comparison of recipes reports, for repeated NVFP4 runs of 8B and 30B-A3B models, relative ranges across three runs of 0.7–9.8% depending on the task (e.g. MMLU 0.9–2.2%, MMLU-Pro 6.3–9.8%, Code 5.6–6.8%) [10].[S] By that yardstick most of the entries of Table 8.1 lie within plausible run-to-run noise; we say "plausible" because those noise references come from different models and benchmarks' settings.[D][U]

Switching to higher precision. The report observes that most of the loss gap arises from forward-pass quantisation: switching the forward pass to BF16 from 8.2T tokens (about the last 18% of training; the report recommends switching shortly before the onset of learning-rate decay) lowers the relative loss gap from about 1.5% to 0.5%, while switching only the backward pass gives no such improvement; switching both at 8.2T matches the FP8 baseline, and switching at $\sim$ 10T helps slightly. Switching the forward pass alone requires about 6% of total computation in high precision. The authors note this is in contrast to Chmiel et al., who reported recovery from backward-pass switching [3, 31].[S]

NVFP4 versus MXFP4 at 8B. In a 1T-token comparison of an 8B hybrid model against a BF16 baseline, with the same recipe (RHT $d = 32$ for MXFP4 to match the block), the relative loss error is about 1.5% for NVFP4 and 2.5% for MXFP4; MXFP4 reaches NVFP4's final loss only with 36% more tokens (1.36T instead of 1T).[S] Since the recipes differ in more than the scale format (block size and scale type both change), the experiment compares full formats and does not isolate which part matters; our MSE analysis (Chapter 4) suggests both do.[D]

8.1.4 Why the evidence is thinner than it looks

- One 12B run with one data blend and, as far as the report states, one seed. The ablation studies of Figure 4 start from a checkpoint at 3.43T tokens, remove one component at a time and compare over the following few hundred billion tokens; they do not run to 10T.[S]
- The FP8 baseline is the authors' own FP8 recipe. Different FP8 recipes can differ in loss by a few tenths of a percent (DeepSeek-V3 reports $< 0.25\%$ against BF16 for its recipe on different models [40]), so a gap of 1.5% is distinguishable but the baseline matters.[S][U]
- The 2026 comparison shows that rankings of recipes do not stabilise until beyond 150B tokens and that early ordering can be reversed, which implies that short ablations are weak evidence for design choices [10]; the NVIDIA ablations are not short in tokens but are limited in number of seeds.[S]
- The recipe's components were tuned on the authors' model family (Mamba-2 hybrids). The CHON study finds that outlier structure differs across attention types and that post-QK operations are sensitive, suggesting that exemptions chosen for one architecture may not carry over [42].[S][U]

8.2 Competing and refining recipes

8.2.1 Quartet and Quartet II

Quartet (MXFP4) combines a Hadamard transform and a QuEST-style MSE-minimising clipping forward pass with an RTN backward pass, built on a scaling law $L = \left[A/(N\,\mathrm{eff}_N)^{\alpha} + B/(D\,\mathrm{eff}_D)^{\beta}\right]^{\gamma} + E$ in which eff_N and eff_D summarise the parameter and data efficiency of the forward and backward quantisation schemes [30].[S] Quartet II moves to NVFP4, combines an RTN forward pass (it is designed to work with recent forward-pass improvements such as Four Over Six scale selection) with MS-EDEN for unbiased backward-pass gradient estimation, and reports end-to-end training of models up to 1.9B parameters on 38B tokens and kernels with up to $4.2\times$ speed-up over BF16 [5]. A scaling-law treatment is given in Chapter 9.[S]

Recipe	Weight quantisation	Tiled RHT	Gradient bias treatment	4/6 applied to
NVIDIA '25	2D, no re-quantisation	Wgrad, $d = 16$, static signs	SR	none
Update 1	1D, re-quantised in backward	Wgrad, $d = 16$, static signs	SR	none
Update 2	1D, re-quantised	Wgrad & Dgrad, $d = 128$, resampled signs	SR	none
Update 3	1D, re-quantised	Wgrad & Dgrad, $d = 128$	MS-EDEN	$\boldsymbol{W}^\top, \boldsymbol{X}^\top$
Mosaic NVFP4	1D, re-quantised	Wgrad & Dgrad, $d = 128$	MS-EDEN	$\boldsymbol{W}, \boldsymbol{X}, \boldsymbol{W}^\top, \boldsymbol{X}^\top$

Table 8.2: Cumulative changes from the NVIDIA'25 recipe to Mosaic NVFP4 (Table 1 of Agrusa et al. [10]); all keep the last 15% of layers in BF16.[S]

8.2.2 TetraJet-v2

TetraJet-v2 trains OLMo-2 models of 70M, 150M and 370M parameters for 52B, 107B and 212B tokens with NVFP4 in all linear layers. Its three contributions are (i) an unbiased "double-block" quantisation, (ii) OsciReset, an algorithm to suppress weight oscillation (weights near a quantisation threshold flip between adjacent grid points across steps; see Nagel et al. [43] for the phenomenon in QAT), and (iii) OutControl, which keeps statically selected persistent outlier activation channels in higher precision (selected in a short window early in training). The authors report that applying RHT in the forward pass performs worse than no RHT there and use it only in the backward pass, which agrees with our simulation (Chapter 6). Reported validation perplexities for the 370M model: BF16 23.70, Quartet 26.16, the NVIDIA recipe 26.20, TetraJet-v2 (base) 25.50 and TetraJet-v2 (full) 25.11, i.e. a reduction of the gap to BF16 of 51.3% on average over the settings they test and a claimed $1.67\times$ end-to-end speed-up over FP8 [6].[S] The scale (up to 370M) is much smaller than the NVIDIA and Mosaic experiments, and the perplexity gaps relative to BF16 (at 370M, $+5.9\%$ for TetraJet-v2-full to $+10.5\%$ for the NVIDIA recipe in the validation set) are not directly comparable with the loss gaps in the large-scale reports, so the results do not transfer directly.[U]

8.2.3 Four Over Six in pretraining

Cook et al. pretrain a 30B-A3B mixture-of-experts hybrid model (the Nemotron 3 Nano architecture) on 1T tokens on 384 B200 GPUs with batch size 3072 and a WSD schedule, using the NVIDIA recipe as baseline. Adding 4/6 brings the training loss 13.0% closer to BF16 (i.e. the gap to BF16 shrinks by 13% of itself); they find that using 4/6 only on activations gives comparable performance; in contrast to PTQ, selecting by MAE is best in pretraining; and, strikingly, selection that is biased in combination with SR (selecting after rounding) outperforms two variants constructed to keep the gradient estimator unbiased, which gave no improvement over the NVFP4 baseline. The overhead on the NVFP4 quantisation kernel is stated as under 15% [7].[S] This is the second place (after Quartet) where a method with some bias outperforms an unbiased one in practice.

8.2.4 "What Matters for NVFP4 Training?"

The October 2026 study by authors from NVIDIA and ISTA compares recipes on two models, Nemotron-H 8B (dense, 52 blocks) and Nemotron 3 Nano 30B-A3B (a MoE with 3.2B active parameters), up to 1T tokens each, with AdamW, WSD and sequence length 8192 [10].[S] It stacks changes on the NVIDIA'25 recipe (Table 8.2) and calls the result Mosaic NVFP4.

Findings reported there:

1. Each update improves training loss; stacked, the total gain is 0.36 percentage points on the MoE.

	Nemotron 3 Nano 30B-A3B				Nemotron-H 8B			
Task	NV'25	Mosaic	noise %	Δ %	NV'25	Mosaic	noise %	Δ %
MMLU 5-shot	67.5	68.4	0.9	+1.3	61.0	60.3	2.2	−1.1
MMLU-Pro 5-shot CoT	45.0	47.1	6.3	+4.7	29.7	27.6	9.8	−7.1
Math avg.	79.4	79.2	4.6	−0.3	51.3	52.3	4.6	+1.9
Common sense avg.	77.6	77.1	1.4	−0.6	76.5	75.9	1.1	−0.8
Code avg.	57.1	56.6	6.8	−0.9	43.4	43.0	5.6	−0.9
RACE 0-shot	80.8	81.1	0.7	+0.4	77.7	78.8	1.1	+1.4

Table 8.3: Downstream scores (0–100) for the NVIDIA'25 and Mosaic NVFP4 recipes at 1T tokens, with the reference noise (relative range over three repeated NVIDIA'25 runs) and the relative difference Δ (Table 4 of Agrusa et al. [10]). Comparing $|\Delta|$ with the noise reference ourselves, only MMLU on the MoE ($+1.3\%$ against 0.9%) and RACE on the dense model ($+1.4\%$ against 1.1%) exceed it.[S]

The largest single gain comes from replacing SR by MS-EDEN and adding 4/6 on backward tensors.

2. On Nemotron-H 8B Mosaic NVFP4 ends within a relative training-loss gap $< 1.2\%$ of BF16, an improvement of 0.24 percentage points over NVIDIA'25. On the MoE the gap is $< 0.81\%$, an improvement of 0.36 percentage points.
3. Run-to-run variability, from three repeats at the 1T horizon: about 0.06% (train) and 0.04% (validation) for the MoE; 0.027% and 0.064% for the dense model. Differences larger than these are unlikely to be noise.
4. Keeping the last 15% of layers in BF16 is *required*: with all layers in NVFP4 the Mosaic recipe diverges on the 8B model beyond 300B tokens. The authors conclude that this indicates a representation failure that the error-reduction techniques cannot fix.
5. Backward-pass techniques show no clear separation when the last 15% is kept in BF16; they separate only in the less stable all-layers-quantised regime (where MS-EDEN on gradients plus 4/6 on $\boldsymbol{X}^\top, \boldsymbol{W}^\top$ is best).
6. Rankings of recipes do not stabilise until well beyond 150B tokens; "weaker recipes often initially appear better than the strongest ones".
7. Downstream accuracy differences between NVIDIA'25 and Mosaic are mostly inside the noise reference (Table 8.3).

The paper therefore supports three claims and contradicts one of the NVIDIA report's design choices. Supported: stochastic rounding (or unbiased surrogates) on gradients; selective high-precision layers; a late-emerging failure mode that makes short ablations unreliable. Contradicted: two-dimensional 16×16 weight scaling is replaced by one-dimensional scaling with re-quantisation in the backward pass, with an RHT also on Dgrad and tile size 128 rather than 16. The authors note that the improvement from resampling stochastic patterns and sign vectors (an extra 0.028 pp on the MoE with a different optimiser configuration) was not included in their headline numbers.[S]

A note on authorship and independence. The comparison is written by authors affiliated with NVIDIA and with ISTA, whose group authored Quartet, Quartet II and MR-GPTQ; two of the techniques evaluated (MS-EDEN, MR-GPTQ-related rotations) come from the latter group and Four Over Six from an NVIDIA/MIT collaboration. This is not a reason to doubt the numbers, but it is the opposite of an independent audit.[U]

8.2.5 Transposition-invariant 2D scaling, full-stack FP4, CHON

Rahimifar et al. argue that the source of FP4 instability in 1D block quantisation is *scale inconsistency under transposition*: in the backward pass, tensors are transposed and values are regrouped into different blocks, so forward and backward see different scales. They propose 2D block quantisation, "truncation-free" scaling and stochastic rounding, MXFP8 for query and key projections, and report stable end-to-end FP4 training on dense models up to 7B parameters and a 30B MoE on up to 100B tokens with under 1.3% degradation in perplexity and downstream accuracy [8].[S] This is the same motivation as the NVIDIA recipe's 2D weight scaling, extended to activations, and it conflicts with the Mosaic recipe's move away from 2D weights; the two groups did not test on the same setting.[U]

Full-Stack FP4 extends NVFP4 to AdamW momentum storage, a Muon-like optimiser step and low-precision attention; on a 3B model with 64B tokens, BF16 and Full-Stack FP4 reach training losses of 2.267 and 2.286 (0.838% gap), and a "LoRA-SVD" device that keeps a compact projection subspace in BF16 reduces the linear-only loss gap from 1.40% to 0.61% [9].[S]

CHON studies where outliers appear in NVFP4 pretraining and finds that, compared with softmax attention, linear attention reduces per-tensor heavy tails but retains persistent block-level spikes; outliers evolve from transient spikes to a small set of persistent "hot channels"; its Hot-Channel Patch re-injects the residual of hot channels, and with protection of post-QK operations the loss gap to BF16 on a 1.3B GLA model with 60B tokens falls from 0.94% to 0.58% [42].[S] This echoes TetraJet-v2's static outlier-channel retention and the structural-outlier literature (e.g. LLM.int8(), SmoothQuant [44, 45]).

8.2.6 AMD MXFP4 on native FP4 hardware

Cim et al. ask why full-pipeline FP4 training diverges even when Fprop activations and Dgrad remain stable. On Llama 3.1 8B pretraining on C4 using MXFP4 on AMD Instinct MI355X, they find that quantising Wgrad is the primary driver of convergence degradation, whereas FP4 in Fprop and Dgrad alone introduces only modest additional token requirements. They also report that stochastic rounding and *randomised* Hadamard rotations fail to stabilise training once Wgrad is quantised, whereas *deterministic* Hadamard rotations consistently restore stable optimisation, and conclude that instability stems from structured micro-scaling errors along gradient paths rather than insufficient stochasticity [46].[S] This conflicts directly with the NVIDIA report, which prescribes SR and a randomised (fixed random sign) transform for Wgrad, and it is on a different format (MXFP4), model and hardware; whether the contradiction is real or a feature of MXFP4's coarser power-of-two scale is unresolved.[U]

8.2.7 Graphcore: UE5M3 scales and a simpler recipe

Hu et al. pretrain a Nemotron-H 8B for 188.7B tokens. They compare Transformer Engine's NVFP4 recipe (current-tensor scaling, RHT, BF16 final layers) with UE5M3 block scales, whose wider range allows *periodic* (delayed) tensor scaling; the proposed recipe uses selective SR on backward gradients, omits RHT and uses FP4 in all eligible linears. Final-window training losses (single seed-42 trajectories; software-emulated for UE5M3) are given in Table 8.4. The report adds that a native NVFP4 ablation which jointly removes RHT and the BF16 final-block exemption is unstable (205 losses above 3 and 89 gradient norms above 1 after step 2500) and finishes at 2.4420, although its throughput is 21.2% higher [14].[S]

Two cautions apply. The Transformer Engine NVFP4 baseline gap here (3.6%) is more than twice the $\sim 1.5\%$ NVIDIA reports at 12B/10T and than the Mosaic dense gap ($< 1.2\%$ at 1T), and the runs are single seeds. The comparisons mix format, scale lifecycle and GEMM numerics. The numbers show that the BF16 final layers and RHT cannot simply be dropped on native NVFP4, and that a wider-range

Run	Final-window loss	Gap to BF16 (derived)
BF16	2.2651	—
UE5M3 FP4, Transformer-Engine settings	2.2902	+1.1%
UE5M3 FP4, proposed block-16 recipe (no RHT, all linears)	2.3090	+1.9%
block-32 control	2.3241	+2.6%
decoded-operand GEMM control	2.3157	+2.2%
Transformer Engine NVFP4 (native)	2.3474	+3.6%
Native NVFP4 without RHT and without BF16 final block	2.4420	+7.8%

Table 8.4: Final-window mean training loss (mean of ten logged losses in the last 250 steps of 30,000) of Nemotron-H 8B runs in Hu et al. [14]; the gap column is our arithmetic from the reported values.[S][D]

Work	Scale	Format	Distinctive elements	Reported result (relative to BF16/FP8)
NVIDIA'25 [3]	12B hybrid, 10T tok.	NVFP4	2D weights, RHT(Wgrad,16), SR grads, 16% BF16	loss gap $< 1\%$ (stable) to $> 1.5\%$ (late) vs FP8; $\sim 0.5\%$ with fwd BF16 switch
Mosaic [10]	8B dense, 30B-A3B MoE, 1T tok.	NVFP4	1D+requant, RHT 128 (W,D), MS-EDEN, 4/6, 15% BF16	gap $< 1.2\%$ (dense), $< 0.81\%$ (MoE); all-layer FP4 diverges > 300B
Four Over Six [7]	30B-A3B MoE, 1T tok.	NVFP4+4/6	per-block scale 4 or 6	loss 13% closer to BF16
Quartet II [5]	up to 1.9B, 38B tok.	NVFP4	MS-EDEN backward, improved RTN forward	"more than 2× lower error than SR"; kernels 4.2× over BF16
TetraJet-v2 [6]	70M–370M, $\leq$212B tok.	NVFP4	OsciReset, OutControl, double-block	51.3% avg. gap reduction; val. ppl 25.11 vs 23.70 BF16 (370M)
Transp.-inv. 2D [8]	$\leq$7B dense, 30B MoE, $\leq$100B tok.	FP4 2D blocks	2D blocks, MXFP8 for QK	$< 1.3\%$ degradation (ppl, tasks)
Full-stack [9]	3B, 64B tok.	NVFP4	LoRA-SVD, FP4 optimiser, attention	gap 0.838% (loss 2.286 vs 2.267)
CHON [42]	GLA 1.3B, 60B tok.	NVFP4	hot-channel patch	gap $0.94\% \rightarrow 0.58\%$
Graphcore [14]	8B hybrid, 188.7B tok.	UE5M3 block scale	delayed tensor scale, no RHT	gap +1.9% (proposed) vs +3.6% (TE NVFP4), single seed
AMD [46]	Llama 3.1 8B, C4	MXFP4	deterministic Hadamard	Wgrad is the failure source; SR / random RHT insufficient
FP4 all the way [31]	7B, $\leq$200B tok.	NVFP4 (Gaudi2)	SR bwd/update, RTN fwd	comparable to BF16 downstream

Table 8.5: Recipes compared (reported results only).[S]

scale may make them unnecessary—but the evidence is a single trajectory per configuration.[U]

8.3 Comparison across recipes

Table 8.5 lines up the recipes. Scale of evidence varies by three orders of magnitude in parameters and tokens, so the reported gaps are not comparable across rows.

8.4 Points of disagreement

1. **2D versus 1D weight scaling.** NVIDIA'25 and the transposition-invariant work argue for forward/backward consistency; Mosaic abandons 2D weights for 1D plus re-quantisation; our simulation shows 2D scaling has substantially higher MSE (real-weight relative MSE 0.0137 versus 0.0090 on SmolLM2-135M, Chapter 13).[E][S]

2. **Where to rotate.** Wgrad only (16) versus Wgrad and Dgrad (128) versus none (Graphcore) versus deterministic transforms (AMD). Our results show that rotation is MSE-neutral or harmful for NVFP4 RTN but reduces SR variance (Chapter 6).
3. **SR versus unbiased surrogates versus biased selection.** NVIDIA: SR; Quartet II/Mosaic: MS-EDEN; Four Over Six: biased selection beats unbiased variants; AMD: SR fails at Wgrad.
4. **How many layers can be FP4.** NVIDIA: all but $\sim$ 15–16%; Quartet II and the transposition-invariant work: possibly all, on small models/short horizons; Mosaic: all-layer FP4 diverges beyond 300B tokens at 8B; Graphcore: all eligible linears with UE5M3 and periodic scaling.

These disagreements are mostly between works that do not share a setting. The comparative study is the first to share one, and it settles some questions (high-precision layers needed at 1T tokens; MS-EDEN/4/6 give small gains) but only for its two architectures and its optimiser.

Chapter 9

Loss Gap, Scaling Laws and the Economics of 4-Bit Training

The previous chapters treated quantisation error. This chapter connects it to model quality: how a relative MSE in a tensor becomes a loss gap in a trained model, how scaling-law formulations describe the penalty, and what the reported gaps are worth in tokens and in net wall-clock terms. Parts of this chapter are derived by us from published numbers, and are labelled as such; the arithmetic of Section 9.4 is illustrative.

9.1 A second-order model of the loss gap

Let $\mathcal{L}(\boldsymbol{\theta})$ be the training loss and consider a perturbed evaluation at $\boldsymbol{\theta} + \boldsymbol{\delta}$, where $\boldsymbol{\delta}$ is zero-mean noise with covariance $\boldsymbol{\Sigma}$ independent of the data, as a surrogate for the forward-pass quantisation noise on weights. A second-order Taylor expansion at a stationary point ($\nabla\mathcal{L} \approx 0$) gives

$$\mathbb{E}\big[\mathcal{L}(\boldsymbol{\theta}+\boldsymbol{\delta}) - \mathcal{L}(\boldsymbol{\theta})\big] \;\approx\; \tfrac{1}{2}\,\mathrm{tr}(\boldsymbol{H}\boldsymbol{\Sigma}), \tag{9.1}$$

with $\boldsymbol{H}$ the Hessian of the loss. If the noise on weight i has relative variance ε_i, i.e. $\Sigma_{ii} = \varepsilon_i w_i^2$, and cross terms are ignored,

$$\Delta\mathcal{L} \;\approx\; \tfrac{1}{2}\sum_i \varepsilon_i\, h_{ii}\, w_i^2. \tag{9.2}$$

The loss penalty is the quantisation error weighted by the *curvature-weighted energy* $h_{ii}w_i^2$ of each parameter. Three consequences follow, none of which is new but all of which are relevant to FP4.[D]

Layer sensitivity. The sum can be grouped by layer. A layer whose curvature-weighted weight energy is large—typically layers close to the output, where perturbations propagate with little further averaging—contributes disproportionately. Heuristics based on this expression (Hessian-trace sensitivity) are the basis of mixed-precision assignment in earlier quantisation work, and the recipes of Chapter 8 reach the same practical conclusion empirically: the final layers must stay in higher precision [3, 10]. We did not compute Hessians for any of the models discussed; the model explains *why* sensitivity differs but we have no measurement of how much.[D][U]

Relative MSE is the right variable, to first order. Because $\Delta\mathcal{L}$ is linear in ε, ranking quantisers by relative MSE is justified when the noise is independent and comparably distributed across layers. This is the logic behind the use of MSE as a proxy for accuracy in several of the cited papers [5, 11]. It breaks

down when the noise is correlated with the weights (RTN shrinkage, Table 7.2), when the Hessian is highly non-diagonal, and when the loss is not at a stationary point.

Weights adapt, activations do not. Equation (9.1) describes noise that is independent of $\boldsymbol{\theta}$. Forward-pass weight quantisation is a deterministic function of $\boldsymbol{\theta}$; during training the master weights move to positions where the quantised function has lower loss (the effect that quantisation-aware training exploits), so the realised gap at convergence is smaller than the first-order estimate. NVIDIA makes the corresponding observation that weights tolerate a coarser scale granularity (the 2D 16×16 scale) "because they can adapt to the FP4 values" [3].[S] Activation noise depends on the input and cannot be absorbed this way. This asymmetry is consistent with the choice of 2D scaling for weights but 1D for activations in the NVIDIA recipe, and it is a plausible reason why the loss gap is dominated by the forward pass.[D][U]

9.2 Why the gap grows late in training

A relative loss gap $g = (\mathcal{L}_q - \mathcal{L})/\mathcal{L}$ has a numerator that, under (9.1), is roughly constant in time once the weights have adapted, and a denominator $\mathcal{L}$ that decreases as training progresses. Even a constant absolute penalty therefore appears as a growing relative gap. In addition, as the learning rate decays, the gradient noise that dithers round-to-nearest (Table 5.2) decays too, so the bias terms of Proposition 5.2 that were previously masked become visible. Both effects agree with the observed pattern in the NVIDIA run, where the gap is $< 1\%$ in the stable phase and widens to above 1.5% as the learning rate decays [3], and with the Mosaic study, where both recipes show an increased loss gap during annealing [10].[S][D] We cannot separate the two mechanisms with the data available.

The finding that switching to higher precision *in the forward pass* (but not in the backward pass) before the decay recovers most of the gap [3] is also consistent with this picture: the forward noise is a persistent perturbation of the function being trained, whereas backward noise acts on the optimisation and shrinks with the learning rate.[S][D]

9.3 Scaling-law formulations

9.3.1 Quartet's parameter and data efficiencies

Quartet fits the validation loss over a grid of model sizes N and token counts D with

$$L(N, D, P_{\text{fwd}}, P_{\text{bwd}}) = \left[\frac{A}{(N \operatorname{eff}_N(P_{\text{fwd}}))^{\alpha}} + \frac{B}{(D \operatorname{eff}_D(P_{\text{bwd}}))^{\beta}} \right]^{\gamma} + E, \tag{9.3}$$

where $\operatorname{eff}_N, \operatorname{eff}_D \leq 1$ summarise the *parameter efficiency* of the forward-pass precision and the *data efficiency* of the backward-pass precision [30].[S] A method is better than another if it has both higher eff_N and higher eff_D. For the isolated forward schemes in MXFP4 the authors fit $\operatorname{eff}_N = 0.42$ (stochastic rounding), 0.59 (round-to-nearest) and 0.64 (QuEST: Hadamard transform with MSE-optimal clipping), together with Gaussian MSEs of 2.77×10^{-2}, 1.37×10^{-2} and 1.32×10^{-2} [30]. They note that eff_N correlates strongly with MSE. Their MSE values for RTN and QuEST (1.37 and 1.32×10^{-2}) match our simulated MXFP4 values (0.0133 and 0.0132, Table 4.5), which cross-checks our quantiser against an independent implementation.[S][E]

A derived relation between eff_N and MSE. If the effect of forward noise is to replace N by $N \operatorname{eff}_N$ in the first term, the loss penalty at fixed N and D is, to first order in $\ln(1/\operatorname{eff}_N)$, $\Delta L \approx c \ln(1/\operatorname{eff}_N)$

Forward method (MXFP4)	MSE (Gaussian)	eff_N	$\ln(1/\mathrm{eff}_N)$	$\ln(1/\mathrm{eff}_N)/\mathrm{MSE}$
SR, AbsMax	2.77e-02	0.42	0.868	31.3
RTN, AbsMax	1.37e-02	0.59	0.528	38.5
QuEST (Hadamard + RMSE clip)	1.32e-02	0.64	0.446	33.8

Table 9.1: Quartet's isolated forward-pass comparison (MXFP4): published eff_N and Gaussian MSE (Table 2 of Castro et al. [30]) with our derived ratio.[S][D]

with $c = \alpha\gamma\,[\cdot]^{\gamma-1}A\,N^{-\alpha}$ ($[\cdot]$ the bracket of the law), i.e. proportional to $\ln(1/\mathrm{eff}_N)$; (9.1) says the same penalty is proportional to the relative MSE. Equating the two predicts $\ln(1/\mathrm{eff}_N) \propto \mathrm{MSE}$ at fixed model and data. Table 9.1 checks this on the three published points: the ratio $\ln(1/\mathrm{eff}_N)/\mathrm{MSE}$ is 31, 38 and 34 ($\pm 12\%$ about its mean), consistent with proportionality given that each eff_N was fitted from noisy training runs. We do not claim more than consistency: three points do not establish a law, and SR noise is not identical to a deterministic MSE.[D][S]

What the efficiencies mean. A parameter efficiency of 0.64 means a quantised-forward model behaves, in the fitted law, like a model with $0.64\,N$ parameters. For the best Quartet configuration the authors state that it reaches the same loss as the strongest prior FP4 method with roughly 15% fewer parameters and $5\times$ less data [30].[S] These are fitted quantities for MXFP4 on Llama-type models of 30M–7B parameters and token-to-parameter ratios of 25–400, not NVFP4 at frontier scale; the same group has not published NVFP4 efficiency factors that we could find.[U]

9.3.2 An empirical slope from NVIDIA's MXFP4 versus NVFP4 comparison

NVIDIA's 8B experiment gives a second, independent handle (Chapter 8): at 1T tokens the relative loss errors against BF16 are about 2.5% for MXFP4 and 1.5% for NVFP4, and MXFP4 reaches NVFP4's loss with $1.36\times$ the tokens [3]. If the two loss curves differ by a pure shift in tokens, a 1% difference in loss equals a factor of 1.36 in tokens, so the local slope of the loss in log-log is

$$\left|\frac{d\ln L}{d\ln D}\right| \approx \frac{0.01}{\ln 1.36} \approx 0.033 \tag{9.4}$$

at that operating point. A doubling of the data lowers the loss by only about 2.3%: the loss is already flat in D, which is the regime of a model trained at hundreds of tokens per parameter. This is our arithmetic on two approximate reported figures; it relies on the assumption of parallel curves and on reading "about 2.5% and 1.5%" as exact.[D][U]

With a Chinchilla-like decomposition $L = E + R\,D^{-\beta}$ and $\rho = (L-E)/L$ the reducible fraction of the loss, the slope is $\beta\rho$, so (9.4) implies $\beta\rho \approx 0.033$: for $\beta = 0.3$, $\rho \approx 0.11$, i.e. about 90% of the loss is irreducible or only slowly reducible in this regime. A loss gap g is then equivalent to a data penalty

$$\kappa = \left(1 + \frac{g}{\rho}\right)^{1/\beta}, \tag{9.5}$$

because a method with a pure data-efficiency penalty has $L_q(D) = L(D/\kappa)$, i.e. $R(\kappa^\beta - 1) = gL$. Table 9.2 evaluates (9.5). In the regime implied by (9.4) ($\rho \approx 0.1$–0.11 for $\beta = 0.3$), a 1.5% gap is worth about $1.6\times$ the tokens; in a more reducible regime ($\rho = 0.6$) the same gap is only about 9%.[D]

		token-equivalence factor κ of the FP4 loss gap			
β	ρ	$g = 0.5\%$	1.0%	1.5%	2.5%
0.3	0.1	1.18	1.37	1.59	2.10
0.3	0.3	1.06	1.12	1.18	1.31
0.3	0.6	1.03	1.06	1.09	1.15
0.5	0.1	1.10	1.21	1.32	1.56
0.5	0.3	1.03	1.07	1.10	1.17
0.5	0.6	1.02	1.03	1.05	1.09

Table 9.2: Token-equivalence factor κ of a relative loss gap g from (9.5), for reducible-loss fraction ρ and data exponent β. These are evaluations of a formula for illustrative parameter values, not fitted quantities.[D]

		κ (extra tokens needed)			
ϕ	s	1.00	1.10	1.25	1.60
0.50	1.5	1.20	1.09	0.96	0.75
0.50	2.0	1.33	1.21	1.07	0.83
0.50	3.0	1.50	1.36	1.20	0.94
0.70	1.5	1.30	1.19	1.04	0.82
0.70	2.0	1.54	1.40	1.23	0.96
0.70	3.0	1.88	1.70	1.50	1.17
0.85	1.5	1.40	1.27	1.12	0.87
0.85	2.0	1.74	1.58	1.39	1.09
0.85	3.0	2.31	2.10	1.85	1.44

Table 9.3: Net time-to-quality speed-up S_{net} for an FP4 share ϕ of baseline step time, GEMM speed-up s, and token penalty κ. Illustrative arithmetic; ϕ and s are not measured.[D]

9.4 Economics: when does the gap eat the speed-up?

A rough but useful model: suppose a fraction ϕ of the baseline step time is spent in GEMMs that move to FP4 and that these run $s\times$ faster than in the baseline format, with the remaining time $(1-\phi)$ unchanged (Amdahl's law). The step time becomes $(1-\phi)+\phi/s$. If the FP4 run needs $\kappa\times$ the tokens to reach the baseline loss, the net time-to-quality speed-up is

$$S_{\text{net}} = \frac{1}{\kappa\left[(1-\phi)+\phi/s\right]}. \tag{9.6}$$

Table 9.3 tabulates it. Three observations: (i) with no gap ($\kappa = 1$) and $s = 2$ the net speed-up is bounded by the FP4 share, e.g. $1.54\times$ at $\phi = 0.7$; (ii) a gap of $\kappa = 1.25$ cuts that to $1.23\times$ and $\kappa = 1.6$ eliminates the gain at $\phi = 0.7, s = 2$ (0.96); (iii) the higher s and ϕ, the more a token penalty matters in absolute terms but the more it can be absorbed.

What are plausible values of s? The FP8 and FP4 peak rates are vendor figures; DeepSeek-V4's report states that on current hardware FP4×FP8 products have the same peak FLOPs as FP8×FP8 [16], so FP4 only helps when both operands are FP4. Graphcore reports a measured $+21.2\%$ model-body token throughput from removing RHT and the BF16 final-block exemption in a native NVFP4 run [14], which shows that the *overheads of the recipe*, not just the GEMM peak, determine ϕ and s; the kernel-level and end-to-end speed-ups reported by TetraJet-v2 ($1.67\times$ over FP8, measured on RTX 5090-class hardware in the paper's appendix tables) and Quartet II (up to $4.2\times$ over BF16 for kernels) come from kernel benchmarks and small models, not from frontier-scale training [5, 6].[S] NVIDIA's blog claims a $7\times$ GEMM speed-up on GB300 relative to Hopper [32] and the MLPerf v5.1 results give a Llama 3.1

405B time of 10 minutes on 5120 GPUs with NVFP4 (final iterations in FP8) [34], but neither isolates NVFP4's contribution against an FP8 run on the same hardware.[V]

The conclusion is not that FP4 pretraining is uneconomical, but that its net benefit depends on three quantities that are not jointly reported by any single source we found: the loss gap at the target scale (translated to κ through a local slope), the GEMM share ϕ, and the speed-up s *including* all recipe overheads (rotations, high-precision layers, re-quantisation). The first two lines of Table 9.3 show that a recipe that closes the gap (from $\kappa \approx 1.6$ towards 1.1) is worth more than doubling s.[D]

9.5 Summary

- To second order, quantisation noise costs $\frac{1}{2}\operatorname{tr}(\boldsymbol{H}\boldsymbol{\Sigma})$ in loss: linear in relative MSE, weighted by curvature, so sensitive layers should stay in higher precision, and weights (which adapt) are more tolerant than activations.[D]
- Published Quartet fits are consistent with $\ln(1/\mathrm{eff}_N) \propto \mathrm{MSE}$.[D][S]
- NVIDIA's MXFP4/NVFP4 comparison implies a flat loss curve at 8B/1T ($d\ln L/d\ln D \approx -0.033$), in which a 1.5% loss gap is worth roughly $1.6\times$ the tokens.[D][U]
- Net speed-up is the FP4 GEMM share and speed-up discounted by κ; no source reports all three quantities for one setting.[D]

Chapter 10

Inference: Post-Training Quantisation, QAT and Distillation

The first public deployment of NVFP4 was for inference, and for most practitioners that is still the relevant use. This chapter separates three situations that the popular discussion often conflates: (i) *post-training quantisation* (PTQ) of an existing checkpoint, (ii) *quantisation-aware training or distillation* (QAT/QAD) that continues training with 4-bit forward passes, and (iii) models that are *trained natively* for a 4-bit deployment format. The experiments in Chapter 13 bear on (i) only, at small scale; everything about (ii) and (iii) is taken from the literature.

10.1 What PTQ to NVFP4 has to do

In weight-and-activation (W4A4) inference both operands of every linear layer are block-quantised, so the error model of Chapter 4 and Chapter 7 applies twice per GEMM. Weight quantisation is done once and can be expensive; activation quantisation is on the critical path and must be a cheap absmax computation (or a fused variant of it). Two structural facts follow from the earlier chapters.

1. **Weights admit search, activations do not.** Choices such as the Four Over Six rule (Section 4.9), MSE-optimal block scales, GPTQ-style error compensation and rotations can be applied offline to weights. For activations the quantiser runs per token at serving time, so only the closed-form operations (absmax scale, optionally 4/6 selection, optionally a fixed Hadamard rotation) are affordable.[D]

2. **The relevant error is not weight MSE alone.** By the sequential-error picture of Section 9.1 the output perturbation is a sum of weight-induced and activation-induced terms; improving only one leaves the other. In our real-weight experiment (Chapter 13) relative weight MSE falls by 15.7% when 4/6 selection is used, and the increase in log-perplexity over the unquantised model (W4A16, SmolLM2-135M) falls by 15.1%, almost the same fraction, as the second-order model of Section 9.1 predicts for a uniform error reduction.[E][D]

10.2 Reported PTQ accuracy of NVFP4

10.2.1 Large reasoning models (vendor)

The original NVFP4 announcement reports PTQ of DeepSeek-R1-0528 from FP8 to NVFP4 with accuracy changes of at most about one point on most benchmarks (Table 10.1) [4].[V] The values are read

Table 10.1: DeepSeek-R1-0528, FP8 versus NVFP4 PTQ, as reported by NVIDIA [4] (vendor, read from chart labels).

Benchmark	FP8	NVFP4	Difference
MMLU-Pro	85	84	−1
GPQA Diamond	81	80	−1
Humanity's Last Exam	15	14	−1
LiveCodeBench	77	76	−1
SciCode	40	40	0
MATH-500	98	98	0
AIME 2024	89	91	+2

Table 10.2: NVFP4 PTQ of very large models without fine-tuning (Table 12 of Xin et al. [47]). MATH-500 / AIME24 / GPQA-Diamond; GSM8K is available for DeepSeek-R1 only.

Model	Method	MATH-500	AIME24	GPQA-D	GSM8K
Llama Nemotron Ultra V1 (253B)	BF16	96.6	75.0	75.7	–
	NVFP4 PTQ	96.2	76.0	74.8	–
DeepSeek-R1 (671B)	Official FP8	95.4	80.0	69.7	96.3
	NVFP4 PTQ	94.2	80.0	69.2	96.1

from chart labels in the blog and are integers. The post gives no sample sizes or run-to-run variance and the AIME 2024 result, two points *higher* for NVFP4, is a reminder that single-run differences of this size on a 30-problem benchmark are noise.[U]

The NVIDIA quantisation-aware-distillation report adds two further no-fine-tuning PTQ results for very large models (Table 10.2) [47].[S] Its authors state the empirical finding that larger models are more robust to quantisation. The same report shows that this does not hold for smaller reasoning models (below).

10.2.2 Open-source models of many sizes (Red Hat)

A Red Hat article reports PTQ with the LLM Compressor library across dense and mixture-of-experts models, evaluated on OpenLLM v1 and v2, HumanEval and MMLU [20].[V] Accuracy recovery relative to BF16, as summarised there, depends on size: about 99% for 70B–235B models, 97–99% for models near 30B, and about 95–98% for 7B–14B models, with Llama-3.1-8B degrading more than Qwen3-8B and -14B (which are close to 98%). MMLU is typically within 3–4 points of BF16 across eleven variants, and HumanEval within one to two points for most models.[V] Most figures are approximate and read from charts; the article reports storage savings (about 1.5–1.8× against FP8 and 3× against FP16) but no speed data. We record the size trend, which agrees with the QAD report, and treat the percentages as indicative.[U]

10.2.3 Controlled comparison of PTQ methods: MR-GPTQ

The most careful comparison of PTQ for 4-bit block formats in our source set is the MR-GPTQ study of Egiazarian et al. [12]. The paper evaluates MXFP4, NVFP4 and integer variants for W4A4 quantisation of the Llama-3 and Qwen3 families, with GPTQ-style weight compensation and block-wise rotations, and reports average accuracy recovery over a task suite. Table 10.3 reproduces the Llama-3.1-8B row group.[S]

Table 10.3: W4A4 accuracy recovery (percent of BF16 average) for Llama-3.1-8B, from Table 1 of Egiazarian et al. [12]. RTN: round to nearest; HT: Hadamard transform; NVINT4: INT4 elements with an E4M3 scale per 16 values.

Format	RTN	MR-GPTQ
NVFP4	94.67	96.08
MXFP4	87.83	93.31
NVINT4	–	97.12
INT4 (group size 32), RTN with HT	94.71	–

Table 10.4: WikiText-2 word perplexity under W4A4 PTQ (Tables 3 and 4 of Cook et al. [7]). "4/6 (MSE)" selects between scale targets 4 and 6 per block by squared error; the other two rules use mean absolute error and maximum error.

	Llama 3			Qwen 3		
	1B	8B	70B	1.7B	8B	32B
BF16	11.98	7.54	2.86	21.06	12.22	9.34
MXFP4	17.67	9.66	5.53	26.95	14.08	11.92
NVFP4, target 6 (default)	14.27	8.43	4.00	23.06	12.68	9.85
NVFP4, target 4 for all blocks	14.75	8.63	4.48	24.43	12.98	10.57
+ 4/6, MSE rule	13.84	8.30	3.83	23.60	12.56	9.84
+ 4/6, MAE rule	13.94	8.33	3.86	23.45	12.63	9.82
+ 4/6, max-error rule	14.06	8.36	4.39	23.32	12.86	9.97

Four points are relevant here. First, in this table NVFP4 RTN recovers 94.7% against 87.8% for MXFP4 RTN: the gap that our synthetic experiments predict from the scale format and block size (Chapter 4) is reproduced on real models.[S] Second, the best 4-bit result is not FP4: NVINT4 with their method reaches 97.1%, in line with the INT-versus-FP analysis in Section 4.8.[S] Third, the authors state that run-to-run variance is about 0.3 points for NVFP4 and about 1 point for INT4, so differences below that are not informative.[S] Fourth, they report that real kernels track their simulation within 0.2–0.3% and that Qwen3 models reach more than 99% recovery with NVFP4, again suggesting a strong dependence on model family.[S]

10.2.4 Four Over Six in PTQ

Section 4.9 derived the idea of choosing, per block, between the scale that maps the block maximum to 6 and one that maps it to 4. Cook et al. [7] report PTQ results on Llama-3 and Qwen3 (WikiText-2 and C4 word perplexity, W4A4). Their ablation of the rule is informative (Table 10.4): scaling *every* block to 4 is worse than the default; picking by MSE was best in most cases; picking by maximum error was clearly worst for Llama-3-70B (4.39 against 3.83 for MSE) though not for Qwen3-1.7B.[S]

Two observations deserve emphasis. The 4/6 rule is not uniformly beneficial: for Qwen3-1.7B the MSE variant is *worse* on WikiText-2 (23.60 against 23.06), and on C4 it is worse for the same model (66.32 against 65.54). For the Llama models it closes 18.8% (1B), 14.6% (8B) and 14.9% (70B) of the WikiText-2 perplexity gap to BF16 (computed from the table), close to the 15.7% reduction in weight MSE we measure for 4/6 on real weights (Chapter 13), which is the behaviour the second-order loss model predicts if the error reduction is uniform across layers.[D][S]

Table 10.5 shows the interaction with other PTQ methods. With AWQ and SmoothQuant the rule improved perplexity for every model in the table; with GPTQ it helped slightly on Llama-3 but hurt on

Table 10.5: 4/6 combined with PTQ algorithms, WikiText-2 word perplexity (Table 5 of Cook et al. [7]; a dash means not run).

	Llama 3			Qwen 3		
	1B	8B	70B	1.7B	8B	32B
RTN	14.27	8.43	4.00	23.06	12.68	9.85
RTN + 4/6	13.84	8.30	3.83	23.60	12.56	9.84
GPTQ	13.73	8.33	–	21.48	12.50	9.67
GPTQ + 4/6	13.67	8.30	–	22.70	12.65	9.66
AWQ	14.04	8.33	3.86	22.20	12.68	9.69
AWQ + 4/6	13.67	8.24	3.71	21.67	12.57	9.64
SmoothQuant	14.17	8.38	3.86	21.99	12.64	9.65
SmoothQuant + 4/6	14.03	8.32	3.80	21.97	12.62	9.63

two of the three Qwen3 models, and the authors report an average *increase* of 34.6% in the gap between NVFP4 and BF16 word perplexity for GPTQ plus 4/6, noting that GPTQ's optimisation was not adapted to the 4/6 rule.[S] This illustrates a general caveat: a quantiser change that lowers per-tensor error can interact with error-compensating algorithms that implicitly assume a fixed rounding map.

10.3 Why PTQ degrades on reasoning models: QAT versus QAD

Plain PTQ is a one-shot rounding of a model that was never exposed to the 4-bit grid. Continuing training with quantised forward passes (QAT) is the textbook remedy. NVIDIA's QAD report argues that for modern post-training pipelines QAT is the wrong tool, and proposes distillation instead [47].[S]

10.3.1 The method

QAT keeps the original task loss (for language models, next-token cross entropy on SFT or RL data) and runs the forward pass in 4-bit. QAD replaces the loss by the Kullback–Leibler divergence from the full-precision teacher to the quantised student,

$$\mathcal{L}_{\mathrm{QAD}} = D_{\mathrm{KL}}\big(p_{\text{teacher}}(\cdot \mid x) \,\big\|\, p_{\text{student}}(\cdot \mid x)\big) = \sum_{y \in V} p_{\text{teacher}}(y \mid x) \log \frac{p_{\text{teacher}}(y \mid x)}{p_{\text{student}}(y \mid x)}, \tag{10.1}$$

so that the target is the teacher's full output distribution rather than the label.[S] The key argument is that a model that has been through SFT, RL and merging encodes behaviour that a few billion tokens of task-loss training will not reproduce, so a task loss either does nothing or acts as an additional, unintended post-training stage. Their diagnostic experiment on Llama Nemotron Super V1 (about 0.3B training tokens, $5{,}000$ held-out samples) shows that QAT achieves cross entropy against labels essentially equal to the BF16 model while its KL divergence to the BF16 model is 0.311, against 0.004 for QAD.[S] Cross entropy on labels is therefore blind to the very drift that matters.

10.3.2 Results

Table 10.6 reproduces the SFT-heavy models and Table 10.7 the RL-heavy models. For the SFT-heavy Llama Nemotron Super V1, QAD recovers AIME25 from 32.3 (PTQ) to 45.6 against a BF16 value of 46.0, whereas QAT reaches 41.5. For Nemotron Nano V2 the differences are small except on GPQA-Diamond (59.0 PTQ, 56.9 QAT, 62.7 QAD against 64.0 BF16). For RL-heavy models the effect is

Table 10.6: QAD versus QAT on SFT-heavy models (Table 2 of Xin et al. [47]).

Model	Method	MATH-500	AIME25	GPQA-D	IFEval-Instr.
Llama Nemotron Super V1	BF16	95.8	46.0	66.5	87.5
	NVFP4 PTQ	91.4	32.3	62.1	86.9
	NVFP4 QAT	94.3	41.5	63.3	87.2
	NVFP4 QAD	94.6	45.6	64.5	87.8
Nemotron Nano V2	BF16	97.8	71.1	64.0	90.3
	NVFP4 PTQ	97.2	69.8	59.0	89.8
	NVFP4 QAT	97.2	67.1	56.9	86.2
	NVFP4 QAD	97.2	71.5	62.7	89.3

Table 10.7: QAD versus QAT on an RL-heavy model, Nemotron 3 Nano 30B-A3B (Table 3(a) of Xin et al. [47]). AA-LCR: Artificial Analysis long-context reasoning; LCB: LiveCodeBench v5.

Method	AA-LCR	AIME25	GPQA-D	LCB-v5	SciCode
BF16	35.9	89.1	73.0	72.1	33.0
NVFP4 PTQ	31.3	85.0	71.6	68.9	30.5
NVFP4 QAT	24.8	83.3	66.0	62.0	25.8
NVFP4 QAD	34.3	87.9	72.7	68.9	32.3

stronger: on Nemotron 3 Nano QAT is *worse* than PTQ on every benchmark shown (for example AIME25 83.3 against 85.0, and LiveCodeBench-v5 62.0 against 68.9), while QAD is within about 1.2 points on AIME25 and recovers LiveCodeBench-v5 to the PTQ level of 68.9 but not to BF16 (72.1).[S]

Two further results from the same report matter for practice. QAD is insensitive to the data used: a mixture of SFT data and BF16-generated outputs for RL prompts performed slightly better than either alone, but all three choices gave comparable results (their Table 11), and the loss only needs inputs, not labels, so unlabeled or synthetic prompts are admissible.[S] And the evidence is entirely from NVIDIA models on NVIDIA's own benchmarks, with single runs; there is no independent replication in our source set.[U]

10.3.3 Reading the QAT/QAD evidence

Three interpretations are consistent with the tables. (a) The failure of QAT on RL-heavy models is a *distribution-shift* effect, as the authors argue: the RL policy is a point in a narrow region of behaviour space, and a cross-entropy fine-tune moves it. (b) It is partly an artefact of the data: if QAT used on-policy samples from the RL model, the cross-entropy objective would be closer to a sampled forward-KL objective and might behave like QAD. The report notes that QAD with model-generated data works; it does not test QAT with on-policy data. (c) The gap between PTQ and BF16 may in part be noise on small benchmarks such as AIME25 (30 problems), where one problem is 3.3 points. We cannot separate these with the available data.[U]

10.4 PTQ in the production recipe of Nemotron 3 Super

Nemotron 3 Super, a 120.6B-parameter (12.7B active) hybrid model pretrained in NVFP4 (Chapter 17), also ships a PTQ checkpoint in NVFP4 whose recipe is documented [15]. The technical report describes: weight scales chosen to minimise MSE and activation scales from maximum values; a mixed assignment

Table 10.8: Nemotron 3 Super checkpoints in three precisions on six benchmarks (selected columns of Table 8 in NVIDIA [15]).

Checkpoint	MMLU-Pro	HMMT	GPQA	LCB v6	RULER-1M	MMLU-ProX
BF16	83.73	94.73	79.23	78.69	91.64	79.35
FP8	83.63	94.38	79.36	78.44	91.43	79.21
NVFP4	83.33	95.36	79.42	78.44	91.60	79.37

found by an AutoQuantize search with a target of 4.75 average bits per element, so that sensitive layers stay in higher precision; and a per-operator choice among NVFP4, FP8 and BF16. In the searched model all sparse-expert GEMMs are NVFP4 while attention and Mamba projection GEMMs are FP8 or BF16, so the checkpoint is mostly, not entirely, FP4; the whole search-plus-quantisation took under two hours on one eight-GPU B200 node with 512 SFT samples of length 4096.[S] The report states a median accuracy of 99.8% of BF16 across its benchmark suite. Table 10.8 shows six benchmarks from the report's comparison of the BF16, FP8 and NVFP4 checkpoints. The three precisions are within the typical noise of such benchmarks: the largest NVFP4–BF16 differences in the table are $+0.63$ (HMMT) and -0.40 (MMLU-Pro).[S]

A second observation from the same report concerns *state* rather than weights. In hybrid models with state-space (Mamba) layers, the recurrent cache can be stored at reduced precision, and the report finds that an FP16 cache leads to longer, more verbose generations (an increase of roughly 37–40% in generated length in its test) relative to the default FP32 cache (up to 40% when combined with W8A8 weights and activations, and up to 37% even with BF16 weights and activations). The authors' training-free fix was an INT16 cache with an FP32 scale per block of 128 along the state dimension, which removed the verbosity increase. This illustrates that low-precision effects can appear in output length rather than in accuracy.[S] We mention it because it is a failure mode of *non-GEMM* low precision that format studies like ours do not cover.

10.5 What PTQ cannot fix

The following limits recur in the sources.

- **Sensitivity to training history.** A third-party NVFP4 PTQ of a 744B-parameter model was served publicly for a few days by a company blog, which reports good throughput but, according to anecdotal user reports, degraded long-horizon agentic coding behaviour (collapsing chains of thought, more tool-rule violations); the vendor model card of that checkpoint reportedly recommended the FP8 version for production [21].[U] The author's thesis is that the model had not been trained for the target precision and that QAT-trained models do not show the problem. This is a company blog, with no controlled measurements, and we classify it as an unverified hypothesis that is nonetheless consistent with the QAD evidence above.
- **Small models.** The size trend in the Red Hat and QAD data is consistent with the quantisation noise sum of Chapter 7: with fewer parameters per layer there is less averaging, and sensitive layers weigh more.[D][U]
- **Benchmark-dependent damage.** PTQ damage is largest on hard reasoning (AIME-class) and code tasks, small on knowledge tasks (MMLU-class), consistent with long outputs amplifying small per-token distribution shifts.[U]

10.6 Summary

At large scale, with careful scale selection and (for the best results) rotation and error compensation, 4-bit weights and activations give results within roughly one to a few percent of the higher-precision model on standard benchmarks. At small scale or for models with heavy RL post-training, plain PTQ loses visibly more, and distillation-based QAT recovers most of it on the evidence of a single group. NVINT4 outperforms NVFP4 once rotation is used in the one controlled comparison we have. None of these results transfer to training numerics without a separate argument, which is the subject of Chapter 8.

Chapter 11

Hardware, Kernels and Systems

Numerical formats are only useful if hardware executes them faster than the alternative. This chapter collects what is publicly known about the throughput, memory and software support of NVFP4 and MXFP4, and adds a simple roofline-type model that shows why measured end-to-end speed-ups are smaller than peak-FLOP ratios. Hardware figures are vendor claims ([V]) unless stated. We had no access to Blackwell hardware, so nothing in this chapter is measured by us.

11.1 Peak throughput and memory

11.1.1 Published peak numbers

NVIDIA's NVFP4 blog gives peak dense and sparse throughput for the smallest supported data type on each generation, in petaFLOP/s [4]: A100 $0.3/0.6$, H100 $1.9/3.9$, B200 $9/18$, B300 $13/18$, GB200 $10/20$, GB300 $15/20$.[V] The chart does not state the precision for each bar, and A100 and H100 numbers are INT8 and FP8 respectively, so the comparison is across different formats. A separate blog on Blackwell Ultra gives, for the dense case, 15 PFLOP/s NVFP4 against 5 PFLOP/s FP8 (and Blackwell at 10 and 5), and states 288 GB of HBM3E at 8 TB/s, a power limit of up to 1,400 W, and, for the attention softmax exponential unit, 10.7 against 5 tera-exponentials per second for Blackwell [33].[V] The NVFP4-to-FP8 dense ratio is therefore $2\times$ on Blackwell and $3\times$ on Blackwell Ultra, matching the MLPerf Training post, which says peak FP4 throughput per clock is $2\times$ FP8 on Blackwell and $3\times$ on Blackwell Ultra [34].[V] The same post warns of a mixed-precision comparison: the quoted "$7.5\times$ over Hopper" divides Blackwell Ultra's NVFP4 number by Hopper's FP8 number.[V]

11.1.2 Memory footprint

The bits-per-element figures follow from the definitions in Chapter 3: $4 + 8/16 = 4.5$ for NVFP4 and $4 + 8/32 = 4.25$ for MXFP4, plus an FP32 per-tensor scale that is negligible for large tensors.[D] Against FP16 and FP8 this gives compression ratios of $16/4.5 = 3.56$ and $8/4.5 = 1.78$, in agreement with the vendor's "about $3.5\times$ against FP16 and $1.8\times$ against FP8" [4].[D][V] Red Hat reports about $3.3\times$ against BF16 for Qwen3-235B, because not all tensors are quantised [20].[V] A blog that served a 744B-parameter model reports roughly 744 GB in FP8 against 420 GB in NVFP4, a ratio of 1.77, and derives a decode bandwidth headroom of about $1.8\times$ from the byte counts while noting that the unquantised shared experts reduce it [21].[U]

With 2D 16×16 weight scaling (Section 3.3.2) the number of scale factors for a weight matrix is $1/256$ rather than $1/16$ of the elements, which would in principle bring a compactly stored weight to

Table 11.1: Vendor-reported dense peak throughput and memory (PFLOP/s; GB; TB/s). Sources: Alvarez et al. [4], NVIDIA [33]. All values are peak theoretical numbers.

	FP8 dense	NVFP4 dense	NVFP4/FP8	HBM (GB)	Bandwidth (TB/s)
H100	1.9–2	–	–	80	–
B200 / GB200	5	9–10	2	192	–
B300 / GB300	5	13–15	3	288	8

$4 + 8/256 \approx 4.03$ bits. However, the hardware tensor-core operand layout expects one scale per 16 elements along the contraction dimension, so a 2D-scaled weight is most naturally stored with the scale replicated, and the storage is the same 4.5 bits.[D][U] We have not seen a source that states which layout deployed kernels use.

11.2 Kernel-level constraints from the Transformer Engine

The Transformer Engine (TE) user guide documents the production training implementation [35].[S] The following details matter for the numerics and for performance.

- The per-tensor scale is $s_{\text{global}} = \text{amax}/(448 \cdot 6)$ and the block scale is $(\text{amax}_b/6)/s_{\text{global}}$, which is the encode scale used throughout Chapter 4 (up to the reciprocal convention).
- The recipe flags allow independent switching of 2D weight scaling, stochastic rounding (gradients only) and the RHT (Wgrad inputs only). The RHT supports BF16 inputs only. These are the same three ingredients ablated in the literature of Chapter 8.
- The NVFP4 GEMM supports only the TN layout, unlike the MXFP8 GEMM. Row-wise and column-wise copies of an operand are stored separately, with the column-wise data and its scales stored transposed; this is why the 2D weight scaling, which makes the two copies numerically equal, is attractive: a weight can be quantised once and transposed.[S][D]
- Scale tensors are padded (first dimension to a multiple of 128, second to a multiple of 4) and must be swizzled into the layout the tensor core reads before the GEMM.
- The global amax must be synchronised across ranks for gathered tensors such as sequence-parallel inputs, whereas block scales need no communication. Quantised all-gather is supported if all ranks share the same global scale.
- The hardware requirement is SM 10.0 or SM 10.3 for training and SM 10.0 or later for inference.

The amax synchronisation is a numerical, not just an engineering, constraint. Because the per-tensor scale determines the headroom of the E4M3 block scales (Section 4.7), using a local rather than global amax would change the dynamic range available on each rank and break consistency of weights across a data-parallel group. The documentation's insistence on a common global scale is consistent with this reading.[D][U]

11.3 A roofline model of the quantisation overhead

Peak FLOP ratios ignore the work of producing 4-bit operands. A BF16 activation tensor must be read, reduced over each block to find absmax, scaled, rounded and written together with its scales. This is a memory-bound elementwise operation. We estimate its cost relative to the GEMM that consumes it.

Table 11.2: Roofline estimate of the quantisation overhead for one forward GEMM, using vendor peak numbers for Blackwell Ultra ($F_4 = 15$, $F_8 = 5$ PFLOP/s, $B = 8$ TB/s). T_q/T_g: standalone quantisation time over GEMM time. Speed-ups are FP8 time over FP4 time, with both operands quantised by a separate, unfused kernel ("unfused"), or with the FP4 quantisation fused into the producer so that its cost vanishes ("fused FP4 vs unfused FP8"). Derived from the formulas in the text; no measurement.

N	T_q/T_g (FP4)	T_q/T_g (FP8)	FP8/FP4 peak ratio	unfused/unfused	fused FP4 / unfused FP8
2048	1.17	0.46	3.0	2.01	4.37
4096	0.59	0.23	3.0	2.32	3.69
8192	0.29	0.11	3.0	2.59	3.34
16384	0.15	0.06	3.0	2.77	3.17
32768	0.07	0.03	3.0	2.88	3.09

Let the activation be $M \times K$ and the weight $K \times N$ with M large enough that the GEMM is compute-bound. Let F be the dense peak rate of the operand format, B the memory bandwidth, and $b_{\text{in}}, b_{\text{out}}$ the bytes read and written per element by the quantiser. Then

$$\frac{T_{\text{quant}}}{T_{\text{gemm}}} = \frac{MK(b_{\text{in}} + b_{\text{out}})/B}{2MKN/F} = \frac{(b_{\text{in}} + b_{\text{out}})\,F}{2NB}. \tag{11.1}$$

For NVFP4, $b_{\text{in}} = 2$ (BF16) and $b_{\text{out}} = 0.5625$ (the 4.5 bits plus negligible global scale), and for FP8, $b_{\text{out}} = 1$. With the Blackwell Ultra numbers $F_4 = 15$ PFLOP/s, $F_8 = 5$ PFLOP/s and $B = 8$ TB/s, the ratios are $2402/N$ for FP4 and $937/N$ for FP8.[D] Table 11.2 gives the consequences for the forward GEMM of one linear layer.

The model says that for a hidden size of 4096 an unfused FP4 quantiser costs 59% of the GEMM time, so that an unfused implementation delivers 2.3× over FP8 rather than the 3× peak ratio, while for $N = 2048$ it delivers 2.0×.[D] This is why activation quantisation is fused into the producing kernel or the GEMM prologue in practice (the NVIDIA research blog describes folding activation quantisation into the GEMM prologue to avoid an extra HBM round trip [48][V]). The column "fused FP4 vs unfused FP8" exceeds 3 because it credits FP4 with a free quantiser; it is an upper bound only and we include it to show that the *reported* ratios between a tuned FP4 stack and a naive FP8 stack can legitimately exceed the peak ratio without any numerical difference.[D]

The model neglects (a) the quantisation of weights (amortised over M tokens), (b) the RHT, which for Wgrad adds a 16×16 matrix product on both operands (Chapter 6) and is itself memory-bound, (c) the transposed copies required for Dgrad and Wgrad, which double the quantisation traffic for backward operands, and (d) the BF16 layers, normalisations, attention and communication, which are not accelerated at all. Item (d) is Amdahl's law, already the basis of the net-speed-up model in Section 9.4.[D]

11.4 Reported speed-ups

Table 11.3 lists the throughput claims we found. They differ in what is measured, and only the last group is end-to-end training throughput.

Three comments. First, the NVIDIA MLPerf post says its 3× figure over Hopper at equal GPU count combines NVFP4 with other software gains, so it is not attributable to the format alone, and it describes "healing" by finishing training iterations in FP8 [34].[V] This healing practice is the same device used in the 12B experiment of Chapter 8. Second, the research blog's lifecycle numbers (about 1.9× for training and up to about 3.0× for inference against FP8) are vendor estimates for specific models. Third, Quartet II's and the MR-GPTQ numbers are against BF16 or FP16 and TetraJet-v2's against FP8, so

Table 11.3: Reported speed-ups attributed to 4-bit formats. "Scope" is the quantity measured.

Claim	Scope	Source and tag	Comparison
7× GEMM, GB300 vs Hopper	kernel	NVIDIA blog [32][V]	mixed precision
2× (GB200), 3× (GB300) FP8 GEMM rate	kernel / peak	newsletter summary [49][U]	FP8
Llama-3.1-405B pretraining ≈ 1.9× FP8	lifecycle	NVIDIA research blog [48][V]	FP8
MLPerf 405B 2,560 GPUs: 18.79 min, 3× Hopper	benchmark	[34][V]	incl. software changes
Quartet II linear layers: > 4× (RTX 5090), up to 2.5× (B200, 22B-scale shapes)	isolated linear layers	[5][S]	BF16
Quartet II 1B-model pretraining > 1.8×	training throughput	[5][S]	BF16
MR-GPTQ inference kernels: up to 3.6× layer-wise and 2.2× end-to-end (B200); 6× and 4× (RTX 5090)	inference, layer and end-to-end	[12][S]	FP16
TetraJet-v2 1.67×	end-to-end	[6][S]	FP8
Graphcore: +21.2% model-body tokens/s (3,212 to 3,894)	ablation: no RHT, all linears FP4	[14][S]	TE NVFP4 recipe
Inference peak up to ≈ 3× vs FP8	serving	NVIDIA research blog [48][V]	FP8
llama.cpp prefill +68% between two builds	prefill	third-party measurement via [36][U]	same file, other build

they cannot be compared with each other, and the Graphcore number compares two *recipes* (with and without the Hadamard transform and BF16 final blocks) rather than two formats. We did not find an independent measurement of end-to-end NVFP4 pretraining throughput on Blackwell hardware in the source set.[U]

11.5 Software ecosystem

11.5.1 Frameworks

NVFP4 is supported for inference in TensorRT-LLM and vLLM, with SGLang support described as upcoming in mid-2025 [4], and for training by the Transformer Engine [35].[V][S] The llama.cpp project assigned a type identifier to NVFP4 and merged CUDA, SYCL and Vulkan kernels between March and April 2026, with a Blackwell-native path following in late April, according to a web article that also carried later corrections [36].[U] The article's initial claim that NVFP4 files are about 25% smaller than the Q4_K_M format was corrected to about 7%; a representative Q4_K_M file loaded at 4.83 bits per weight, so the nominal 4.5 bits of NVFP4 does not translate into a large saving against a mixed-bit-width 4-bit GGUF format.[U] The same article states that no independent benchmark suite had compared

NVFP4 and Q4_K_M GGUFs for quality, so claims of quality parity there are unsupported at the time of writing.[U]

11.5.2 Other hardware

Non-NVIDIA hardware is mostly MX-aligned. The OCP MX specification defines MXFP4 with E8M0 scales and block size 32 [2], and OpenAI's gpt-oss models (Chapter 17) are distributed in MXFP4 at 4.25 bits per parameter, with 60.8 GiB for the 120B model and 12.8 GiB for the 20B model [17].[S] AMD's study of MXFP4 pretraining on native-FP4 hardware is discussed in Chapter 8 [46]. Huawei's HiFloat4 uses a three-level scaling hierarchy on Ascend NPUs (Chapter 14). DeepSeek-V4 states that FP4×FP8 operations currently have the same peak FLOP/s as FP8×FP8 on existing hardware and could theoretically be $1/3$ more efficient on future hardware, so the model's FP4 routed experts currently buy memory, not compute, in its stack [16].[S]

11.6 Energy

NVIDIA's NVFP4 blog reports an energy-per-token chart for a GPT-MoE-1.8T model: 10 J per token on Hopper, 0.4 on Blackwell and 0.2 on Blackwell Ultra, interpreted as up to 25× and 50× improvements.[V] The post credits FP4 support together with liquid cooling and architectural changes and does not isolate the contribution of the number format, so these figures cannot be used as a measure of what 4-bit arithmetic alone saves.[V] We know of no independent measurement.

11.7 Summary

Peak throughput of 4-bit tensor cores is 2× (Blackwell) to 3× (Blackwell Ultra) that of FP8, and memory per weight is 1.78× smaller. The measurements of Quartet II support the roofline picture: its authors report that on a B200 (theoretical 4× over BF16) the smaller matrix shapes are "entirely dominated by the quantisation overhead" and real speed-ups appear only from about the 3B-parameter scale shapes, reaching 2.5× at 22B, while on an RTX 5090 (theoretical 8×) they reach more than 4× [5].[S] The roofline model of Section 11.3 indicates that the cost of producing 4-bit operands erodes a third or more of the peak gain at hidden sizes of a few thousand unless it is fused, and Amdahl effects from the unaccelerated parts remove more. The published end-to-end ratios are consistent with this: they are well below 3× where they are measured against a tuned FP8 baseline, and larger where the baseline differs. No source in our set provides an independent, controlled end-to-end training throughput measurement.

Chapter 12

An Audit of the Empirical Evidence

Earlier chapters reported numbers where they arose. This chapter steps back and asks how much weight the numbers can bear. It does three things: it puts the headline loss gaps from different papers into common units; it estimates how much of the downstream-benchmark differences in the flagship 12B comparison could be sampling noise; and it assembles a ledger of the main claims with the strength of the evidence behind each. Everything in this chapter other than the cited inputs is our own arithmetic ([D]) on reported figures.

12.1 Putting loss gaps into common units

Papers report the gap between a low-precision and a baseline run in four incompatible ways: relative training loss, relative validation loss, validation perplexity, and downstream accuracy. A relative loss gap $g = (L_q - L_0)/L_0$ depends on the absolute loss level L_0, which varies with tokenizer and data; a perplexity ratio corresponds to a difference in nats $\Delta = \ln(\mathrm{ppl}_q/\mathrm{ppl}_0)$ and is therefore tokenizer-specific but additive. We convert what is reported into both Δ (nats per token) and the relative gap $g = \Delta/L_0$, using $L_0 = \ln \mathrm{ppl}_0$ where only perplexities are given.

Three things follow. First, the quoted relative gaps are not interchangeable across studies even when they carry the same name: a "1% gap" is 0.02 nats at $L_0 \approx 2.27$ and 0.03 nats at $L_0 = 3.17$.[D] Second, the ranking of recipes by Δ is consistent with their ranking by g within each study, so the within-study comparisons in Chapter 8 do not depend on the normalisation. Third, the numbers show how much worse the less-polished recipes are in small-scale reimplementations than in the authors' own flagship run: the NVIDIA recipe as reimplemented in the TetraJet-v2 study is 3.2% in relative perplexity-implied loss on a 370M model, while the authors' own 12B run reports 1–1.5% [3, 6].[S] The difference may reflect scale, data and token horizon, or reimplementation differences; the sources do not allow us to separate them.[U]

12.2 Sampling noise in the downstream benchmarks of the 12B comparison

The flagship evidence for NVFP4 pretraining is the downstream comparison of Table 8.1, with a single run per precision. Even if both runs were identical models, finite test sets make benchmark scores noisy. For a benchmark with n items and accuracies p_a, p_b the standard error of the difference, treating the two

Table 12.1: Reported low-precision-versus-baseline gaps converted to nats per token (Δ) and relative gap (g). Baselines: BF16 for the first three groups, authors' FP8 where noted. Inputs from the cited tables; conversions are our arithmetic.

Study (scale)	Method	L_0	L_q	Δ (nats)	g (%)
Graphcore [14]	UE5M3 scale, TE-style	2.2651	2.2902	0.0251	1.11
(8B hybrid, final 250-step window)	proposed recipe	2.2651	2.3090	0.0439	1.94
	decoded-GEMM variant	2.2651	2.3157	0.0506	2.23
	block-32 variant	2.2651	2.3241	0.0590	2.60
	TE NVFP4 recipe	2.2651	2.3474	0.0823	3.63
	native, no RHT, no BF16 blocks	2.2651	2.4420	0.1769	7.81
Full-stack [9] (3B, 64B tok.)	NVFP4 full-stack	2.267	2.286	0.0190	0.84
TetraJet-v2 [6] (370M, val. ppl)	TetraJet-v2-full	3.165	3.223	0.0578	1.83
	TetraJet-v2-base	3.165	3.239	0.0732	2.31
	Quartet	3.165	3.264	0.0988	3.12
	NVIDIA recipe (as reimplemented)	3.165	3.266	0.1003	3.17

sets of answers as independent, is

$$\mathrm{SE}_\Delta = \sqrt{\frac{p_a(1-p_a)}{n} + \frac{p_b(1-p_b)}{n}}. \tag{12.1}$$

This overstates the error when the two models answer correlated sets of items (a paired analysis would be tighter), and it ignores training-seed variance, which is typically larger. We therefore use it only as a floor on what a difference must exceed to be distinguishable from test-set sampling.[D] The benchmark sizes are from our memory of the public test sets rather than retrieved from a source and may be off for the exact splits used in the report; they should be treated as approximate.[U]

Table 12.2 shows that for the knowledge and commonsense benchmarks (MMLU, MMLU-Pro, HellaSwag, PIQA, ARC, OpenBookQA, Winogrande) the differences are all within $z = \pm 1.6$ and the largest tasks (MMLU-Pro, $n \approx 12{,}000$) are consistent with equality to within 0.04 points. For code benchmarks the standard error is 3.6–5.4 points, larger than the observed 2.5–3.2 point deficits, so the code deficit that the authors flag cannot be distinguished from noise by this measure alone. Three entries have $|z| > 2$: GSM8k ($+3.2$, $z = 2.8$) and MGSM ($+3.7$, $z = 3.5$) favour NVFP4, and MATH (-1.8, $z = -2.4$) favours FP8. Under the null hypothesis of no difference one expects about 0.55 of twelve comparisons to exceed $|z| = 2$; three do. The mixed signs are not what a systematic NVFP4 deficit would produce, and sampling noise from the *training run* (a different random initialisation or data order) is not captured here at all. We conclude that the table supports "no large difference at the level of 2–4 points on these tasks", but that it cannot support any ranking of the two precisions on individual tasks.[D]

A second use of the same arithmetic is to say what a benchmark can detect. At 95% confidence a difference between two independent measurements of accuracy near 0.5 must exceed $1.96\sqrt{2p(1-p)/n}$: 10.8 points for $n = 164$ (HumanEval-sized), 7.1 for $n = 378$, 3.8 for $n = 1{,}319$ (GSM8k-sized), 2.0 for $n = 5{,}000$ and 1.3 for $n = 12{,}000$.[D] A 30-problem benchmark such as AIME has $n = 30$ per attempt and a floor of about ± 18 points for a single attempt; this is why the reasoning-model PTQ tables in Chapter 10, which report AIME-class numbers with sub-five-point differences, are best read as stable only when averaged over many samples, which the sources do not always state.[D][U]

Benchmark	n	FP8	NVFP4	diff.	SE (unpaired)	z
MMLU (5-shot)	14,042	77.36	76.57	-0.79	0.50	-1.57
MMLU-Pro (5-shot)	12,032	62.62	62.58	-0.04	0.62	-0.06
GSM8k CoT	1,319	89.08	92.27	+3.19	1.13	+2.82
MATH	5,000	83.32	81.48	-1.84	0.76	-2.42
MGSM	2,500	81.87	85.53	+3.66	1.04	+3.51
HumanEval+	164	59.93	57.43	-2.50	5.44	-0.46
MBPP+	378	59.11	55.91	-3.20	3.59	-0.89
ARC Challenge	1,172	91.81	91.81	+0.00	1.13	+0.00
HellaSwag	10,042	83.83	83.09	-0.74	0.52	-1.41
OpenBookQA	500	47.60	47.40	-0.20	3.16	-0.06
PIQA	1,838	82.64	82.70	+0.06	1.25	+0.05
Winogrande	1,267	80.58	78.77	-1.81	1.60	-1.13

Table 12.2: Test-set sampling noise for the downstream comparison of Table 8.1. Scores from NVIDIA [3]; item counts n are approximate and from memory ([U]); standard error and z are our computation ([D]). Benchmarks whose size we could not state with confidence are omitted.

12.3 A ledger of the main claims

Table 12.3 lists the principal empirical claims in the literature on NVFP4 and related formats as of October 2026, the strongest evidence for each, the degree of independent confirmation, and our verdict on how firmly it is established. "Independent" means a group other than the proposing authors, and "controlled" means the compared methods were trained under the same data, optimiser and token budget.

12.4 Recurrent weaknesses

Reading the sources against each other, the following weaknesses recur and should be kept in mind whenever a number from this report is used.

Single seeds and tiny scale of replication. Most pretraining results are single runs. The one study that measured repeat-run spread reports relative ranges across three NVFP4 runs of 0.7–9.8% on downstream metrics depending on the task [10].[S] At the loss level the spread is much smaller, which is why validation loss is the preferred metric for ablations, but it is the downstream numbers that appear in headlines.

Baselines are not shared. FP8 baselines are each group's own, BF16 baselines are of different quality, and recipes differ in more than the quantity under study (block size, scale format, rotation size, high-precision layers, schedule). Rows in a comparison table are therefore rarely differences of one factor.

Different training horizons. Gaps grow with training length (Chapter 9); studies at tens of billions of tokens do not bound the gap at trillions. Conversely, the single trillion-token-scale study (the 12B run) has the thinnest controls.

Evaluation in the wrong precision. Many studies evaluate low-precision-trained models in BF16, which can hide a train–inference mismatch; others evaluate in the training precision. A model trained

with RHT and 2D scaling and then deployed with a different PTQ path has not been tested in the way users will use it. We found no study that reports both for the same checkpoint.[U]

Metrics that hide the failure. The QAD report shows that cross entropy on labels can be indistinguishable between a model that is close to the teacher and one that has drifted (KL of 0.004 versus 0.311 with the same cross entropy). The same lesson applies to training: validation loss on held-out pretraining data may not reflect degradation on long chains of reasoning.[S]

Vendor and secondary sources. Hardware performance, energy and several accuracy numbers come from the manufacturer; some secondary articles repeat them without independent verification [51, 52]. We have marked these.

12.5 Summary

The quantisation-error mathematics is on firm ground, and it agrees with the real-weight and synthetic experiments. The inference evidence is consistent across several groups: near-lossless at large scale, a few points of loss at small scale. The pretraining evidence is positive but thin: one multi-trillion-token run from the proposing group with benchmark differences that are statistically indistinguishable from zero for most tasks and unresolved for others, plus a larger number of smaller studies that disagree on which recipe components matter. That the disagreement is mostly about ablations, not about whether the headline works, is encouraging; but the headline itself has, as of this writing, one independent large-scale confirmation at most (the Nemotron 3 Super run, which is from the same company).[S][U]

Table 12.3: Ledger of principal claims. Verdicts: *supported* (more than one independent source or an exact derivation), *plausible* (one source, no obvious conflict), *contested* (sources disagree), *unverified* (vendor or secondary claim).

Claim	Best evidence	Independent? Controlled?	Verdict
NVFP4 has lower quantisation error than MXFP4 for the same data	Exact formulas and Monte Carlo (Chapter 4); real-weight MSE (Chapter 13); vendor single example [4]	yes; yes (synthetic)	supported[D][E]
NVFP4 W4A4 PTQ loses about 1–3 points on typical benchmarks at large scale	Red Hat [20], QAD report [47], Nemotron 3 Super [15], MR-GPTQ [12]	partly (MR-GPTQ, Red Hat); partly	supported[S][V]
PTQ damage is larger for small models and RL-heavy models	QAD report; Red Hat size trend	no (same NVIDIA group for RL); no	plausible[S]
QAD recovers accuracy that QAT does not	QAD report only	no; partly	plausible[S]
NVFP4 pretraining of a 12B model to 10T tokens matches FP8 within about 1–1.5% loss	NVIDIA report [3]; Nemotron 3 Super 25T tokens (parity claimed, internal) [15, 50]	no; no	plausible, single group[S]
All-layer FP4 pretraining is stable at scale	Graphcore (8B, 188.7B tokens) vs Mosaic (all-layer diverges at 8B after 300B)	yes; partly	contested[S]
RHT on Wgrad is necessary	NVIDIA ablation; Graphcore and AMD results suggest otherwise	yes	contested[S]
RHT benefits NVFP4 RTN tensor MSE	Own experiment: it does not for heavy-tailed data (Chapter 6)	n/a	contested (own result against assumption)[E]
Stochastic rounding is required for unbiased gradients	Exact (Chapter 5); Quartet RTN backward counter-example	yes	supported as a bias statement; not as a requirement[D][S]
Four Over Six improves PTQ and pretraining	[7]; used in Mosaic and Quartet II	yes (Mosaic); partly	supported (small effect)[S]
MXFP4 needs about 36% more tokens than NVFP4	NVIDIA 8B experiment	no	plausible, single run[S]
NVINT4 can beat NVFP4 with rotation	INT-vs-FP study; MR-GPTQ; own experiment	yes	supported for PTQ; not shown for training[S][E]
FP4 training gives $\geq 1.5\times$ end-to-end speed-up on Blackwell	Vendor blogs, MLPerf; Quartet II $> 1.8\times$ BF16 at 1B	no	unverified for FP8 baselines[V][S]
Energy efficiency up to $50\times$ vs Hopper	NVIDIA blog	no	unverified (not format-attributable)[V]
Community consensus that 4-bit pretraining is ready	Newsletters repeating the report	no	unverified (Chapter 16)[U]

Chapter 13

Our Numerical Experiments

This chapter collects the numerical experiments we ran, states what each can and cannot show, and presents the two experiments that are not already reported inside the chapters where their results are used: format error on real weights with perplexity (Section 13.3) and training of a toy model under simulated recipes (Section 13.4). Table C.1 in Appendix C catalogues all fourteen experiments; Appendix B reproduces the code.

13.1 What fake-quantisation experiments can show

All experiments are float32 CPU simulations. A tensor is quantised to the 4-bit grid, de-quantised, and the computation proceeds in float32. This reproduces the *numerical values* that a correct 4-bit kernel would feed to its multipliers, provided the quantiser is implemented to specification. It does not reproduce (i) accumulator behaviour inside the tensor core, whose width we could not determine from the sources (Chapter 7), (ii) kernel-level details such as the order of reductions, the random-number streams used for stochastic rounding, or fused operations, (iii) throughput, memory or power. Three consequences for interpretation:

- Results about *error of the quantiser* (Chapters 4–6) are reliable to the sampling uncertainty of Appendix A.5, and exact formulas agree with simulation (Proposition 4.2), which validates the implementation.
- Results about *model quality* (this chapter) are valid for the specific small models and tasks used, and say nothing about frontier scale.
- The simulated recipes are our reading of the published descriptions. Where a description was incomplete we made a choice and state it.

The independent study of Egiazarian et al. [12] reports that real kernels track simulation within 0.2–0.3 points for inference, which supports the use of fake quantisation for inference-style studies; for training we know of one check, by Graphcore, which reports that a probe-matched emulator reproduces native behaviour in deterministic controls [14].[S]

13.2 Scoreboard: what we expected and what we found

Table 13.1 lists the questions we posed before running the experiments, with the expectation we held, and the outcome. We include it because several outcomes contradict the prior, and because a list of only the confirmations would overstate how well the standard picture holds.

Table 13.1: Expectations and outcomes of the synthetic and real-weight experiments.

Question	Prior expectation	Outcome
Does the quadrature formula match Monte Carlo?	yes (it is exact)	yes, to 3–4 digits for $G = 4$–128, Gaussian and Laplace (Table 4.1)
Is NVFP4 better than MXFP4 mainly by block size or scale format?	block size (the usual narrative)	scale format: at equal 4.25 bits E4M3 beats E8M0 by 22% (Gaussian); block halving gives a further 12% (Table 14.1)
Does INT4 beat FP4 at $G = 16$ on Gaussian data?	unsure	yes: 0.00726 against 0.00886; crossover between $G = 32$ and 64 (Chapter 4)
Does the E4M3 rounding-penalty model hold?	approximately	yes for $G \leq 16$; over-predicts by 20–30% for $G \geq 32$ (Table 4.4)
Does a Hadamard transform reduce NVFP4 RTN error?	modest gain (as often asserted)	**no**: unchanged on Gaussian, worse on heavy-tailed data; helps INT4 and MXFP4 (Chapter 6)
Does RHT reduce SR dot-product variance?	yes	yes: $6.6\times$ at $d = 16$, $\approx 22\times$ at $d = 128$ on spiky data
Does RHT help the one-step weight-gradient error?	yes (as in NVIDIA ablation)	**no** in our metric (Table 6.5)
Does SR need headroom to be unbiased?	yes	yes; costs 25% MSE (Section 5.2)
Is accumulator width a concern?	minor	FP16/BF16 accumulation reaches 0.03/0.20 relative error at $K = 65536$; FP32 negligible (Table 7.1)
Is 2D scaling worse in MSE?	yes (coarser tiles)	yes: 0.0137 vs 0.0090 on real weights (Table 13.2)
Does 2D scaling erase low-norm channels?	not anticipated	yes; prediction matches simulation (Table 15.2)
Does MSE rank formats the same way as perplexity?	yes	not across families (Table 13.3)

The two surprises relative to the usual narrative are the behaviour of the Hadamard transform with round-to-nearest, explained by the spike-block mechanism of Chapter 6, and the importance of the scale format relative to the block size. Both are statements about MSE on synthetic distributions and about weights of small models; whether they carry over to the activations and gradients of a large model in training is not tested by us.[U]

13.3 Real weights and perplexity

13.3.1 Design

We quantised the linear-layer weights inside the transformer blocks (all projections of attention and MLP; embeddings and the output head left in float) of SmolLM2-135M and Qwen2.5-0.5B with round-to-nearest, without any calibration data, for eleven weight formats, and measured the relative MSE per layer. For SmolLM2 we additionally evaluated the WikiText-2 test perplexity of the model with quantised weights (W4A16: weights in the 4-bit format, activations in float32). The perplexity evaluation uses 64 windows of 512 tokens (32,704 predicted tokens), a small sample, so differences of a few hundredths in perplexity are within noise but the large differences below are not (for reference, an unquantised-versus-quantised difference of 4 perplexity points is about 0.2 nats per token). Weights are quantised along the input dimension in blocks of 16 or 32 as the format requires.

13.3.2 Weight error

Table 13.2 shows that on real weights the ordering of the synthetic experiments is reproduced. NVFP4 has relative MSE 0.0090 in both models, close to the synthetic Gaussian value 0.0090 (Table 4.5):

Format	SmolLM2-135M	Qwen2.5-0.5B
NVFP4 B16	0.00896	0.00891
NVFP4 B16 + 4/6	0.00755	0.00756
NVFP4 16x16 (2D)	0.01372	0.01457
MXFP4 B32 (ceil)	0.01426	0.01471
MXFP4 B32 (floor/OCP)	0.01382	0.01392
INT4 B32 (FP32 scale)	0.01026	0.01084
NVINT4 B16	0.00794	0.00824
NVFP4 B16 + RHT16	0.00907	0.00909
MXFP4 B32 + RHT32	0.01366	0.01368
INT4 B32 + RHT32	0.00934	0.00929
NVINT4 B16 + RHT16	0.00738	0.00732

Table 13.2: Mean relative MSE of weight quantisation over the linear layers inside the transformer blocks (round-to-nearest). [E]

weights of these models are close to Gaussian within blocks of 16, with excess kurtosis between 0.9 and 2.4 by projection type (Table C.6). The numbers for MXFP4 (0.0138–0.0147 depending on the scale rule) and NVFP4 with 4/6 (0.0076) also match the synthetic values within a few percent. Three findings are specific to real weights:

- **2D scaling costs** 53–64% **more error** than 1D (0.0137 against 0.0090 for SmolLM2 and 0.0146 against 0.0089 for Qwen2.5-0.5B), a larger cost than the synthetic Gaussian case would suggest (0.0124 against 0.0090 in Table 15.2 at $r = 1$); the excess presumably reflects channel-norm variation within tiles, which is the mechanism of Section 15.4.[E][U]
- **Rotation changes the ranking.** With a Hadamard transform of size equal to the block, INT4 improves substantially (0.0103 $\rightarrow$ 0.0093 for $G = 32$; NVINT4 0.0079 $\rightarrow$ 0.0074), MXFP4 improves slightly (0.0143 $\rightarrow$ 0.0137), and NVFP4 gets *worse* (0.0090 $\rightarrow$ 0.0091, a small effect). After rotation NVINT4 has 0.0074 against NVFP4's 0.0091 and is therefore the best 4.5-bit format in MSE; without rotation NVFP4 with 4/6 (0.0076) is within 3% of rotated NVINT4. This is the real-weight counterpart of the INT-versus-FP result.[E][S]
- **Layer types are nearly homogeneous.** The relative MSE for NVFP4 across the seven projection types of SmolLM2 varies by less than 1.3% (Table C.6), even though kurtosis varies by a factor of three. Only the 2D variant shows spread (0.0132–0.0141).[E]

13.3.3 Perplexity

Table 13.3 gives the perplexity results. W4A16 NVFP4 raises the perplexity of the 135M model from 20.2 to 24.6 (+0.196 nats), a large penalty for a tiny model quantised without calibration, in the range expected from the PTQ literature for small models (Chapter 10). The 4/6 rule reduces the increase by 15.1% in nats (0.196 $\rightarrow$ 0.166) for a 15.7% reduction in weight MSE; the near-equality of the two ratios is what the second-order model predicts when the error is reduced uniformly. Across *families*, however, MSE does not predict perplexity: MXFP4 (ceil rule) has 59% higher MSE than NVFP4 but a 2.3$\times$ larger perplexity increase, and NVINT4 has 11% lower weight MSE than NVFP4 but a 8% larger perplexity increase. The ratio in the last column is about 22 for the two NVFP4 variants, 27 for NVINT4, 28 for INT4 with FP32 scale and 32 for MXFP4.

We do not know the cause. Two candidate explanations are consistent with the data. (i) The error has a different composition: the FP4 grid is relatively precise for the small weights and coarse only for

Configuration	perplexity	Δ ln ppl	weight rel. MSE	Δ ln ppl / MSE
BF16-equivalent (FP32)	20.217	0.000	–	–
W4A16 NVFP4 B16	24.586	0.196	0.00896	21.9
W4A16 NVFP4 B16 + 4/6	23.868	0.166	0.00755	22.0
W4A16 MXFP4 B32 (ceil)	31.856	0.455	0.01426	31.9
W4A16 INT4 B32 (FP32 scale)	26.966	0.288	0.01026	28.1
W4A16 NVINT4 B16	24.996	0.212	0.00794	26.7
W4A16 NVFP4 B16 + RHT16	27.493	0.307	0.00907	33.9
W4A4 NVFP4 B16	29.262	0.370	–	–
W4A4 NVFP4 B16 + 4/6	28.204	0.333	–	–
W4A4 MXFP4 B32 (ceil)	71.426	1.262	–	–

Table 13.3: SmolLM2-135M, WikiText-2 test perplexity (64 windows of 512 tokens) with weight-only 4-bit quantisation (round-to-nearest, activations in float32). Δ ln ppl is the perplexity increase in nats per token over the unquantised model; the last column is that increase per unit of relative weight MSE.[E]

the largest, whereas the INT4 grid has constant absolute precision; if large-magnitude weights matter less per unit of squared error than the bulk, an error profile that concentrates in the top cell (Table 4.3) is cheaper than an equal total MSE spread uniformly. (ii) The ceil-rule MXFP4 and the INT4 formats shrink or inflate the weights systematically (Table 7.2), and a coherent gain error in every layer is more damaging than random error of the same energy. Neither hypothesis was tested, and the one-model, one-dataset setting does not support further conclusions. What the data do show is that MSE is a sound guide *within* a family where the grid is fixed and the scale rule varies (NVFP4 with and without 4/6), and a poor guide across families.[E][U]

13.3.4 Activation quantisation and rotation

Two further results from the same experiment (Table 13.3) matter. First, quantising the activations of every linear layer as well (W4A4, same quantiser for the activations) raises the log-perplexity increase from 0.196 to 0.370 for NVFP4 (perplexity 29.3) and from 0.166 to 0.333 with 4/6 (28.2), so on this model activation quantisation costs about as much as weight quantisation; for MXFP4 with the ceil rule perplexity rises from 31.9 to 71.4. The ratio of NVFP4 to MXFP4 damage is therefore much larger for W4A4 than for W4A16.[E] Second, applying a size-16 Hadamard rotation to the weights before NVFP4 quantisation increases perplexity from 24.6 to 27.5 even though the weight MSE is almost unchanged (0.00907 against 0.00896). The rotation changes the error without changing its energy, and the model is more sensitive to the rotated error. Why we do not know; one candidate is that rotation spreads the quantisation error of large weights onto all coordinates of a block, including those of small weights, and this removes the structure (large weights are relatively accurately represented) from which the unrotated error benefits. This is a single-model result and confirms that MSE does not capture the effect of rotation on quality.[E][U]

13.4 Toy-scale training under simulated recipes

13.4.1 Design

To test the training recipes at all, we trained a decoder-only transformer on byte-level WikiText-2 (vocabulary 256, $d = 128$, 4 layers, 4 heads, context 128, about 0.8M weights in the quantised linear layers) with AdamW, a warm-up–stable–decay learning-rate schedule (warm-up over the first 5% of steps, con-

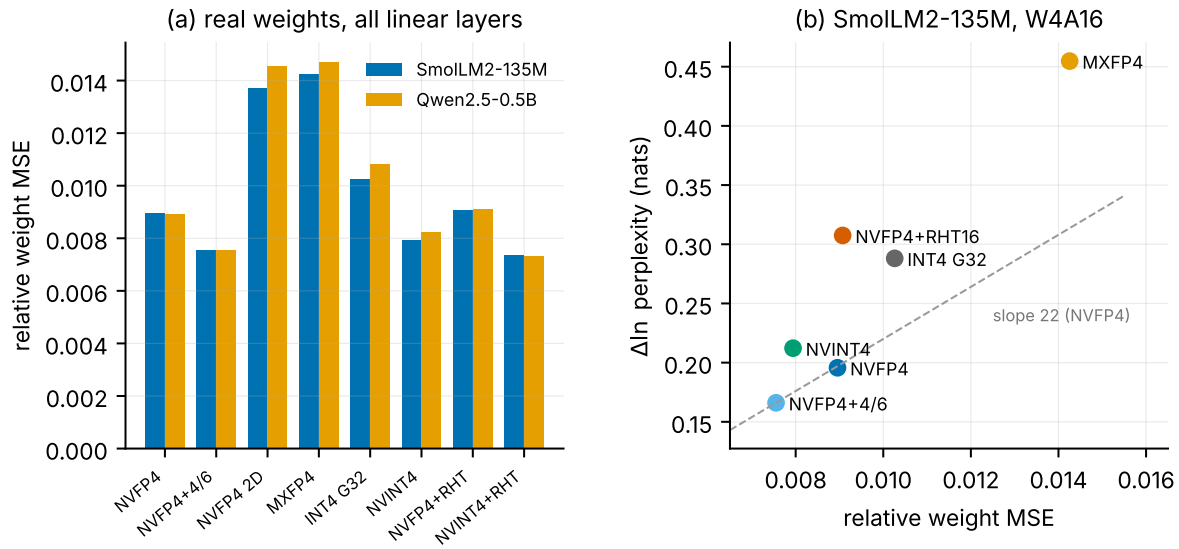


Figure 13.1: (a) Relative weight MSE by format for the two small models. (b) Increase in log-perplexity against weight MSE for SmolLM2-135M (W4A16); the dashed line has the slope observed for NVFP4 and passes through the origin.[E]

Recipe	seed 0	seed 1	mean gap to FP32	relative gap
FP32 (no quantisation)	2.0930	2.0806	+0.0000	+0.00%
NVFP4, RTN everywhere	2.1104	2.1187	+0.0278	+1.33%
NVIDIA recipe (2D, SR, RHT16)	2.1110	2.0964	+0.0169	+0.81%
NVIDIA recipe, last block FP32	2.1044	–	+0.0114	+0.54%
NVIDIA recipe + 4/6	2.1085	–	+0.0155	+0.74%
MXFP4, SR, RHT32	2.1177	–	+0.0246	+1.18%

Table 13.4: Toy-scale training: final validation cross entropy (nats per byte) after 500 steps and the gap to the FP32 run of the same seed. A dash means the run was not completed. [E]

stant to 70%, then linear decay to 5% of the peak), batch of 16 sequences (2048 tokens), 500 steps, so about one million training tokens, in float32 on a single CPU thread. Every linear layer of the four transformer blocks runs its three GEMMs through the fake-quantised autograd function of Appendix B; embeddings, layer norms, attention (softmax, score products) and the output head are in float32. Recipes follow Chapter 8: NVFP4 round-to-nearest everywhere ("naive"); the NVIDIA recipe (2D weight tiles, SR for gradients, RHT of size 16 on Wgrad inputs); that recipe with the last block kept in float32 (a toy version of selective high precision); the recipe with 4/6; and an MXFP4 recipe with SR and RHT of size 32. The initial weights and the data order depend only on the seed, so runs with the same seed share initialisation and batches. Validation loss is the mean cross entropy on 640 random test windows of 128 bytes, evaluated with the fixed seed 99, in nats per byte.

13.4.2 Results

Table 13.4 and Figure 13.2 show the outcome. All simulated 4-bit recipes train stably and end between 0.5% and 1.3% above the FP32 run of the same seed in validation cross entropy (mean over the seeds available), with no divergence. The ranking at seed 0 is: last block in FP32 (+0.0114 nats), 4/6 (+0.0155), naive RTN (+0.0174), NVIDIA recipe (+0.0180), MXFP4 (+0.0247). Two seeds exist only for the FP32, naive and NVIDIA runs. They disagree on the sign of the difference between the naive

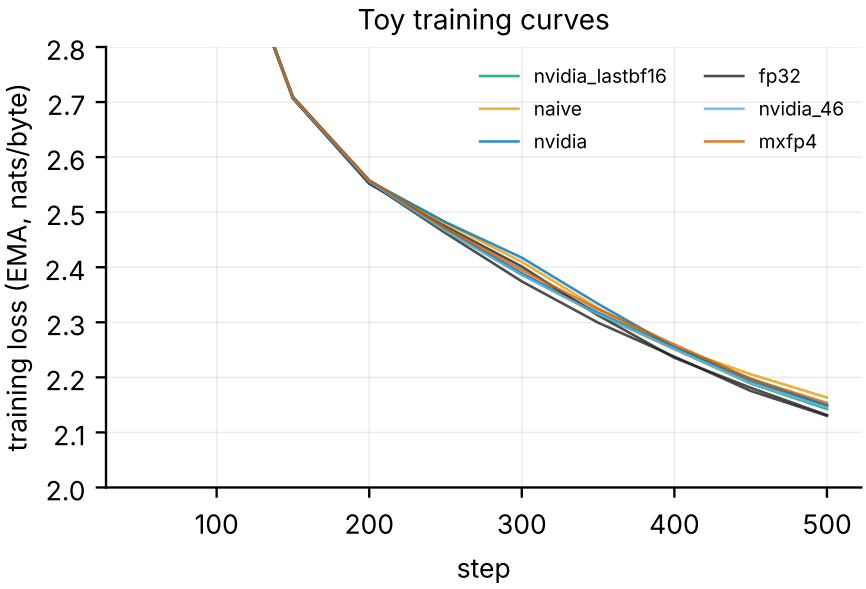


Figure 13.2: Toy training: exponentially averaged training loss against step for the completed runs (two seeds where available). The curves are indistinguishable at this scale.[E]

and NVIDIA recipes: at seed 0 naive is better by 0.0006 nats and at seed 1 the NVIDIA recipe is better by 0.0223, and the FP32 runs themselves differ between seeds by 0.0124. The seed-to-seed spread of the gap (about 0.01–0.02 nats) is as large as every difference between recipes in the table. We therefore cannot rank the recipes; the data support only that (i) all of them train, (ii) the gap to FP32 is of order one percent at this scale and horizon, and (iii) nothing here contradicts, or confirms, the ordering suggested by the literature. Runs for the remaining recipes (SR only, RHT only, 2D only, 1D with requantisation) and second seeds for the other recipes were not completed within the time available and were cancelled.[E]

13.4.3 Limitations of the toy experiment

The model has under a million quantised weights and trains on about a million tokens; its blocks have a contraction dimension of 128 or 512, so most of the effects that matter at scale (late-training noise, rare outliers, massive activations, million-step horizons) are absent. The gradient-noise level is large relative to quantisation noise at this batch size. The measured gaps are therefore a floor-level test of the plumbing (does the recipe train at all, does it diverge) rather than evidence on gaps at scale. We report it because the literature contains claims about individual recipe ingredients that are stated as universal, and a null result at toy scale is information: effects of the order of 0.5% are not resolvable here.[U]

Chapter 14

Alternatives and Competing Formats

NVFP4 is one point in a crowded design space. This chapter compares it with the alternatives that appear in the 2025–2026 literature: MXFP4, integer formats with fine-grained scales, FP4 with wider-range scales (UE5M3), Huawei's HiFloat4, 8-bit formats, and the weight-only INT4 route taken by several production models. Where we have measurements of our own they are cited from Chapter 4 and Chapter 13.

14.1 Iso-bit comparison of 4-bit block formats

A fair comparison fixes the storage budget. Table 14.1 takes the synthetic results of Table 4.5 and groups them by bits per element, counting the tensor-level FP32 scale as free.

Three points follow.

1. **Most of NVFP4's advantage over MXFP4 is the scale format, not the block size.** At the same 4.25 bits, an E4M3-scaled FP4 with $G = 32$ has 22% lower MSE than MXFP4 with the OCP floor rule on Gaussian data (0.01032 against 0.01320) and 57% lower on the outlier-channel distribution. Halving the block size from 32 to 16 at a cost of 0.25 bit then gives a further 12% on Gaussian data ($0.01032 \rightarrow 0.00904$). The power-of-two scale is therefore the larger of the two inefficiencies in MXFP4, a conclusion that follows from the penalty analysis of Section 4.5.[E][D] (This is a format-level statement. The $G = 32$ E4M3 variant is not a standardised format.)

2. **INT4 wins on light-tailed data and loses on heavy-tailed data.** NVINT4 has 18% lower MSE than NVFP4 on Gaussian data and 22% to 59% higher MSE on the Laplace, t_3 and outlier-channel inputs.[E] Which regime real tensors are in determines the winner, which is what the empirical INT-versus-FP studies find (below).

3. **Four Over Six dominates at equal bits.** At 4.5 bits, the 4/6 selection rule gives the lowest MSE among the options in this table on all four distributions except Gaussian, where NVINT4 is marginally ahead (0.00744 against 0.00756), at the cost of one extra quantisation pass per block (Section 4.9).[E]

MSE is not accuracy, as emphasised in Chapter 9 and Chapter 13, but it is the quantity that all these formats were designed to reduce and the table shows what each design choice buys on that criterion alone.

Table 14.1: Relative MSE of block formats at equal storage cost, round-to-nearest, 2^{21} samples per entry (own experiment, from Table 4.5). The bits column counts element bits plus scale bits divided by the block size. Lower is better; bold marks the best entry in each group and distribution.

Bits	Format	Gaussian	Laplace	t_3	Outl. chan.
4.25	MXFP4 floor (OCP rule)	0.01320	0.01587	0.02019	0.02783
4.25	MXFP4 ceil	0.01329	0.01753	0.02280	0.02554
4.25	NVFP4-type, G=32, E4M3 scale	**0.01032**	**0.01070**	**0.01160**	**0.01206**
4.25	INT4, G=32, FP32 scale (8-bit scale would add 0.25 bit)	0.00941	0.01469	0.02107	0.01776
4.5	NVFP4 (G=16, E4M3 scale)	0.00904	**0.00860**	**0.00830**	**0.00668**
4.5	NVINT4 (G=16, E4M3 scale)	**0.00744**	0.01052	0.01323	0.00958
4.5	NVFP4 + Four Over Six (MSE)	**0.00756**	**0.00761**	**0.00763**	**0.00634**

14.2 Integer formats with fine-grained scales

14.2.1 The INT-versus-FP study

Chen et al. of ByteDance and HKU present the most systematic comparison of fine-grained integer and floating-point formats [11].[S] The headline results, from the paper's abstract and text, are:

- At *8 bits* with block size 32, MXINT8 is superior to MXFP8 in both algorithmic accuracy and hardware efficiency, and the paper proposes a symmetric clipping method that removes gradient bias in INT training so that MXINT8 training is nearly lossless.
- At *4 bits*, FP formats (MXFP4, NVFP4) often hold an accuracy advantage, but NVINT4 can surpass NVFP4 when Hadamard rotation is applied.
- The crossover depends on the crest factor of the block: INT wins when the crest factor is small (flat blocks), FP when it is large.

The paper argues from these results that a one-size-fits-all FP trajectory for accelerators is suboptimal and that MXINT8 offers a better balance of accuracy, power and efficiency.[S] This is a hardware-policy conclusion on which the authors have an evident stake in advocacy; the accuracy comparisons stand independently of it. Our own experiments agree on the mechanism and the 4-bit direction (Section 4.8, Table 14.1, and Chapter 6, where NVINT4 improves with a Hadamard rotation while NVFP4 RTN does not).[E]

14.2.2 INT4 in production models

Two production uses of integer 4-bit weights are relevant. Kimi K2 Thinking applies INT4 quantisation-aware training during post-training and ships INT4 weight-only quantised MoE components; the model card reports roughly a 2× generation speed-up in low-latency mode and does not state the group size [18].[V] A company blog claims that the GLM-5 report applied INT4 QAT only during supervised fine-tuning and not RL, and that Kimi and DeepSeek-V4 apply QAT throughout post-training [21]; we did not verify the GLM-5 claim against the primary report.[U] Weight-only INT4 on the MoE experts is a different design point from W4A4 block-scaled FP4: it requires no 4-bit tensor-core support (weights are dequantised in the kernel) and therefore runs on Hopper and earlier, at the price of no compute speed-up for the 4-bit parts.[D]

14.3 Power-of-two scales: MXFP4

MXFP4 is the OCP standard [2], and is the format actually used in two of the most prominent open-weight deployments. OpenAI's gpt-oss models quantise the MoE weights (more than 90% of the parameters) to MXFP4 at 4.25 bits per parameter after post-training, so that the 120B model fits on one 80 GB GPU [17].[S] DeepSeek-V4 applies MXFP4 quantisation-aware training to the MoE expert weights and to the query–key path of its lightning indexer, and describes the FP4-to-FP8 dequantisation as lossless provided that the ratio between the largest and smallest FP4 sub-block scale within each FP8 quantisation block (128×128) stays below a threshold, which the authors verified empirically for their weights [16].[S]

The DeepSeek statement is the same condition that our analysis of E4M3 range gives (Section 4.7): a scale ratio inside the dynamic range of the scale format is absorbed exactly. The threshold depends on how the 128×128 FP8 scale is chosen, which the report does not detail.[D][U]

Why would a laboratory with Blackwell-class infrastructure choose MXFP4 over NVFP4 for inference weights? None of the sources we read states the reason; three candidate reasons are as follows (our speculation, [U]). First, MXFP4 is an open standard supported across vendors, so a checkpoint can serve on non-NVIDIA hardware. Second, QAT can compensate for the extra rounding error of the E8M0 scale because the weights adapt to the grid, so the format-level MSE penalty (Table 14.1) may shrink in effect, by the weight-adaptation argument of Section 9.1.[D][U] Third, the scale arithmetic is cheaper (a shift rather than a floating-point multiply) in hardware, which matters for the dequantise-then-compute paths on current chips.[D]

The training-side use of MXFP4 has an interesting asymmetry. AMD reports that MXFP4 pretraining on native-FP4 hardware fails at the weight-gradient GEMM, that stochastic rounding and *random* Hadamard transforms do not fix it, and that a *deterministic* Hadamard transform does [46].[S] We discuss this in Chapter 8; it is mentioned here as evidence that the MXFP4 training recipe is not simply the NVFP4 recipe with a different scale.

14.4 Wider-range scales: UE5M3

The "finer is not better" report of Fasoli et al. identifies an anomaly: for microscaling formats the quality of a quantised model can *degrade* as the block size is reduced below a threshold, because narrow tensor distributions combine with the limited dynamic range of the quantised scales so that scales underflow [13].[S] They develop a theoretical framework that agrees with data from pretrained models and propose an unsigned E5M3 scale format (UE5M3) that matches E4M3 in accuracy while removing the need for a global per-tensor scaling operation. Our analysis of the E4M3 range gives the same mechanism and quantifies the threshold: precision loss begins when the ratio of a block's absolute maximum to the tensor's falls below about 3.5×10^{-5}, and a block is flushed to zero below about 2.2×10^{-6} (Section 4.7). UE5M3 adds one exponent bit; with conventional exponent biases the span between the smallest and largest normal scale magnitudes grows from about 2^{14} for E4M3 to about 2^{30}, a factor of about 2^{16}, and both thresholds move by a corresponding factor (our arithmetic; exact encoding conventions are those of the cited paper).[D]

Graphcore's UE5M3 pretraining report builds a full recipe on this idea [14]. The measurements are summarised in Table 12.1: in the final-window losses of an 8B hybrid trained for 188.7B tokens (single seed), the BF16 baseline is 2.2651; UE5M3 scales with a TE-style delayed tensor scale reach 2.2902 and the authors' proposed recipe 2.3090, compared with 2.3474 for the TE NVFP4 recipe and 2.4420 for a native-NVFP4 configuration without RHT and without BF16 exempt blocks. The comparison supports the claim that wider-range scales are a useful ingredient, but the recipes differ in several ways at once

Table 14.2: Relative loss error against BF16 after 50B tokens, each format at its best stabiliser configuration (Table 3 of Taghian et al. [19]). OpenPangu-1B is averaged over four seeds; the other rows are single runs. Best configurations: HiF4+RHT; MXFP4 with truncation-free scaling, SR and RHT; NVFP4 with per-tensor scaling, SR and RHT.

Model	Active params	MXFP4	NVFP4	HiF4
OpenPangu-1B	1B	1.75%	1.26%	1.25%
Llama3-8B	8B	1.44%	0.99%	0.85%
Qwen3-MoE-30B	4.6B	1.55%	0.99%	0.88%

(delayed tensor scaling, RHT, high-precision blocks) and the experiment is a single seed.[S][U]

A caution on interpretation. The wider-range scale format is only needed when the dynamic range of the tensor, plus the E4M3 scale quantisation, exceeds what the per-tensor scale can absorb. In a well-conditioned tensor with a conventional per-tensor amax the range limit is not binding; the format helps in the presence of massive outliers (Table 4.9) and in training recipes that avoid per-tensor reductions.[D][E]

14.5 HiFloat4: hierarchical scaling

Huawei's HiFloat4 (HiF4) uses a different design: a block of 64 elements carries a shared unsigned E6M2 top-level scale, two levels of 1-bit micro-exponents that refine 8- and 16-element subsets (the third level governing four contiguous elements), and S1P2 (equivalently E1M2) element encodings, for 32 bits of metadata per block and 4.5 bits per value overall, the same as NVFP4 [19].[S] The scale hierarchy needs a floating-point multiply only once per 64 elements, with the finer levels reducing to shifts.

The Ascend pretraining study compares HiF4, MXFP4 and NVFP4 with a fixed recipe on OpenPangu-1B, Llama3-8B and Qwen3-MoE-30B for 50B tokens each [19].[S] Relative loss error against BF16, with each format at its best configuration, is in Table 14.2.

With the best configuration of each format the HiF4 advantage over NVFP4 is 0.01, 0.14 and 0.11 points, small compared with the single-seed uncertainty. The headline claims of the paper are about *stabilisation requirements*: on OpenPangu-1B plain HiF4 reaches 1.55% with no stabiliser, MXFP4 with all three stabilisers 1.79%, and NVFP4 diverges under every combination that lacks the per-tensor scale (plain, SR, RHT, SR+RHT); with per-tensor scaling alone it reaches 2.00%, with per-tensor scaling and RHT 1.42%, and with SR added 1.27%.[S] Three cautions apply. (i) The runs are on Ascend NPUs without native FP4 or HiF4 kernels: all GEMMs execute in BF16 with simulated quantisation, so no speed claim follows. (ii) "NVFP4 diverges" refers to NVFP4 *without its FP32 per-tensor scale*, which is not NVFP4 as specified by NVIDIA (Section 3.3.2); the authors' point is that the format cannot be used without it, and they explain this by its range of 22 binades against 69 for HiF4. That is consistent with our E4M3 range analysis, but it is a statement about a missing component, not about the complete recipe.[D] (iii) The reported advantage of HiF4 for SR is the opposite of the NVIDIA result: SR hurt HiF4, adding 4.56% on its own in the 1B ablation.[S] We read this as evidence that stabiliser conclusions are format- and implementation-specific, which is the lesson of Chapter 8 again.

A second Huawei paper evaluates HiF8 and HiF4 for post-training inference, finding that HiF4's hierarchical scaling avoids the accuracy collapse seen with integer formats at 4 bits, that INT8 suits narrow-range data while FP formats suit high-variance data, and that the formats are compatible with SmoothQuant and SVDQuant [53].[S] We did not extract its numerical tables.

14.6 Eight-bit baselines

The 4-bit formats are measured against 8-bit training, so the 8-bit state of the art is the yardstick. Two reference points:

- **DeepSeek-V3's FP8 recipe** uses 1×128 tiles for activations and 128×128 blocks for weights, higher-precision accumulation (the report notes that the tensor-core accumulation is limited to about 14 bits, with a maximum error of about 2% at inner dimension 4096), and reports a relative loss error below 0.25% against BF16 [40].[S] This is the relevant benchmark against which the 1–1.5% NVFP4 gap should be read; the 4-bit gap is four to six times larger.[D]
- **MXFP8 recipes** of Mishra et al. show that MXFP8 pretraining matches BF16 sessions for models of up to 8B parameters trained on up to 15T tokens, provided the E4M3 element type is used for all tensors, including activation gradients, together with a specific number-conversion algorithm for the scales [23].[S] The paper's conversion algorithm computes the shared exponent with a ceiling, i.e. rounds the power-of-two scale up.[S] The rounding-up rule is also the scale rule that avoids clipping in our MXFP4 analysis (Section 4.5).[D]

The accumulation limit mentioned in the DeepSeek report reappears at 4 bits in a milder form: our analysis shows that intra-block FP4 products are exact in fixed point and that the dominant accumulation error is across blocks (Chapter 7).[D][E]

14.7 Weight-only and rotation-based 4-bit methods

Outside the block-scaled-FP world there is a large literature on weight-only 4-bit quantisation (GPTQ [54], AWQ [55]) and on rotation-based W4A4 methods (QuaRot [39], QuIP# [38]) that use integer elements. The conceptual relation to the block formats is close: a group-wise INT4 with group size 128 is a block format with a large block, rotations flatten the blocks so that INT4 works, and the fine-grained FP scale of NVFP4 is an alternative to rotation for handling outliers within a block.[D] MR-GPTQ, which combines GPTQ with block-wise rotations tailored to MXFP4 and NVFP4, shows that these ideas stack: rotations plus GPTQ compensation raise MXFP4 recovery from 87.8% to 93.3% and NVFP4 from 94.7% to 96.1% on Llama-3.1-8B [12]. The gain is larger for the format with the larger intrinsic error, which is the expected behaviour if the methods remove part of an error that depends on the format.[S][D]

14.8 Summary

- At equal storage, E4M3 scales beat E8M0 scales by more than a halving of the block size does; MXFP4's main handicap is its scale.[E]
- INT4 with fine-grained scales is competitive or better on light-tailed data and after rotation; FP4 is better on heavy-tailed data.[E][S]
- UE5M3 and HiF4 address the dynamic-range limits of the scale; the evidence for them is from one group each and, for HiF4, simulated on non-native hardware.[S]
- Production 4-bit deployments use MXFP4 (gpt-oss, DeepSeek-V4) or INT4 (Kimi K2) weights with QAT, not NVFP4 PTQ, in the flagship cases we found; NVFP4's strongest production appearance is the NVIDIA-trained Nemotron 3 Super (Chapter 17).[S][U]

Chapter 15

Strengths, Weaknesses and Failure Modes

This chapter gathers, in one place and with quantitative backing where we have it, the advantages and disadvantages of NVFP4 and of 4-bit microscaling training and inference in general, and then catalogues the failure modes reported in the literature or found in our experiments.

15.1 Advantages

1. **Memory and bandwidth.** 4.5 bits per element is $3.56\times$ less than FP16 and $1.78\times$ less than FP8 for weights and activations (Table 3.3).[D] Memory-bound decode scales with the bytes moved, so an upper bound on the decode speed-up over FP8 weights is $1.78\times$.[D]
2. **Compute.** Dense peak throughput is $2\times$ (Blackwell) or $3\times$ (Blackwell Ultra) that of FP8 (vendor; Table 11.1).[V] Kernel-level measurements by independent groups confirm large gains for isolated linear layers, though smaller on data-centre parts at small shapes (Chapter 11).[S]
3. **Lower error than other 4-bit block formats at equal storage.** From the exact analysis and experiments: roughly $1.5\times$ lower MSE than MXFP4 on Gaussian data (0.00904 against 0.01329–0.01474) and a larger factor on heavy-tailed data (Table 4.5, Table 14.1).[E][D]
4. **Real-model PTQ results close to higher precision at scale.** Accuracy recovery of 97–99% or more for 30B-plus models and a median 99.8% for the Nemotron 3 Super checkpoint, with larger models more robust (Chapter 10).[S][V]
5. **Pretraining at scale appears feasible.** A 12B model for 10T tokens within 1–1.5% loss of FP8, and a 120B-parameter model for 25T tokens with parity claimed [3, 15].[S]
6. **A moderate recipe.** Compared with earlier 4-bit work, the recipe is a small number of well-understood ingredients (Chapter 8), each of which has a transparent mathematical rationale (Chapter 5, Chapter 6).[D]
7. **Open tooling.** The Transformer Engine implementation is documented and open source; major inference engines support NVFP4 checkpoints [35].[V]

15.2 Disadvantages

1. **Residual loss gap.** All the reported gaps are positive; the smallest reported for pretraining are about 0.5–0.6% in relative loss and are obtained with extra measures (a high-precision forward-pass switch near the end, or compensation modules), and the gap grows late in training (Chapter 9).[S]

2. **Recipe complexity and overhead.** Selective high-precision layers, a Hadamard transform, stochastic rounding, 2D scaling and a tensor-level amax synchronisation all add work and bugs; one independent study reports that removing the RHT and the BF16 exempt blocks raises throughput by 21.2% in its setting [14].[S]
3. **Hardware lock-in.** Native NVFP4 exists only on Blackwell-generation NVIDIA GPUs (SM 10.0/10.3 for training); other vendors implement MXFP4 or their own formats (Chapter 11).[V]
4. **Scale arithmetic.** E4M3 scales are not powers of two: a floating-point multiply per block (cheap, but not free), a second-level tensor scale that must be synchronised, and a limited range that produces the failure modes below.[D]
5. **Awkward kernels.** TN layout only, padded and swizzled scale tensors, transposed copies for backward operands.[V]
6. **Weak evidence outside NVIDIA's own model family.** Most of the pretraining evidence is on Mamba–Transformer hybrids; the controlled replication covers two architectures [10].[S]
7. **Accuracy damage in specific regimes.** Small models, RL-trained reasoning models and long-horizon tasks (Chapter 10).[S][U]

15.3 Catalogue of failure modes

Table 15.1 lists the failure modes with the mechanism, the strongest evidence and the known mitigation. The sections that follow expand the ones for which this report has original material.

15.4 Failure mode I: low-norm channels inside two-dimensional tiles

The Nemotron 3 Super report describes a failure that is both subtle and instructive. During pretraining the number of zero-valued weight-gradient elements grew until, by the end of training, zero-valued elements made up 7% of the parameters; the pattern correlated with expert-layer channels whose norms were converging to zero (low-norm FC1 output channels aligned with low-norm FC2 input channels). In a controlled comparison with Nemotron 3 Nano, NVFP4 training produced roughly $3\times$ as many zero-valued weight gradients as BF16 training at the same token horizon, and switching back to BF16 returned the count to baseline. The authors attribute the zeros to underflows in the data gradient of FC2, "primarily because two-dimensional weight quantisation blocks span high and low magnitude channels", and later to underflows in the forward pass of FC1 [15].[S]

This is a direct consequence of the 2D weight scaling of Section 3.3.2, and we can quantify it. A 16×16 tile has a single scale set by the largest of its 256 entries. An element rounds to zero when its magnitude is below $\mathrm{amax}_{\mathrm{tile}}/24$, because the zero cell of the E2M1 grid extends to 0.25 in units where the tile maximum is 6. For a channel (row) whose entries have standard deviation r times that of the other channels in its tile, the zero fraction is

$$\mathbb{P}(\text{zero}) \approx 2\Phi\left(\frac{\bar{A}_{256}}{24\,r}\right) - 1, \qquad \bar{A}_{256} \approx 3.04, \tag{15.1}$$

where Φ is the standard normal distribution function and $\bar{A}_{256}$ is the mean maximum of 256 absolute standard normal variables (our Monte Carlo estimate), under the assumption that the tile maximum is set by the high-norm channels.[D] With one-dimensional 1×16 blocks the scale is set within the channel and the zero fraction stays at 6.8% whatever the channel norm.

Table 15.1: Failure modes of 4-bit block-scaled training and inference.

Failure mode	Mechanism	Evidence	Mitigation
Gradient bias from RTN (dead zone)	Round-to-nearest maps small gradients to zero; bias adds a floor to SGD convergence	Own toy: RTN mean 0 against true 0.12 (Chapter 5); [31]	SR; unbiased surrogates (MS-EDEN)
Weight-gradient GEMM instability	Wgrad contracts over the token dimension; errors add coherently	AMD: Wgrad is the failure source [46]; NVIDIA RHT ablation	RHT on Wgrad; SR; FP8 Wgrad
Late-training loss gap	Quantisation noise relatively larger as the learning rate decays	12B run [3]; Chapter 9	Switch the forward pass to higher precision near the end
All-layer divergence	Sensitive layers (last blocks, projections) lack headroom	Mosaic: diverges after $\sim$ 300B tokens [10]	Keep $\sim 15\%$ of layers in BF16/MXFP8
Underflow in 2D weight tiles	A tile spanning high- and low-norm channels zeroes the small ones	Nemotron 3 Super; own experiment (Section 15.4)	1D weights with re-quantisation; MXFP8 for affected layers
Zero gradients in expert layers	Low-norm channels attenuate and underflow, 7% of parameters with zero Wgrad	Nemotron 3 Super [15]	Higher precision for the affected tensors
Scale-range collapse	Ratio of block max to tensor max beyond the E4M3 range flushes or coarsens blocks	Own analysis (Section 4.7); [13, 14]	UE5M3 scales; tensor-scale management
Rotation harms heavy tails	RHT converts cheap spike blocks into flat blocks which sit in the coarse 4–6 cell	Own experiment (Chapter 6)	Apply RHT only where variance matters (SR operands)
Accumulation error	Long contractions in low-precision accumulators	Own: FP16/BF16 accumulation error orders of magnitude above FP32 at $K = 65536$ (Chapter 7)	FP32 accumulation across blocks
PTQ collapse on RL models	Quantisation noise perturbs a narrow RL policy; task-loss QAT shifts it further	QAD report [47]	QAD
Output verbosity from state quantisation	Recurrent state at low precision accumulates rounding bias	Nemotron 3 Super, up to 40% longer outputs	Scaled INT16 state; FP32
Evaluation noise	Small benchmark sizes and single seeds	Chapter 12	Report seeds, intervals
Train–serve mismatch	Model trained with one rounding path and served with another	No direct study (Chapter 12)	Evaluate in deployment precision

Table 15.2 tests this on synthetic 1024×1024 Gaussian weights in which a fraction f of the rows has a norm ratio r, with the low rows placed at random so that nearly every tile also contains high-norm rows. The prediction is accurate (within 0.02 of the measured value at $f = 0.1$) and shows how abrupt the effect is: at $r = 0.3$ a third of the entries of the small channel are lost, at $r = 0.1$ about four-fifths, and for $r \leq 0.03$ the channel is erased completely. The relative MSE of the affected rows is then 1.0: the information in these channels is gone, yet the whole-matrix MSE is almost unchanged (0.012–0.013 against 0.009 for 1D), because the lost channels carry little energy.[E] Total MSE therefore hides the failure, which is the point at which loss-based diagnostics miss a phenomenon that a count of zero gradients detects.

Two caveats. In our synthetic test low-norm rows are scattered; in real weights low-norm channels cluster, which makes whole tiles low-norm and protects them, so the damage is concentrated at the boundaries of clusters. And dead channels in FC1 and FC2 that are *aligned* (as the report observes) are a property of how the network trains, not of the quantiser: the paper's comparison with a BF16 model trained to 10T tokens, which shows a similar number of zeros, suggests that NVFP4 accelerates a pattern that long training produces anyway.[S] We have not tested whether 1D weight scaling with re-quantisation (as in the comparative study) avoids the pattern.[U]

Norm ratio r	low rows f	1D: zero frac.	2D: zero frac.	2D: predicted	2D rel. MSE (low rows)
1	0.1	0.069	0.102	0.101	0.012
0.3	0.1	0.069	0.323	0.328	0.059
0.1	0.1	0.068	0.780	0.795	0.487
0.03	0.1	0.068	1.000	1.000	1.000
0.01	0.1	0.067	1.000	1.000	1.000
1	0.5	0.068	0.100	0.101	0.012
0.3	0.5	0.068	0.306	0.328	0.052
0.1	0.5	0.068	0.749	0.795	0.438
0.03	0.5	0.068	0.999	1.000	0.998
0.01	0.5	0.068	1.000	1.000	1.000

Table 15.2: Fraction of the entries of low-norm rows that are quantised to zero, for 1D (1×16) and 2D (16×16) NVFP4 weights, with the 2D prediction $2\Phi(3.04/(24r)) - 1$ (own experiment, 1024×1024 Gaussian weights, one draw per row; f is the fraction of low-norm rows, placed at random).[E][D]

15.5 Failure mode II: forward noise and the late-training gap

The NVIDIA 12B report shows that most of the loss gap comes from the forward pass, and that switching the forward pass to BF16 for the last 18% of training reduces the gap from about 1.5% to 0.5% (Chapter 8). Our second-order model of Section 9.1 explains why this is expected: the loss increase is $\frac{1}{2}\,\mathrm{tr}(\boldsymbol{H}\boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma}$ the covariance of the quantisation noise, which is a fixed fraction of the signal power and does not shrink with the learning rate. SGD noise, which sets the "temperature" of the optimisation, does shrink with the learning rate during decay, so the relative importance of quantisation noise grows exactly when the model is fine-tuning its last digits of loss.[D] Nemotron 3 Super tested the analogous healing step (all tensors to MXFP8 for the last 1T tokens before annealing) and found it improved the loss trajectory but not downstream accuracy [15], so loss gap and capability do not move together.[S]

15.6 Failure mode III: instabilities that are not numerical

Several of the reported instabilities may reflect interactions with architecture and optimiser, not a property of the format. The all-layer divergence at 8B after 300B tokens in the comparative study, the dependence on the NVFP4 per-tensor scale in the Ascend study (NVFP4 diverged without it), and the problem with the Mamba output projection in the Nemotron report all point at specific tensors. The recurring recommendation to keep the final blocks in higher precision is consistent with a simple sensitivity argument (the output blocks have the largest curvature per unit of noise) but the evidence for it is ablation-based, with short horizons and few seeds.[S][U]

15.7 Which failures are most likely to bite in practice

Taking the evidence at face value, a practitioner is most likely to meet, in order: (i) accuracy loss from a poorly matched PTQ on a small or RL-trained model (cheap to detect; QAD is the documented remedy); (ii) a few percent of throughput left on the table because quantisation kernels are not fused; (iii) a silent, hard-to-detect capability loss from underflowing channels in 2D-scaled weights or in low-magnitude gradients (the zero-gradient count is the diagnostic); (iv) late-training loss gap; and only then (v) divergence, which the recipe's exemptions are designed to prevent.[U]

Chapter 16

Reception, Perception and Controversies

This chapter describes how NVFP4 has been received, and the main points of contention. We state at the outset that the evidence is thin and uneven. We did not retrieve forum discussions such as Hacker News threads; one web search aimed at them returned only newsletters, summaries and a Reddit snapshot. What follows is therefore (a) the tone of the secondary press, (b) two small snippets of social-media discussion, (c) a company blog on a failed deployment, and (d) the more substantial and more reliable signal from the research literature, where papers argue with one another. We tag the secondary and social material [U] throughout because it is a convenience sample.

16.1 Sources and how much they weigh

16.2 Press and newsletter reception

The trade press coverage in late October 2025 followed the NVIDIA paper's framing closely: a 4-bit pretraining method "that matches 8-bit performance", based on a 12B-parameter hybrid model trained on 10T tokens and an 8B comparison in which MXFP4 needed 36% more data [51, 52].[U] The VentureBeat piece includes a quotation from an NVIDIA product director and no external expert, and itself notes in its text that the results come from NVIDIA's paper and that inference gains are asserted rather than measured; it also notes that "half the memory" is relative to 8-bit training [51]. Both articles report that the loss gap appears late in training and that coding tasks were the one area in which the NVFP4 model fell behind.[U] A newsletter issue in May 2026 treats the work as the main story of the week, quotes the MMLU-Pro scores (62.58 against 62.62) as a gap "within statistical noise" and states that all four techniques of the recipe were needed, and advises readers pricing training clusters against FP8 throughput to revisit their arithmetic [49].[U] The last point, that the economics hinge on the throughput ratio, is the correct question and it is the one that Section 9.4 and Chapter 11 address; the newsletter, like the press, does not give a measured number.

A Reddit post of May 2026 summarised the paper as the "first public 4-bit pretraining run at multi-trillion-token scale"; the two top comments in the retrieved snapshot were a joke about a new GPU for AI inference only and a speculation that if 4-bit matches 8-bit then 2-bit and ternary might be next [56].[U] These carry little information, other than that the topic is perceived as news and that extrapolation to lower precision is a natural reaction.

Table 16.1: Sources on reception, with type and our assessment of how much weight they carry.

Source	Type	Date	Weight
VentureBeat [51]	trade press summary of the NVIDIA paper; one NVIDIA quote	28 Oct 2025	low: restates vendor claims, no outside expert
TechTalks [52]	same author, longer	10 Nov 2025	low: partly paywalled
AI Weekly newsletter [49]	newsletter issue	26 May 2026	low: restates results
Reddit r/machinelearningnews snapshot [56]	social media (two top comments)	18–20 May 2026	very low: anecdote
Red Hat developer article [20]	vendor engineering blog with results	4 Feb 2026	medium for PTQ data; vendor
umans.ai GLM-5.2 post [21]	company blog on a public serving experiment	mid-2026	low-medium: transparent about what is measured; anecdotal quality
InsiderLLM llama.cpp guide [36]	web guide with published corrections	2026 (corrected Sep)	low: second-hand
NVIDIA blogs [4, 32–34, 48]	vendor marketing and technical blogs	2025–2026	vendor claims only
Research papers (Chapters 8, 14)	arXiv preprints, mostly not yet peer reviewed	2025–2026	highest, but see Chapter 12

16.3 Practitioner reception: the inference side

The llama.cpp guide records the state of consumer-hardware support in 2026: NVFP4 kernels merged for CUDA, SYCL and Vulkan, an apparent 25% size saving over Q4_K_M that was later corrected to about 7% and then reported at higher size for a specific file, and "early Reddit reports" that the quality gap against Q4_K_M is small on large models and possibly worse on small ones [36].[U] The article says explicitly that no independent benchmark suite had compared the two formats' quality. This is typical of the information environment: confident claims at publication, corrections afterwards, and no controlled data. It is also a reminder that for local inference the relevant competitor is not FP8 but an existing, mature 4-bit integer scheme with mixed bit widths.[D]

The Red Hat article (4 February 2026) is the most data-rich practitioner source [20]. Its message is positive but graded by model size, and it explicitly says smaller models vary more with calibration choice and may require model-specific tuning. It reports no speed numbers.[V]

16.4 A case of disappointment: GLM-5.2

The umans.ai post ("fast, cheap, and not worth serving") describes a public experiment serving a third-party NVFP4 PTQ of a 744B-parameter mixture-of-experts model on its own GPUs over several days

[21].[U] Throughput of 200 or more tokens per second and median time-to-first-token under two seconds are reported as operational measurements; memory is stated as about 744 GB in FP8 against about 420 GB in NVFP4. Quality is described through anecdotal reports: early praise, later chain-of-thought collapse, more tool-rule violations, and at least one user switching back to FP8 after hours of work. The checkpoint's own model card is said to have warned that it was not quantisation-aware trained and recommended the FP8 version for production.

The post's thesis is that the decisive variable is whether the model was *trained for the target precision*, not the NVFP4 format; it contrasts the model with Kimi K2.7-Code, described as QAT-trained at 4 bits, whose NVFP4 conversion is said to hold parity (SWE-bench Verified 74.1 against 74.3, Terminal-Bench 71.9 against 72.5, figures attributed to NVIDIA). We cannot check these figures and the post's authors flag that its strongest quality claims rest on anecdotes from a small, unquantified group. Its value is that it is an honest account of what happens when a PTQ checkpoint of a very large model meets long-horizon agentic coding, a regime in which benchmark scores like those in Table 10.1 are least informative. It is consistent with the QAT/QAD evidence of Chapter 10, but it is neither controlled nor reproduced.[U]

16.5 Controversies in the research literature

The research community's engagement with NVFP4 is visible in how papers position themselves against the NVIDIA recipe. The main lines of dispute are as follows.

"Matches FP8" versus a measurable gap. The framing in the press and in NVIDIA blogs is parity. The paper itself reports a loss gap of 1–1.5%, rising late in training, and flags the coding deficit. Independent studies find non-negligible gaps in their settings: $< 1.2\%$ (dense) and $< 0.81\%$ (MoE) in the comparative study *after* refinement of the recipe [10], and 3.6% for the TE NVFP4 recipe in the Graphcore setting [14]. Our audit (Chapter 12) finds that the downstream-benchmark claim of parity cannot be distinguished from noise for most tasks, but that this is not evidence of equality either.[S][D]

How much is really 4-bit? The recipe keeps about 16% of linear layers in BF16, attention, embeddings and the head in higher precision, and master weights and optimiser state in FP32. A fair characterisation is that NVFP4 accelerates the bulk linear GEMMs of a hybrid model, and the system-level gains are bounded accordingly (Amdahl's law, Section 9.4).[D] The "first 4-bit pretraining at trillion-token scale" claim is the authors' own, in the report's words; whether the label 4-bit training is accurate depends on whether one counts GEMMs or parameters.[U]

Integer versus floating point. The ByteDance–HKU study argues that fine-grained integer formats offer a better balance of accuracy, power and efficiency than the FP trend of the hardware industry, and that NVINT4 can beat NVFP4 with rotation [11].[S] The MR-GPTQ results and our experiments support the accuracy comparison at 4 bits for PTQ. The disagreement is about hardware policy: whether accelerators should dedicate silicon to FP4 or to MXINT8 and INT4. This dispute is unresolved and hardware roadmaps have not changed visibly in response.[U]

Open standard versus vendor format. MXFP4 is an OCP standard with multi-vendor backing; NVFP4 is NVIDIA's format. Two of the largest open-weight efforts in our source set, gpt-oss and DeepSeek-V4, use MXFP4 [16, 17], and Kimi K2 uses INT4 [18]; the model that has NVFP4 as its training format is NVIDIA's own [15]. NVIDIA's NVFP4 blog lists partners (AWS, Cohere, Google Cloud, Kimi, Microsoft AI, Mistral, OpenAI, Perplexity, Reflection, Runway) as active collaborators,

without results [32].[V] The evidence therefore does not show widespread adoption by independent laboratories for *training*, only for NVIDIA's own models, as of the date of this report. We stress that absence from the literature does not prove absence in practice: closed laboratories may train in FP4 without publishing.[U]

Vendor-attributed performance claims. The $7\times$ GEMM figure compares Blackwell Ultra FP4 with Hopper (a different precision); the $7.5\times$ figure divides an NVFP4 number by an FP8 number; the $50\times$ energy figure combines architecture, cooling and format; the MLPerf $3\times$ combines NVFP4 and software (Chapter 11). Each is individually defensible as a statement about a product and each is open to misreading as a statement about the format.[V]

Simulation versus hardware. Several research results (Ascend HiF4, many Quartet-style studies, and parts of the Graphcore work) use emulated quantisation on hardware without native FP4. The comparative study and the real-kernel results of MR-GPTQ suggest that simulation tracks hardware within a few tenths of a percent for inference [12], but for training the match between emulation and native kernels (rounding of accumulation, RNG streams, tile ordering) has been checked in few places; Graphcore reports a probe-matched emulator that reproduces native model behaviour in deterministic controls [14].[S][U]

16.6 What the community does not seem to be arguing about

Equally informative is what is missing from the sources. There is little public challenge to the basic mathematics (block scaling, E4M3 scale, SR for gradients), no published refutation of the headline 12B result and no public report of a failed attempt to replicate it at scale. That may be because replication at 10T tokens is beyond almost everyone's budget, and because negative results are rarely published. The absence of a refutation is not evidence of replication.[U]

16.7 Summary

Reception outside the research literature is mostly a restatement of the vendor's framing with modest scepticism about its scope; practitioners are positive about NVFP4 as an inference checkpoint format on large models and sceptical about quality for small models and agentic coding, in anecdote only. The research literature is where the substantive debate occurs, centred on which recipe components matter, whether integer formats are better, and how much of the loss gap is acceptable. We rate our evidence for the first as low and for the second as medium.

Chapter 17

Case Studies

This chapter examines the production or near-production uses of 4-bit formats for which public documentation exists: Nemotron 3 Super (NVFP4 pretraining and PTQ), DeepSeek-V4 (MXFP4 QAT for experts and for the attention indexer), gpt-oss (MXFP4 weights), Kimi K2 Thinking (INT4 QAT), and applications of NVFP4 beyond text LLMs described in an NVIDIA research blog. The cases differ along the axes that matter: which tensors are 4-bit, whether 4-bit is used in training or only at deployment, and whether the quantisation is post hoc or trained in.

17.1 Summary comparison

17.2 Nemotron 3 Super

17.2.1 Pretraining

The Nemotron 3 Super technical report and the associated Nemotron documentation describe a 120.6B-parameter hybrid mixture-of-experts model with 12.7B active parameters (12.1B excluding embeddings) pretrained with NVFP4 over a 25T-token horizon on B200 GPUs, using the Transformer Engine's NVFP4 GEMM kernels through the cuBLAS backend [15, 50].[S] It is the largest publicly documented NVFP4 pretraining run in our sources, and the only one at a token count above the original 12B experiment's 10T.

The precision assignment is given in the report's Table 3 and the documentation: NVFP4 for all linear layers by default; BF16 for the final 15% of the network, the latent projections, the multi-token-prediction layers, the QKV and attention projections and the embedding layers; and MXFP8 for the Mamba output projection, which the report says mitigates a high incidence of underflows seen when it was quantised to NVFP4 at smaller scales.[S] The recipe is the one of Chapter 8: 2D 16×16 weight scaling, 1×16 blocks for activations and gradients, RHT on Wgrad inputs, SR on gradients.

17.2.2 Findings that go beyond the original paper

Three findings are new relative to the original NVIDIA report.

1. **Zero weight-gradients.** The growth of zero-valued weight-gradient elements to 7% of parameters, the $3\times$ excess over BF16 at the same horizon in the Nemotron 3 Nano controlled comparison, the reversibility on switching to BF16, and the attribution to underflow in 2D-scaled weight tiles that span high- and low-magnitude channels. We reproduced the mechanism synthetically in Section 15.4.[S][E]

Table 17.1: Production-scale uses of 4-bit formats (as documented by the developers; all claims are the developers' own).

Model	Format	Stage	Tensors in 4-bit	Verification
Nemotron 3 Super (120.6B, 12.7B active) [15]	NVFP4	pretraining, 25T tokens; also PTQ checkpoint	most linear layers (except final 15%, attention projections, etc.)	internal; open weights
DeepSeek-V4-Pro / Flash (1.6T / 284B) [16]	MXFP4	QAT in post-training; native FP4 in RL rollout	routed expert weights; indexer QK path	internal; preview release
gpt-oss-120b / 20b [17]	MXFP4	post-training quantisation of MoE weights	MoE weights ($> 90\%$ of parameters)	open weights
Kimi K2 Thinking [18]	INT4 (weight-only)	QAT in post-training	MoE components	model card; open weights

2. **Healing did not matter downstream.** Promoting all tensors to MXFP8 at 19T tokens, one trillion tokens before annealing, improved the loss trajectory but did not improve downstream accuracy, so the final model uses NVFP4 throughout.[S] This contrasts with the 12B report, where the forward-switch reduced the loss gap, and is consistent with the claim in Chapter 12 that loss and capability gaps need not move together. We note the possibility that the downstream benchmarks at this scale are too coarse to see a 0.5% loss gap.[U]
3. **An NVFP4 PTQ checkpoint with search.** The deployed NVFP4 checkpoint is not the pretraining-format weights; it is a separate PTQ with per-layer precision search (Chapter 10). The decision to quantise a model pretrained in NVFP4 again after post-training indicates that post-training (SFT, RL) is done in higher precision.[S][U]

17.2.3 Accuracy of the deployed checkpoints

Table 8 of the report, partly reproduced in Table 10.8, shows BF16, FP8 and NVFP4 checkpoints within about half a point on the selected benchmarks, and a median relative accuracy of 99.8% for the NVFP4 PTQ checkpoint. The report also shows the SSM-cache verbosity result, a non-GEMM low-precision effect (Chapter 10). The report does not document run-to-run variability of the evaluations, and the model, the recipe and the benchmarks are all by the same organisation.[S][U]

17.3 DeepSeek-V4

DeepSeek-V4 is a preview series of two MoE models, V4-Pro (1.6T parameters, 49B activated) and V4-Flash (284B, 13B activated), pretrained on 33T and 32T tokens respectively, with context length up to one million tokens [16].[S] Its use of 4-bit arithmetic differs from the NVFP4 pretraining recipe in three ways.

4-bit is applied in post-training by QAT, not in pretraining. The report introduces FP4 quantisation-aware training during post-training for the MoE expert weights, which it describes as a major source of GPU memory occupancy, and for the query–key path of the lightning indexer of its compressed sparse attention, where QK activations are cached, loaded and multiplied entirely in FP4 to accelerate long-context score computation [16].[S] The index scores are additionally cast from FP32 to BF16, giving a $2\times$ speed-up of the top-k selector with 99.7% recall of KV entries.[S]

The format is MXFP4, simulated through FP8. The FP32 master weights are quantised to FP4 and dequantised back to FP8 for computation. The authors state that this dequantisation is lossless because FP8 (E4M3) has two more exponent bits than FP4 (E2M1), as long as the ratio between the largest and smallest FP4 sub-block scale (1×32 tiles) within each FP8 quantisation block (128×128 tiles) does not exceed a threshold, which they verified empirically on the current weights. The consequence is that the entire QAT pipeline reuses the existing FP8 training framework, with the backward pass computing gradients against the same FP8 weights and passing them to the FP32 master weights, equivalent to a straight-through estimator. During RL rollout and inference, which need no backward pass, native FP4 weights are used, so that sampling is consistent with deployment.[S]

Peak FLOPs are the same on current hardware. The report says FP4×FP8 operations have the same peak FLOP/s as FP8×FP8 on existing hardware, and could theoretically be one third more efficient on future hardware; on current hardware the FP4 experts therefore reduce memory and memory traffic rather than compute [16].[S]

The case is a counterpoint to the NVFP4 pretraining story: a laboratory that trains at scale in FP8 and gets the 4-bit memory benefit through QAT of the weights that dominate memory. It also gives a direct industrial counterpart to the QAT-versus-PTQ point (Chapter 10). The report does not publish an accuracy comparison between the FP4 QAT and the unquantised model in the sections we extracted, so we cannot assess the degradation it incurs.[U]

17.4 gpt-oss

The gpt-oss model card states that the MoE weights, which are responsible for more than 90% of the parameter count, are quantised to MXFP4 at 4.25 bits per parameter, as part of post-training. The 120B model then fits on a single 80 GB GPU and the 20B model can run in as little as 16 GB; the checkpoint sizes are 60.8 GiB and 12.8 GiB [17].[S] The model card does not report how the quantisation affects benchmark results in the sections we extracted, and does not say whether other components are quantised. The significance for this report is that MXFP4, the format that scores worse than NVFP4 in our synthetic and real-weight MSE experiments, is the format of one of the most widely used open-weight releases, and that this is a case where the weights adapt to the format (post-trained with quantisation), which reduces the benefit of the NVFP4 scale format as discussed in Chapter 14.[U]

17.5 Kimi K2 Thinking

The Kimi K2 Thinking model card says that INT4 quantisation-aware training is used during post-training to support native INT4 inference, with INT4 weight-only quantisation applied to the MoE components; it reports a roughly $2\times$ generation speed-up in low-latency mode and does not state the group size [18].[V] It is thus a weight-only scheme on hardware that does not need 4-bit tensor cores. A company blog discussed in Chapter 16 claims that Kimi's QAT extends through post-training including RL whereas another laboratory's INT4 QAT covered only supervised fine-tuning; we did not verify this against primary sources.[U]

17.6 NVFP4 beyond text: video, KV cache, attention

NVIDIA's research blog of 29 June 2026 collects applications beyond language-model GEMMs [48]; all of the following are vendor-reported ([V]).

- **Video generation.** LongLive-2.0, built on Wan2.2-TI2V-5B, is described as the first end-to-end NVFP4 recipe for long video: 720p generation at 45.7 frames per second; a W4A4 frozen NVFP4 backbone with LoRA adapters for distillation, with per-GPU peak memory falling from 70.5 to 49.0 GB; inference peak memory from 36.4 to 19.4 GB and throughput from 24.8 to 32.0 FPS. Per-iteration time for 64-second autoregressive training falls by up to $2.1\times$ against BF16 with sequence parallelism.
- **NVFP4 KV cache.** A cache up to about 50% smaller than FP8 with under 1% accuracy loss and about 5% better than an MXFP4 cache, with dequantisation overhead below 2% using a fused kernel; K-smoothing (subtracting channel means before quantisation) is used.
- **FP4 attention.** SageAttention3 uses NVFP4 for both matrix products on Blackwell, with the attention probabilities (in $[0, 1]$) rescaled per token before 4-bit quantisation so that the cosine similarity of the probability map rises from 93.3% to 99.5% (NVFP4) against 98.37% for MXFP4; it reaches 1038 TOPS on an RTX 5090, about $5\times$ FlashAttention-2, and 2.4–$3\times$ end-to-end on video diffusion. The blog cautions that FP4 attention still carries more accuracy risk than FP4 GEMMs.

These applications show the scope of the format beyond LLM linear layers. They are, however, presented with limited methodology and no independent replication in our source set. The FP4 attention case is the most interesting from a numerical viewpoint: the post-softmax probabilities are non-negative and bounded by one, so the sign bit of E2M1 is wasted and the dynamic-range issues are those of a scale chosen for a $[0, 1]$ distribution; the blog's rescaling trick involves the constant 448×6, the same constant that appears in the two-level encode scale of Section 3.3.2, which we read as fixing the scaled maximum in advance because the maximum of a row of probabilities is not known to be one.[U]

17.7 Lessons across the cases

1. **Where 4-bit goes depends on what dominates cost.** Experts in MoE models dominate memory, so every MoE case quantises the experts; the dense-GEMM compute story is secondary on today's hardware (DeepSeek-V4 states so explicitly).[S]
2. **Trained-in quantisation beats post-hoc.** All three production cases other than NVIDIA's PTQ checkpoint use QAT or native training, and the one failure story (Chapter 16) is a PTQ of a model not trained for 4-bit.[S][U]
3. **Format choice follows ecosystem.** The NVIDIA model uses NVFP4, the others MXFP4 or INT4. The data do not show a quality reason; they show an ecosystem pattern.[U]
4. **Exemptions are universal.** No case quantises everything: in each documented case some components stay in higher precision (final layers, attention projections, embeddings, MTP layers, the Mamba output projection, the indexer scores or the recurrent state in the examples above).[S]

Chapter 18

Open Problems and Outlook

The preceding chapters leave a set of questions whose answers are not in the literature we retrieved. This chapter lists them, explains what evidence would settle each, and gives our assessment of the likely direction. Predictions are labelled as such ([U]); nothing here is a result.

18.1 Open problems

18.1.1 Independent replication at scale

The one multi-trillion-token NVFP4 result is from the format's proposer; the second (Nemotron 3 Super at 25T tokens) is from the same company. The comparative study at 1T tokens is independent but covers two architectures and small effects. What is missing is a replication by a group with different data, architecture and optimiser at $\geq$ 1T tokens that reports seeds and full ablations. Without it the strongest accurate statement is "works for NVIDIA's models".[U] The cost of such a replication is on the order of the compute budget of a mid-sized laboratory; the information it would give is high.

18.1.2 Removing the exemptions

The recipes keep 15–16% of linear layers, attention, embeddings and the head in higher precision. Whether those exemptions can be eliminated is contested (Table 12.3): Graphcore trains all eligible linears in FP4 with UE5M3 scales and periodic scaling, while the comparative study reports divergence beyond 300B tokens at 8B. A systematic study should vary the exemption set while holding everything else fixed, and measure the per-layer sensitivity $\mathrm{tr}(\boldsymbol{H}_\ell \boldsymbol{\Sigma}_\ell)$ of Section 9.1 directly, which would turn the rule of thumb ("the last blocks") into a measured quantity. Our model predicts that the answer depends on layer curvature, which is cheap to estimate with Hutchinson-type trace estimators on a small number of batches, but no source reports having done so.[D][U]

18.1.3 A theory of the late-training gap

Why the gap widens during learning-rate decay is explained qualitatively by the second-order model, but we have no quantitative prediction. A testable form: at fixed token count, the gap should scale with the ratio of quantisation-noise power to gradient-noise power; this predicts that doubling the batch size (halving gradient noise) increases the gap, and that the healing benefit depends on the forward-pass share of noise. No source tests this. Our toy training experiments (Chapter 13) are too short and small to say anything about it.[D][U]

18.1.4 Scale-format design

We have exact error formulas for absmax scaling (Section 4.3) and a model for the E4M3 rounding penalty (Section 4.5.1) that is accurate to a few percent at small block sizes but over-predicts by 20–30% for $G \geq 32$; the discrepancy is unexplained and probably reflects the correlation between the rounding of the scale and the position of the block maximum. A more accurate model would allow principled design of the scale format (mantissa bits, exponent bits, signed or unsigned, bias) as a function of the target tensor statistics. UE5M3, HiF4's hierarchy and Four Over Six are three points in this design space; a joint optimisation under a 4.5-bit budget has not been done in the literature we found.[U]

18.1.5 When to rotate

Rotation improves SR dot-product variance on spiky data by $6.6\times$ at $d = 16$ and about $22\times$ at $d = 128$ in our experiments, but hurts NVFP4 RTN on heavy-tailed data and shows no benefit in the one-step weight-gradient error metric (Chapter 6). The recipes disagree on where to rotate (Wgrad only, Wgrad and Dgrad, nowhere, deterministic). A rule that predicts, from a tensor's measured statistics, whether to rotate would be valuable. Our analysis suggests the criterion is the block crest-factor distribution and whether the downstream use is variance-limited (SR products) or bias-limited (RTN weights), but we did not test it on real activations from a trained large model.[U]

18.1.6 Low-precision optimiser states, communication and attention

Everything in this report concerns GEMM operands. The FP4 attention of SageAttention3 (cosine similarity of the probability map 99.5% with NVFP4) and the NVFP4 KV cache are vendor-reported with limited methodology [48]. Optimiser states in FP4, FP4 all-gather and reduce-scatter of gradients, and 4-bit master weights have been explored in part (the full-stack FP4 study uses an FP4 optimiser [9]) but without a systematic accounting of their errors. A full-stack picture of where bits can be removed, and the loss cost per tensor class, does not exist.[U]

18.1.7 Reinforcement learning with FP4 rollouts

DeepSeek-V4 uses native FP4 weights in RL rollouts so that sampling matches deployment [16], and the QAD report shows that task-loss QAT damages RL-trained models [47]. The training–inference mismatch in RL (a policy sampled from one numerical implementation and optimised in another) is a known source of instability in the RL literature; how much of it 4-bit adds, and whether a 4-bit rollout policy needs importance-weight corrections of a different size from an 8-bit one, are open questions on which we found no data.[U]

18.1.8 Joint scaling laws

Quartet's fit of parameter and data efficiencies is the only published attempt to place low-precision training in a scaling-law framework (Chapter 9). We showed that the three published efficiencies are mutually consistent with $\ln(1/\mathrm{eff}_N) \propto \mathrm{MSE}$ ([D]), a one-parameter form that could be tested with more points. What is missing is a law that predicts the gap as a function of (N, D, format) validated at scales above a few billion parameters; the evidence at 10T tokens is a single point.[U]

Table 18.1: Relative MSE of block-scaled integer quantisation with E4M3 scales, $G = 16$, round-to-nearest, 2^{21} samples (own experiment), and NVFP4 for reference.

	NVFP4	INT2	INT3	INT4	INT5	INT6	INT8
Gaussian	0.00905	0.343	0.0398	0.00746	0.00175	0.00048	0.00012
Laplace	0.00859	0.327	0.0531	0.01051	0.00249	0.00070	0.00018

18.1.9 Evaluation methodology

We showed that the downstream tables in the headline comparison cannot distinguish the two precisions on most benchmarks (Chapter 12), and that cross-entropy hides drift that KL reveals (Chapter 10). Open: a standard battery of long-horizon, agentic and calibration-sensitive evaluations with reported intervals, run on checkpoints in the precision in which they will be served. The community's anecdotes about agentic degradation (Chapter 16) show that this is where practitioners look first, and where published benchmarks say least.[U]

18.1.10 Hardware co-design

Whether future accelerators should add native INT4/MXINT8 paths alongside FP4, support wider-range scale formats natively, offer hardware stochastic rounding (the TE documentation says SR is hardware-accelerated on Blackwell [35]) or hardware Hadamard units is a design question to which the arithmetic in Chapter 11 (quantisation overhead of order 0.1–$1\times$ the GEMM time for unfused kernels) contributes: the cost of making 4-bit operands is comparable with the saving in multiplying them, so operand-formation hardware may be worth more than faster multipliers.[D][U]

18.2 Below four bits?

A natural extrapolation, voiced even in an informal social-media comment, is to ask whether the approach extends to two or three bits. We computed the relative MSE of block-scaled integer grids of $b = 2$–8 bits with E4M3 scales and $G = 16$ on Gaussian and Laplace data (Table 18.1). Each bit removed multiplies the error by about 4 in the regime where the uniform-quantisation approximation holds; going from 4 to 3 bits multiplies it by 5.3 on Gaussian data and from 3 to 2 bits by 8.6.[E]

If the loss gap scales linearly with relative MSE, as the second-order model of Chapter 9 and the Quartet fits suggest, a 3-bit block format would have a loss gap about five times that of 4-bit, so that a 1.5% gap becomes roughly 8%, equivalent by the slope of (9.4) to a data penalty of $e^{0.08/0.033} \approx 11\times$ in tokens (if the local slope applied over so large a change, which it would not); a ternary format (≈ 0.34 MSE) is far outside that regime and could only work with training that adapts weights to the grid (QAT) rather than with post-hoc quantisation or with quantisation of all GEMM operands.[D][U] This is an extrapolation from a toy model at one operating point and should not be read as a forecast, but it explains why proposals for sub-4-bit training generally apply to weights only, with higher-precision activations.

18.3 Outlook

We offer five judgements, with the caveat that they are predictions.

1. **NVFP4 inference will become routine for large models and remain contested for small ones.** The size trend in the PTQ data is consistent across sources, and QAD is a documented remedy.

Expect checkpoints released in NVFP4 by default for 70B-plus models.[U]

2. **Pretraining in 4-bit will spread slowly.** The benefit is bounded by Amdahl's law and the κ penalty (Section 9.4); a gap of 1–1.5% is worth 1.35–1.6× in tokens by our slope-based estimate, which absorbs much of a 2× GEMM speed-up. Groups with FP8 infrastructure and good FP8 baselines will adopt 4-bit if the late-training gap closes.[D][U]

3. **The recipes will simplify.** The direction of the 2026 papers (Graphcore, HiF4, Four Over Six, transposition-invariant scaling) is fewer stabilisers via better formats and scale choices, not more. Format design (wider-range scales, hierarchical scales, error-driven scale choice) is the active frontier.[S][U]

4. **The integer–float question will be decided by hardware economics, not by accuracy alone.** At 4 bits the accuracy comparison is close, workload-dependent and sensitive to rotation; area and energy per MAC will decide.[U]

5. **The evidence base will improve only if reporting does.** Seeds, intervals, deployment-precision evaluations and open recipes are the cheapest improvements and the most neglected.[U]

Chapter 19

Conclusions

This report examined NVFP4 and related block-scaled 4-bit floating-point formats from three angles: the mathematics of the quantiser, the published empirical record of September 2025 to October 2026, and a set of small numerical experiments of our own. We summarise what we consider established, what we consider plausible, and what remains open, and then list the limitations that readers should keep in mind.

19.1 What is established

1. **The format advantage over MXFP4 is mathematical, and mostly due to the scale.** Exact expected-MSE formulas for absmax-scaled E2M1 blocks (Proposition 4.2) agree with Monte Carlo to three or four digits. At equal storage an E4M3-scaled block with $G = 32$ already has 22% lower MSE than MXFP4 on Gaussian data; halving the block size adds roughly another 12%. On real weights NVFP4 has about 37% lower relative MSE than MXFP4 (SmolLM2-135M, ceil rule) and the perplexity penalty of W4A16 quantisation of the small model is smaller by a factor of about 2.3 in log-perplexity.[D][E]

2. **The rounding mode matters through bias, not variance alone.** Round-to-nearest is a biased rounding for the small, dithering-dependent gradients of training: in our toy it maps a small positive signal to a zero mean, where stochastic rounding recovers it (Table 5.2). Stochastic rounding is unbiased only with scale headroom (the $16/17$ factor), which costs 25% in MSE on Gaussian data.[D][E]

3. **Rotations are not a free lunch.** A random Hadamard transform does not lower the error of NVFP4 round-to-nearest and in our experiments increases it on heavy-tailed data, because it turns blocks that FP4 represents cheaply (one large value) into flat blocks that sit in the coarse upper cells. It does reduce the variance of stochastic-rounding dot products on spiky data by factors from 6.6 to about 22 (rising with the rotation size), which is where the literature applies it.[E]

4. **Scale range is a real limit.** The E4M3 block scale loses precision when a block's maximum is below 3.5×10^{-5} of the tensor's and flushes below 2.2×10^{-6}; with 2D weight tiles a channel $10\times$ smaller than its neighbours loses about four-fifths of its entries, and one $30\times$ smaller is erased (Section 4.7, Section 15.4). Total MSE hides this.[D][E]

5. **Inference accuracy is near-lossless at large scale and not at small scale.** Across vendor, third-party and academic sources, NVFP4 PTQ of models above 30B parameters recovers 97–99% or more of BF16 accuracy, smaller models lose more, and RL-trained reasoning models lose most under task-loss QAT while distillation-based QAT recovers them (Chapter 10).[S][V]

19.2 What is plausible but thinly supported

1. **Large-scale pretraining in NVFP4 is feasible with a loss gap of about 1–1.5%.** The evidence is a single 12B run to 10T tokens from the proposing group, with downstream differences that our noise audit shows are statistically indistinguishable from zero on most benchmarks (Chapter 12), and a 120B-parameter model whose NVFP4 parity is asserted but whose details are internal (Chapter 17). An independent comparative study at 1T tokens finds gaps below 1.2% (dense) and 0.81% (MoE) with a refined recipe and divergence when all layers are quantised.[S]
2. **Which parts of the recipe are necessary is unresolved.** Rotation of the weight-gradient inputs, 2D weight scaling, stochastic rounding and the BF16 exemptions each have defenders and critics; sources disagree, and the studies do not share a setting (Chapter 8).[S]
3. **The end-to-end economic benefit is bounded and not measured.** Peak throughput of FP4 is 2–$3\times$ that of FP8, but quantisation overhead, Amdahl effects and the token penalty of the loss gap reduce it (Chapter 11, Section 9.4). No independent end-to-end training throughput measurement against a tuned FP8 baseline exists in our source set.[D][U]

19.3 What remains open

Replication of the headline results by independent groups at the trillion-token scale; a measured, per-layer account of which layers need high precision; a quantitative theory of the late-training gap; joint design of scale formats under a 4.5-bit budget; FP4 for attention, KV caches, optimiser states and communication; the interaction of 4-bit numerics with reinforcement learning; and the integer-versus-floating-point hardware question (Chapter 18).

19.4 Limitations of this report

- **No hardware.** We had no access to Blackwell or any GPU. All our experiments are float32 CPU fake-quantisation simulations. They test the numerical definitions of the quantisers, not kernels, accumulator widths or throughput.
- **Small scale.** The real-weight experiments use two models of 135M and 0.5B parameters; the toy training uses a byte-level model of about a million parameters for 500 steps. They cannot speak to behaviour at 10^{10} parameters and 10^{13} tokens.
- **Vendor and secondary sources.** Hardware, energy and several accuracy numbers are the manufacturer's; accuracy numbers from NVIDIA's blog were read from chart labels. Some sources are trade articles, newsletters and company blogs, which we marked and weighted accordingly.
- **Thin community evidence.** Our evidence on perception comes from a handful of articles and two social-media comments; we did not retrieve forum threads such as Hacker News.
- **Single-seed literature.** Most 2026 papers we cite report single runs; where seed variance is reported we used it, and where it is not our statements about ranking are provisional.
- **Retrieval cut-off and coverage.** The record is as retrieved up to 27 October 2026. Papers we did not find, and results presented only in talks or code repositories, are not represented. Some sources were read only in part (abstract, selected tables), as indicated in the text.
- **Unverified recollections.** Test-set sizes used in the noise audit of Chapter 12 are from memory and marked uncertain.

19.5 Closing assessment

In a year NVFP4 has gone from a vendor announcement to the numeric format of a 120B-parameter model trained on 25T tokens, a standard checkpoint format for large-model inference, and a baseline against which every 4-bit proposal is measured. The mathematics supports the design choices that distinguish it from MXFP4. The evidence that it can replace FP8 for frontier-scale pretraining without cost is real but concentrated in one organisation's runs; the evidence that it is a safe default for small models and reasoning-heavy post-trained models is weaker. The most useful contribution the community could make in the next year is not another recipe but independent, seeded, deployment-precision replications with shared baselines.

Appendix A

Supplementary Derivations

This appendix collects derivations that are used in the main text but whose details would interrupt it, and adds a numerical check where a closed form is claimed. Results in the main chapters that have proofs there (Proposition 4.2, Proposition 5.1, Proposition 5.2, Proposition 6.1, Corollary 6.3) are not repeated.

A.1 Dynamic range of the block-scaled formats

The "number of binades" of a format is $\log_2$ of the ratio between its largest and smallest non-zero representable magnitudes. This is the quantity that the HiF4 study uses when it says NVFP4 covers 22 binades against 69 for HiF4 [19], and we derive it here for the formats of this report.

E2M1 element. The non-zero magnitudes are $0.5, \ldots, 6$, so the ratio is 12 and the element alone spans $\log_2 12 = 3.58$ binades.

E4M3 scale. The OCP E4M3 format has exponent bias 7, a maximum finite value of 448, and subnormals with spacing 2^{-9}, so the smallest positive value is 2^{-9}. The ratio is $448 \cdot 2^9 = 229{,}376$, or $\log_2 = 17.81$ binades. For an *unsigned* E4M3 scale (the NVFP4 scale is non-negative) the same values apply.

NVFP4 without the tensor scale. The largest representable magnitude is $6 \cdot 448 = 2688$ and the smallest is $0.5 \cdot 2^{-9} = 2^{-10}$. The ratio is $2688 \cdot 1024 = 2.75 \times 10^6$, i.e. $\log_2 = 21.4$ binades, which rounds to the 22 quoted by the HiF4 authors. With the FP32 tensor scale the coverage is extended by the dynamic range of the tensor scale, i.e. by the range of FP32 ($\approx 2^{\pm 126}$ for normals): the *static* range is huge, but the *simultaneous* range within a tensor remains 21.4 binades because every block scale is relative to the same tensor scale. This is the content of Section 4.7.

MXFP4. The E8M0 scale spans 2^{-127} to 2^{127} (254 binades) and with the element range 257.6 binades. No tensor scale is needed, which is the main design advantage of the power-of-two scale: it never underflows, at the price of the rounding inefficiency analysed in Section 4.5.

UE5M3 scale. With five exponent bits and bias 15 the normal range is 2^{-14} to $2^{16} \cdot (2 - 2^{-3})$, a ratio of about 2^{31} for normals alone; including subnormals with spacing 2^{-17} the ratio becomes about 2^{33} (33 binades), and with the element 36.6 binades. This relies on the conventional IEEE-like bias and normal/subnormal convention, which the cited paper may modify; the point is the order of magnitude.[U]

Table A.1: Simultaneous dynamic range in binades (our arithmetic, [D]). "Block scale only" is the span of the scale format; "with element" adds the 3.58 binades of E2M1.

Scale format	block scale only	with element
E4M3 (NVFP4, within one tensor scale)	17.8	21.4
E8M0 (MXFP4)	254	257.6
UE5M3 (conventional bias; [U])	≈ 33	≈ 36.6

A.2 Overflow-free tensor scale with the Four Over Six rule

With the standard encode scale $s_{\text{enc}} = 6 \cdot 448/A_x$ the value presented to the E4M3 rounder for a block with maximum A_b is $v_b = 448A_b/A_x \leq 448$ (Equation (4.7)), so no block scale overflows. If a block may be scaled to a target maximum of 4 instead of 6, the block scale is $A_b/4$, i.e. 1.5 times larger than $A_b/6$, and the presented value is $1.5 \cdot 448A_b/A_x$, which exceeds 448 for blocks with $A_b > A_x/1.5$. For the block containing the tensor maximum it would be 672 and overflow. The fix adopted by Cook et al. [7] replaces 448 by 256 in the tensor-scale computation: the presented values are then at most 256 (target 6) or $1.5 \cdot 256 = 384$ (target 4), both below 448. The price is that the available range of the block scale below the top is reduced by $\log_2(448/256) = 0.81$ binades, i.e. the thresholds of Section 4.7 move up by a factor 1.75 (to 6.1×10^{-5} for the loss of precision and 3.9×10^{-6} for flushing).[D]

A.3 The gain of a rounding quantiser

Let $\hat{\boldsymbol{x}} = \mathcal{Q}(\boldsymbol{x})$ and $\boldsymbol{e} = \hat{\boldsymbol{x}} - \boldsymbol{x}$. The *gain* of the quantiser and the energy ratio are

$$g = \frac{\langle \hat{\boldsymbol{x}}, \boldsymbol{x} \rangle}{\|\boldsymbol{x}\|^2} = 1 + \frac{\langle \boldsymbol{e}, \boldsymbol{x} \rangle}{\|\boldsymbol{x}\|^2}, \qquad r = \frac{\|\hat{\boldsymbol{x}}\|^2}{\|\boldsymbol{x}\|^2} = 1 + \frac{2\langle \boldsymbol{e}, \boldsymbol{x} \rangle + \|\boldsymbol{e}\|^2}{\|\boldsymbol{x}\|^2}. \tag{A.1}$$

Eliminating $\langle \boldsymbol{e}, \boldsymbol{x} \rangle$ gives

$$r = 2g - 1 + \text{MSE}, \qquad \text{i.e.} \qquad g = \frac{1 + r - \text{MSE}}{2}. \tag{A.2}$$

The identity holds for any quantiser and any distribution. For a *centroid* quantiser (each output equal to the conditional mean of its cell, as at a Lloyd–Max optimum) the error is orthogonal to the output, $\langle \boldsymbol{e}, \hat{\boldsymbol{x}} \rangle = 0$, which yields $g = r = 1 - \text{MSE}$: it shrinks the signal by exactly its error. Round-to-nearest onto a fixed grid is not a centroid quantiser and the cell centroids for a bell-shaped density lie closer to zero than the cell midpoints, so g can differ from $1 - \text{MSE}$. We used (A.2) to check the Gaussian entries of Table 7.2: for NVFP4 RTN, $2g - 1 + \text{MSE} = 2(0.9955) - 1 + 0.00904 = 1.0001$ against the measured $r = 1.0001$; for MXFP4 with the ceil rule $2(0.9898) - 1 + 0.01334 = 0.9930$ against 0.9930; for NVFP4 with SR $2(0.9968) - 1 + 0.01883 = 1.0124$ against 1.0124. The table is therefore internally consistent. The consequence discussed in Chapter 7 is that the sign and size of $g - 1$ depend on the grid and scale rule: shrinkage for NVFP4 RTN (0.45%) and a larger one for MXFP4 with the ceil rule (1.0%), a slight gain for INT4 (+0.06%), and near-unity for SR with headroom (0.03%).[D][E]

A.4 Stationary excess loss of SGD on a quadratic

We derive the statement used in Chapter 9 and Chapter 15 that the loss excess due to optimiser noise scales with the learning rate while that due to forward-pass quantisation noise does not.

Setup. Let $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \boldsymbol{H}\boldsymbol{\theta}$ with $\boldsymbol{H} = \operatorname{diag}(h_1, \dots, h_d)$, and SGD with step η and gradient noise of covariance $\boldsymbol{C} = \sigma^2 \boldsymbol{I}$:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta(\boldsymbol{H}\boldsymbol{\theta}_t + \boldsymbol{\xi}_t), \qquad \boldsymbol{\xi}_t \sim (0, \boldsymbol{C}). \tag{A.3}$$

Each coordinate is an AR(1) process, $\theta_{t+1} = (1 - \eta h)\theta_t - \eta\xi_t$, with stationary variance

$$\operatorname{Var}(\theta) = \frac{\eta^2\sigma^2}{1 - (1 - \eta h)^2} = \frac{\eta\sigma^2}{h(2 - \eta h)}. \tag{A.4}$$

The expected excess loss in coordinate i is $\frac{1}{2}h_i \operatorname{Var}(\theta_i) = \frac{\eta\sigma^2}{2(2-\eta h_i)}$ and the total is

$$\mathbb{E}[\mathcal{L}] = \sum_i \frac{\eta\sigma^2}{2(2 - \eta h_i)} \xrightarrow{\eta h_i \ll 1} \frac{\eta\sigma^2 d}{4} = \frac{\eta \operatorname{tr}(\boldsymbol{C})}{4}. \tag{A.5}$$

This is independent of $\boldsymbol{H}$ at leading order, which is the standard result that the stationary loss of SGD with a constant step is proportional to the step and the noise level.

Numerical check. We simulated 256 independent chains in dimension $d = 64$ with h_i logarithmically spaced between 0.1 and 10, $\sigma = 1$, for $400/\eta$ steps (forty relaxation times of the slowest mode), and averaged the loss over the second half ([E]). Table A.2 shows the ratio of the empirical excess to the leading-order prediction $\eta\sigma^2 d/4$: it approaches one as $\eta \to 0$ (1.010 at $\eta = 0.0125$), and exceeds one by the factor $\frac{1}{d}\sum_i 2/(2 - \eta h_i)$ predicted by the exact AR(1) sum at larger steps (at $\eta = 0.1$, $h_{\max} = 10$ so $\eta h_{\max} = 1$).

Table A.2: Excess loss of SGD on a noisy quadratic ($d = 64$, $h_i \in [0.1, 10]$, $\sigma = 1$): simulation against $\eta\sigma^2 d/4$ (own experiment).

η	simulated	$\eta d\sigma^2/4$	ratio
0.1	1.847	1.600	1.155
0.05	0.851	0.800	1.064
0.025	0.412	0.400	1.029
0.0125	0.202	0.200	1.010

Comparison with quantisation noise. Forward-pass weight-quantisation noise with covariance $\boldsymbol{\Sigma}_q$ (independent of the step size) adds $\frac{1}{2}\operatorname{tr}(\boldsymbol{H}\boldsymbol{\Sigma}_q)$ to the loss at any η (Equation (9.1)), so the ratio of quantisation-noise excess to SGD-noise excess is

$$\rho(\eta) = \frac{\frac{1}{2}\operatorname{tr}(\boldsymbol{H}\boldsymbol{\Sigma}_q)}{\eta \operatorname{tr}(\boldsymbol{C})/4} = \frac{2\operatorname{tr}(\boldsymbol{H}\boldsymbol{\Sigma}_q)}{\eta \operatorname{tr}(\boldsymbol{C})} \propto \frac{1}{\eta}. \tag{A.6}$$

In a learning-rate decay in which η falls by a factor F (for example a decay to 10% of the peak, $F = 10$), the relative weight of quantisation noise grows by F while its absolute size is unchanged. This is a derivation, not a measurement on a language model: in a real network the SGD-noise term is not independent of $\boldsymbol{H}$ and the noise covariance changes during training, and the weights adapt to the quantisation noise so that the effective $\boldsymbol{\Sigma}_q$ is smaller than the nominal one (Section 9.1). The result supports the *direction* of the late-training effect only.[D][U]

A.5 Monte Carlo uncertainty of the synthetic experiments

The relative-MSE values in Chapter 4 are Monte Carlo estimates over 2^{21} samples. To state their uncertainty we repeated three representative quantisers 32 times on independent samples of 2^{16} values and computed the standard error of the mean of the 32 results ([E]), which is then scaled to the full sample as $\mathrm{SE}/\sqrt{32}$ per chunk size; the numbers in Table A.3 are the standard errors of the mean of the 32 chunk values, i.e. for a total of 2^{21} samples.

Table A.3: Mean and standard error of relative MSE over 32 independent chunks of 2^{16} samples (2^{21} in total).

Distribution	Quantiser	mean	standard error
Gaussian	NVFP4 (G=16)	0.009062	1.1×10^{-5}
Gaussian	MXFP4 floor (G=32)	0.013234	1.8×10^{-5}
Gaussian	NVFP4 + 4/6	0.007557	5.2×10^{-6}
Laplace	NVFP4 (G=16)	0.008573	1.3×10^{-5}
Laplace	MXFP4 floor (G=32)	0.016089	2.5×10^{-5}
Laplace	NVFP4 + 4/6	0.007594	7.9×10^{-6}

Relative standard errors are 0.07–0.16%. The differences between formats reported in the text (for example 0.00904 against 0.01329) are therefore many standard errors wide, and differences in the third significant digit between entries computed in different runs (such as 0.00904 in one table and 0.00906 here) are consistent with sampling noise of this size. Note that these are the errors of the *estimator of expected MSE* under a stated distribution; they say nothing about the representativeness of the distribution for real tensors, which is the larger source of uncertainty.

A.6 The tile-zero probability with a single large channel

We justify the zero-fraction formula of Section 15.4. In a 16×16 NVFP4 weight tile the scale is $\hat{s} \approx A_t/6$ with A_t the tile maximum, and the element is quantised as $\mathrm{RTN}(w/\hat{s})$. The E2M1 grid has zero as its lowest level and 0.5 as the next; the midpoint between them is 0.25, and ties resolve to even, i.e. to zero. An element is therefore zeroed iff $|w|/\hat{s} \le 0.25$, i.e. $|w| \le A_t/24$. Take a low-norm row whose entries are $\mathcal{N}(0, r^2\sigma^2)$, in a tile whose maximum A_t is attained among high-norm rows with $\mathcal{N}(0, \sigma^2)$ entries. The maximum of $n = 256$ absolute standard normals has mean $\bar{A}_{256} = 3.04$ (our Monte Carlo, 2×10^5 tiles); if all rows were high-norm this would be the tile maximum in units of σ. For a low row, $\mathbb{P}(\text{zero}) = \mathbb{P}(r|Z|\sigma \le \bar{A}_{256}\sigma/24) = 2\Phi(\bar{A}_{256}/(24r)) - 1$. The approximation replaces the random A_t by its mean, and ignores that when a large fraction of the tile's rows are low-norm the maximum falls (the measured zero fraction for $f = 0.5$ is 2–5 percentage points lower than predicted at $r = 0.3$ and $r = 0.1$). The 1.0 entries for $r \le 0.03$ are exact to the printed precision because $\bar{A}_{256}/(24 \cdot 0.03) = 4.2$ and $2\Phi(4.2) - 1 = 0.99997$.[D][E]

A.7 Variance of the quantisation-error estimate in a GEMM output

This section records the algebra behind (5.6). For a dot product $y = \sum_{k=1}^{K} a_k b_k$ with independent zero-mean relative errors α_k, β_k of variance $\varepsilon_a, \varepsilon_b$ on the operands, the perturbation of the output is, to first order, $\Delta y = \sum_k (a_k b_k)(\alpha_k + \beta_k)$, with variance $\sum_k (a_k b_k)^2(\varepsilon_a + \varepsilon_b)$ if the errors are independent across k. The relative error of the output is its standard deviation divided by $|y|$; for random-sign products $|y|$

is of order $\sqrt{\sum_k (a_k b_k)^2}$ so

$$\frac{\sqrt{\mathrm{Var}(\Delta y)}}{|y|} \approx \sqrt{\varepsilon_a + \varepsilon_b}, \tag{A.7}$$

independent of K. The independence is the reason that the relative GEMM error does not average down with the contraction length. When the product has a systematic component (the mean of $a_k b_k$ non-zero) the signal grows as K while the noise grows as $\sqrt{K}$ and the relative error falls as $1/\sqrt{K}$; we do not rely on that case.[D]

A.8 Notation reference

Symbol	Meaning
G	block size (number of elements sharing one scale)
$A,\ A_b,\ A_x$	absolute maximum of a block, of block b, of the tensor
$s,\ \hat{s}$	ideal and stored (rounded) block scale
δ	relative rounding error of the scale
$Q_{\max}$	largest level of the element grid (6 for E2M1, 7 for INT4)
$\mathrm{MSE},\ \mathrm{QSNR}$	relative mean squared error; $-10 \log_{10} \mathrm{MSE}$
η	normalised error $\mathrm{MSE}\, Q_{\max}^2 / \mathbb{E}[A^2]$ (Chapter 4); also learning rate in Appendix A.4
$\mathrm{RTN},\ \mathrm{SR}$	round-to-nearest(-even); stochastic rounding
$\boldsymbol{H}_d$	normalised Hadamard matrix of order d
κ	block crest factor (Chapter 4); also token-equivalence factor (Chapter 9); context disambiguates
$\mathrm{eff}_N,\ \mathrm{eff}_D$	parameter and data efficiency of Quartet's scaling law
$\phi,\ s$	FP4 share of step time; GEMM speed-up

Appendix B

Simulation Code

All experiments of this report are CPU simulations in PyTorch (float32) of quantisers applied to synthetic or real tensors; no experiment used a GPU or Blackwell hardware. The code is reproduced below so that every [E] number can be regenerated: first the shared quantiser library, then the quantised linear layer used in the toy training experiment, then the experiment scripts that carry the main logic. Scripts that only tabulate or plot are listed by name at the end. File names are relative to the directory `code/` of the report's working tree; results are written to `results/` as JSON. Line numbers refer to the listing, not to the file on disk, and long lines are wrapped.

B.1 Quantiser library (`fp4lib.py`)

```
"""fp4lib: reference fake-quantisation routines for E2M1 microscaled formats.

All functions return *dequantised* tensors (float32) so that the numerical
effect of a format can be studied on any CPU. Nothing here uses a GPU or the
Blackwell hardware conversion instructions; rounding rules follow the
descriptions in the cited primary sources (NVIDIA 2025, OCP MX v1.0).
"""
import math
import torch

FP4_GRID = torch.tensor([0.0, 0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 6.0])
FP4_MAX = 6.0
E4M3_MAX = 448.0


# ---------------------------------------------------------------------------
# element rounding
# ---------------------------------------------------------------------------
def rtn_fp4(x: torch.Tensor) -> torch.Tensor:
    """Round-to-nearest-even onto the E2M1 grid, saturating at +-6.
    Ties go to the neighbour whose 2-bit-exponent/1-bit-mantissa code is even."""
    a = x.abs().clamp(max=FP4_MAX)
    o = torch.zeros_like(a)
    o = torch.where(a > 0.25, torch.full_like(a, 0.5), o) # tie 0.25 -> 0
    o = torch.where(a >= 0.75, torch.full_like(a, 1.0), o) # tie 0.75 -> 1
    o = torch.where(a > 1.25, torch.full_like(a, 1.5), o) # tie 1.25 -> 1
    o = torch.where(a >= 1.75, torch.full_like(a, 2.0), o) # tie 1.75 -> 2
    o = torch.where(a > 2.5, torch.full_like(a, 3.0), o) # tie 2.5 -> 2
    o = torch.where(a >= 3.5, torch.full_like(a, 4.0), o) # tie 3.5 -> 4
    o = torch.where(a > 5.0, torch.full_like(a, 6.0), o) # tie 5.0 -> 4
    return torch.sign(x) * o


def sr_fp4(x: torch.Tensor, gen: torch.Generator = None) -> torch.Tensor:
    """Stochastic rounding onto the E2M1 grid (unbiased for |x|<=6)."""
```

```
    grid = FP4_GRID.to(x.device)
    a = x.abs().clamp(max=FP4_MAX)
    idx = (torch.bucketize(a, grid, right=True) - 1).clamp(0, 6)
    lo, hi = grid[idx], grid[idx + 1]
    p = (a - lo) / (hi - lo)
    u = torch.rand(a.shape, generator=gen, device=a.device)
    o = torch.where(u < p, hi, lo)
    return torch.sign(x) * o


def round_minifloat(x, e_bits, m_bits, bias, max_val):
    """RNE rounding of non-negative reals to an unsigned mini-float with
    subnormals, saturating at max_val (used for E4M3 / E5M3 / E3M4 scales)."""
    a = x.abs().clamp(max=max_val)
    min_exp = 1 - bias
    e = torch.floor(torch.log2(a.clamp(min=1e-38)))
    e = e.clamp(min=min_exp)
    step = torch.pow(2.0, e - m_bits)
    q = torch.round(a / step) * step
    return torch.sign(x) * q.clamp(max=max_val)


def e4m3(x):
    return round_minifloat(x, 4, 3, 7, 448.0)


def ue5m3(x):
    # unsigned E5M3: bias 15, all codes usable, max normal 1.875*2^15 ... we use
    # the IEEE-like definition without reserved codes: max = 1.875 * 2^16 / 2
    return round_minifloat(x, 5, 3, 15, 1.875 * 2 ** 16)


def e8m0_ceil(s):
    return torch.pow(2.0, torch.ceil(torch.log2(s.clamp(min=2.0 ** -127))))


def e8m0_floor(s):
    return torch.pow(2.0, torch.floor(torch.log2(s.clamp(min=2.0 ** -127))))


def e8m0_nearest_log(s):
    return torch.pow(2.0, torch.round(torch.log2(s.clamp(min=2.0 ** -127))))


# ---------------------------------------------------------------------------
# block helpers
# ---------------------------------------------------------------------------
def _blocks(x, B):
    assert x.shape[-1] % B == 0, (x.shape, B)
    return x.reshape(-1, B)


def _round_elem(v, rounding, gen=None):
    return rtn_fp4(v) if rounding == "rtn" else sr_fp4(v, gen)


# ---------------------------------------------------------------------------
# formats
# ---------------------------------------------------------------------------
def quant_fp4_ideal_scale(x, B=16, rounding="rtn", gen=None):
    """E2M1 elements with an *unquantised* (FP32) absmax block scale: the
    idealised upper bound used by much of the analysis."""
    xb = _blocks(x.float(), B)
    amax = xb.abs().amax(1, keepdim=True)
    s = amax / FP4_MAX
    s = torch.where(s == 0, torch.ones_like(s), s)
    q = _round_elem(xb / s, rounding, gen)
    return (q * s).reshape(x.shape)

```

```
105
106 def quant_mxfp4(x, B=32, scale_mode="ceil", rounding="rtn", gen=None):
107     """OCP MXFP4: E2M1 elements, one E8M0 (power-of-two) scale per block.
108     scale_mode: 'floor' = OCP reference (2^(floor(log2 amax)-2), may clip),
109                 'ceil' = round the ideal scale up (no clipping; NVIDIA's advice),
110                 'near' = round log2 of ideal scale to nearest integer."""
111     xb = _blocks(x.float(), B)
112     amax = xb.abs().amax(1, keepdim=True)
113     ideal = amax / FP4_MAX
114     if scale_mode == "floor":
115         s = torch.pow(2.0, torch.floor(torch.log2(amax.clamp(min=2.0 ** -126))) - 2.0)
116     elif scale_mode == "ceil":
117         s = e8m0_ceil(ideal)
118     elif scale_mode == "near":
119         s = e8m0_nearest_log(ideal)
120     else:
121         raise ValueError(scale_mode)
122     s = torch.where(amax == 0, torch.ones_like(s), s)
123     q = _round_elem(xb / s, rounding, gen)
124     return (q * s).reshape(x.shape)
125
126
127 def quant_nvfp4(x, B=16, rounding="rtn", gen=None, tensor_amax=None,
128                 m_fp8=448.0, scale_fmt="e4m3", return_scales=False, headroom=1.0):
129     """NVFP4 as in NVIDIA (2025) App. B: FP32 tensor scale + E4M3 block scale.
130     m_fp8 is the E4M3 range used to derive the tensor scale (448 standard)."""
131     xf = x.float()
132     xb = _blocks(xf, B)
133     gamax = xf.abs().max() if tensor_amax is None else torch.as_tensor(tensor_amax)
134     s_enc = (FP4_MAX * m_fp8) / gamax
135     s_dec = 1.0 / s_enc
136     amax_b = xb.abs().amax(1, keepdim=True)
137     sdec_b = amax_b / (FP4_MAX * headroom)
138     fmt = e4m3 if scale_fmt == "e4m3" else ue5m3
139     stored = fmt(sdec_b * s_enc) # E4M3 stored decode scale
140     ok = stored > 0
141     s_enc_b = torch.where(ok, 1.0 / (stored * s_dec), torch.zeros_like(stored))
142     q = _round_elem(xb * s_enc_b, rounding, gen)
143     deq = q * stored * s_dec
144     deq = torch.where(ok, deq, torch.zeros_like(deq))
145     out = deq.reshape(x.shape)
146     if return_scales:
147         return out, stored
148     return out
149
150
151 def quant_nvfp4_46(x, B=16, select="mse", rounding="rtn", gen=None):
152     """'Four Over Six' (Cook et al. 2025): per block try block-max mapped to 6
153     and to 4, keep the lower-error candidate. Tensor scale uses 256 (not 448)."""
154     xf = x.float()
155     xb = _blocks(xf, B)
156     gamax = xf.abs().max()
157     s_enc = (FP4_MAX * 256.0) / gamax
158     s_dec = 1.0 / s_enc
159     amax_b = xb.abs().amax(1, keepdim=True)
160     cands = []
161     for M in (6.0, 4.0):
162         stored = e4m3((amax_b / M) * s_enc)
163         ok = stored > 0
164         s_enc_b = torch.where(ok, 1.0 / (stored * s_dec), torch.zeros_like(stored))
165         q = _round_elem(xb * s_enc_b, "rtn")
166         deq = torch.where(ok, q * stored * s_dec, torch.zeros_like(q))
167         cands.append(deq)
168     e6 = (cands[0] - xb)
169     e4 = (cands[1] - xb)
170     if select == "mse":
171         m6, m4 = (e6 ** 2).sum(1, keepdim=True), (e4 ** 2).sum(1, keepdim=True)
172     elif select == "mae":
173         m6, m4 = e6.abs().sum(1, keepdim=True), e4.abs().sum(1, keepdim=True)
```

```
    else:
        m6, m4 = e6.abs().amax(1, keepdim=True), e4.abs().amax(1, keepdim=True)
    out = torch.where(m4 < m6, cands[1], cands[0])
    return out.reshape(x.shape)


def quant_nvfp4_2d(x, tile=16, rounding="rtn", gen=None):
    """NVFP4 with 16x16 square scale tiles (used for weights in NVIDIA 2025)."""
    R, C = x.shape
    assert R % tile == 0 and C % tile == 0
    xf = x.float()
    t = xf.reshape(R // tile, tile, C // tile, tile).permute(0, 2, 1, 3).reshape(-1, tile *
        tile)
    gamax = xf.abs().max()
    s_enc = (FP4_MAX * E4M3_MAX) / gamax
    s_dec = 1.0 / s_enc
    amax_b = t.abs().amax(1, keepdim=True)
    stored = e4m3((amax_b / FP4_MAX) * s_enc)
    ok = stored > 0
    s_enc_b = torch.where(ok, 1.0 / (stored * s_dec), torch.zeros_like(stored))
    q = _round_elem(t * s_enc_b, rounding, gen)
    deq = torch.where(ok, q * stored * s_dec, torch.zeros_like(q))
    return deq.reshape(R // tile, C // tile, tile, tile).permute(0, 2, 1, 3).reshape(R, C)


def quant_int(x, bits=4, B=32, scale="fp32", rounding="rtn", gen=None, sym_max=None,
              tensor_scale=False):
    """Symmetric block INT-b quantiser, grid {-(2^(b-1)-1) .. 2^(b-1)-1}.
    scale: 'fp32' (ideal), 'e8m0' (MXINT, rounded up), 'e4m3' (NVINT w/ tensor scale)."""
    qmax = float(2 ** (bits - 1) - 1) if sym_max is None else sym_max
    xf = x.float()
    xb = _blocks(xf, B)
    amax = xb.abs().amax(1, keepdim=True)
    s = amax / qmax
    if scale == "e8m0":
        s = e8m0_ceil(s)
    elif scale == "e4m3":
        gamax = xf.abs().max()
        s_enc = (qmax * E4M3_MAX) / gamax
        s = e4m3(s * s_enc) / s_enc
    s = torch.where(s == 0, torch.ones_like(s), s)
    v = xb / s
    if rounding == "rtn":
        q = torch.round(v)
    else:
        lo = torch.floor(v)
        u = torch.rand(v.shape, generator=gen)
        q = lo + (u < (v - lo)).float()
    q = q.clamp(-qmax, qmax)
    return (q * s).reshape(x.shape)


# ---------------------------------------------------------------------------
# metrics
# ---------------------------------------------------------------------------
def rel_mse(x, xq):
    x = x.double(); xq = xq.double()
    return (((x - xq) ** 2).sum() / (x ** 2).sum()).item()


def qsnr_db(x, xq):
    return -10.0 * math.log10(rel_mse(x, xq))


# ---------------------------------------------------------------------------
# Hadamard transforms
# ---------------------------------------------------------------------------
def hadamard(n):
    assert n & (n - 1) == 0
```

```
    H = torch.ones(1, 1)
    while H.shape[0] < n:
        H = torch.cat([torch.cat([H, H], 1), torch.cat([H, -H], 1)], 0)
    return H / math.sqrt(n)


def rht_blocks(x, d=16, signs=None, gen=None):
    """Block-diagonal random Hadamard transform along the last dim (tile d)."""
    H = hadamard(d)
    if signs is None:
        signs = (torch.randint(0, 2, (d,), generator=gen).float() * 2 - 1)
    M = (signs[:, None] * H) # S_d H_d (rows sign-flipped)
    sh = x.shape
    y = x.reshape(-1, d) @ M.T
    return y.reshape(sh), M
```

B.2 Quantised linear layer (`qlinear.py`)

```
"""Fake-quantised linear layer implementing the three GEMMs (Fprop, Dgrad, Wgrad) of an
NVFP4/MXFP4 training recipe, in float32 on CPU. Used only for the toy-scale experiment."""
import torch, math
from fp4lib import *

class Recipe:
    def __init__(self, fmt='none', w_2d=False, w_bwd='hp', sr_grad=False, rht_wgrad=0,
                 rht_dgrad=0, four_over_six=False, rtn_grad_headroom=True):
        self.fmt = fmt # 'none' | 'nvfp4' | 'mxfp4'
        self.w_2d = w_2d # 16x16 tiles for weights (same quantised W in all passes)
        self.w_bwd = w_bwd # for 1D weights: 'hp' = re-quantise the master weight along the
            other axis
        self.sr_grad = sr_grad
        self.rht_wgrad = rht_wgrad
        self.rht_dgrad = rht_dgrad
        self.four_over_six = four_over_six
        self.gen = torch.Generator().manual_seed(0)
        self.sign_cache = {}

    def signs(self, d):
        if d not in self.sign_cache:
            self.sign_cache[d] = (torch.randint(0, 2, (d,), generator=self.gen).float() * 2 -
                1)
        return self.sign_cache[d]

    # -- elementary quantisers; x has blocks along its LAST dim
    def q_rtn(self, x):
        if self.fmt == 'nvfp4':
            return quant_nvfp4_46(x, 16) if self.four_over_six else quant_nvfp4(x, 16)
        if self.fmt == 'mxfp4':
            return quant_mxfp4(x, 32, 'ceil')
        return x

    def q_grad(self, x):
        if not self.sr_grad:
            return self.q_rtn(x)
        if self.fmt == 'nvfp4':
            return quant_nvfp4(x, 16, rounding='sr', gen=self.gen, headroom=16 / 17)
        if self.fmt == 'mxfp4':
            return quant_mxfp4(x * 0.75, 32, 'ceil', 'sr', self.gen) / 0.75
        return x

    def q_weight(self, W):
        if self.fmt == 'nvfp4' and self.w_2d:
            return quant_nvfp4_2d(W, 16)
        return self.q_rtn(W)
```

```
class QLinearFn(torch.autograd.Function):
   @staticmethod
   def forward(ctx, X, W, recipe):
      ctx.recipe = recipe
      ctx.save_for_backward(X, W)
      if recipe.fmt == 'none':
         return X @ W.T
      Xq = recipe.q_rtn(X)
      Wq = recipe.q_weight(W)
      ctx.Wq = Wq
      return Xq @ Wq.T

   @staticmethod
   def backward(ctx, dY):
      X, W = ctx.saved_tensors
      r = ctx.recipe
      if r.fmt == 'none':
         return dY @ W, dY.T @ X, None
      # ---- Dgrad: dX = dY @ W ; contraction over `out`
      dYq = r.q_grad(_maybe_rht(dY, r.rht_dgrad, r))
      if r.w_2d:
         Wb = ctx.Wq # identical quantised weight, shape (out,in)
         Wb_c = _maybe_rht(Wb.T.contiguous(), r.rht_dgrad, r) if r.rht_dgrad else Wb.T.
             contiguous()
         Wb_q = Wb_c if r.rht_dgrad == 0 else r.q_rtn(Wb_c)
      else:
         Wt = _maybe_rht(W.T.contiguous(), r.rht_dgrad, r) # (in,out): blocks along out
         Wb_q = r.q_rtn(Wt)
      dX = dYq @ Wb_q.T
      # ---- Wgrad: dW = dY^T @ X ; contraction over tokens
      Gt, Xt = dY.T.contiguous(), X.T.contiguous() # (out,N), (in,N): blocks along N
      if r.rht_wgrad:
         sv = r.signs(r.rht_wgrad)
         Gt, _ = rht_blocks(Gt, r.rht_wgrad, signs=sv)
         Xt, _ = rht_blocks(Xt, r.rht_wgrad, signs=sv)
      Gq = r.q_grad(Gt)
      Xq = r.q_rtn(Xt)
      dW = Gq @ Xq.T
      return dX, dW, None


def _maybe_rht(x, d, r):
   if not d:
      return x
   y, _ = rht_blocks(x, d, signs=r.signs(d))
   return y


class QLinear(torch.nn.Module):
   def __init__(self, din, dout, recipe):
      super().__init__()
      self.weight = torch.nn.Parameter(torch.randn(dout, din) / math.sqrt(din))
      self.recipe = recipe

   def forward(self, x):
      sh = x.shape
      y = QLinearFn.apply(x.reshape(-1, sh[-1]), self.weight, self.recipe)
      return y.reshape(*sh[:-1], -1)
```

B.3 Experiment scripts

B.3.1 Exp. 2: analytic MSE and the scale-range anomaly (`exp2_analytic_and_anomaly.py`)

```
"""Experiment 2: (a) exact quadrature of expected MSE for absmax block scaling with an
```

```
    unquantised scale vs Monte-Carlo; (b) the 'finer is not better' E4M3 scale-range anomaly.
    """
import sys, json, math
sys.path.insert(0, '/home/claude/paper1_nvfp4/code')
import numpy as np, torch
from scipy.special import erf
from fp4lib import *

def err_fp4(u):
    t = torch.tensor(u, dtype=torch.float32)
    return (t - rtn_fp4(t)).numpy().astype(np.float64)

def err_int(u, qmax=7):
    return u - np.clip(np.round(u), -qmax, qmax)

def analytic_rel_mse(G, dist, grid='fp4'):
    """E[(x - s q(x/s))^2]/E[x^2] with s = max|x|/qmax unquantised, G iid per block,
    max element exact (zero error), the other G-1 elements iid truncated to [0,m]."""
    if dist == 'gaussian':
        g = lambda z: 2 * np.exp(-z ** 2 / 2) / math.sqrt(2 * math.pi)
        F = lambda z: erf(z / math.sqrt(2))
        mmax = 7.5
    else:
        g = lambda z: math.sqrt(2) * np.exp(-math.sqrt(2) * z)
        F = lambda z: 1 - np.exp(-math.sqrt(2) * z)
        mmax = 16.0
    qmax = 6.0 if grid == 'fp4' else 7.0
    u = np.linspace(0, qmax, 4001)
    e2 = (err_fp4(u) if grid == 'fp4' else err_int(u)) ** 2
    m = np.linspace(1e-4, mmax, 6000)
    h = G * g(m) * F(m) ** (G - 1) # density of block max
    inner = np.empty_like(m)
    for i, mi in enumerate(m):
        z = mi * u / qmax
        integrand = (mi / qmax) ** 2 * e2 * g(z) / F(mi) * (mi / qmax)
        inner[i] = np.trapezoid(integrand, u)
    total = np.trapezoid(h * inner, m)
    return (G - 1) / G * total

out = {'analytic': {}, 'mc': {}}
torch.manual_seed(7)
N = 1 << 22
for dist in ['gaussian', 'laplace']:
    if dist == 'gaussian':
        x = torch.randn(N)
    else:
        u = torch.rand(N).clamp(1e-7, 1 - 1e-7) - 0.5
        x = -torch.sign(u) * torch.log1p(-2 * u.abs()) / math.sqrt(2)
    out['analytic'][dist] = {}
    out['mc'][dist] = {}
    for G in [4, 8, 16, 32, 64, 128]:
        a_fp = analytic_rel_mse(G, dist, 'fp4')
        a_int = analytic_rel_mse(G, dist, 'int4')
        m_fp = rel_mse(x, quant_fp4_ideal_scale(x, G))
        m_int = rel_mse(x, quant_int(x, 4, G))
        out['analytic'][dist][G] = {'fp4': a_fp, 'int4': a_int}
        out['mc'][dist][G] = {'fp4': m_fp, 'int4': m_int}
        print(dist, G, 'fp4 analytic %.5f mc %.5f | int4 analytic %.5f mc %.5f' % (a_fp, m_fp,
              a_int, m_int), flush=True)

# ---- anomaly: one massive outlier of size A*sigma in a Gaussian tensor ----
anom = {}
torch.manual_seed(11)
n = 1 << 20
base = torch.randn(n)
for A in [1e2, 1e3, 1e4, 1e5, 1e6]:
    x = base.clone(); x[12345] = A
    anom[A] = {}
    for B in [4, 8, 16, 32, 64]:
```

```
        # exclude the block holding the outlier from the metric
        mask = torch.ones(n, dtype=torch.bool); lo = (12345 // B) * B; mask[lo:lo + B] =
            False
        def r(q):
           return rel_mse(x[mask], q[mask])
        anom[A][B] = {
           'ideal': r(quant_fp4_ideal_scale(x, B)),
           'E4M3': r(quant_nvfp4(x, B)),
           'UE5M3': r(quant_nvfp4(x, B, scale_fmt='ue5m3')),
           'E4M3 +4/6': r(quant_nvfp4_46(x, B)) if B == 16 else None,
        }
    print('anomaly A=%g' % A, {B: round(anom[A][B]['E4M3'], 4) for B in anom[A]}, flush=True)

out['anomaly'] = {str(k): v for k, v in anom.items()}
json.dump(out, open('/home/claude/paper1_nvfp4/results/exp2_analytic_anomaly.json', 'w'),
    indent=1)
```

B.3.2 Exp. 4: stochastic rounding in GEMM (`exp4_sr_gemm.py`)

```
"""Experiment 4: stochastic rounding statistics, estimator bias, GEMM error scaling."""
import sys, json, math
sys.path.insert(0, '/home/claude/paper1_nvfp4/code')
import numpy as np, torch
from fp4lib import *

out = {}
# ---- (a) per-value SR/RTN error profile on the E2M1 grid (unit scale, no block scaling)
xs = torch.linspace(0, 6, 1201)
g = torch.Generator().manual_seed(0)
T = 4000
mean_sr = torch.zeros_like(xs); var_sr = torch.zeros_like(xs)
acc = torch.zeros_like(xs); acc2 = torch.zeros_like(xs)
for _ in range(T):
   q = sr_fp4(xs, g); acc += q; acc2 += q * q
mean_sr = acc / T; var_sr = acc2 / T - mean_sr ** 2
grid = FP4_GRID
idx = (torch.bucketize(xs, grid, right=True) - 1).clamp(0, 6)
lo, hi = grid[idx], grid[idx + 1]
var_theory = (xs - lo) * (hi - xs)
rtn_err = rtn_fp4(xs) - xs
out['sr_profile'] = {
   'max_abs_bias': float((mean_sr - xs).abs().max()),
   'max_var_dev': float((var_sr - var_theory).abs().max()),
   'avg_var_sr_uniform': float(var_theory.mean()),
   'avg_mse_rtn_uniform': float((rtn_err ** 2).mean()),
   'ratio': float(var_theory.mean() / (rtn_err ** 2).mean()),
}
np.save('/home/claude/paper1_nvfp4/results/sr_profile.npy',
      np.stack([xs.numpy(), mean_sr.numpy(), var_sr.numpy(), var_theory.numpy(), rtn_err.
          numpy()]))
print('SR profile', out['sr_profile'], flush=True)

# ---- (b) dead-zone toy: constant small gradient components beside a block outlier
torch.manual_seed(3)
B = 16
mu = torch.full((B,), 0.0); mu[0] = 6.0 # one big element sets the scale (amax -> 6)
mu[1:] = torch.tensor([0.05, 0.1, 0.15, 0.2, 0.05, 0.1, 0.15, 0.2, 0.05, 0.1, 0.15, 0.2,
    0.05, 0.1, 0.15])
steps = 20000
for noise in [0.0, 0.05, 0.3]:
   acc_rtn = torch.zeros(B); acc_sr = torch.zeros(B)
   gen = torch.Generator().manual_seed(1)
   for t in range(steps):
      gt = mu + noise * torch.randn(B, generator=gen)
      gt[0] = mu[0] # keep the outlier fixed so the scale is 1.0
      acc_rtn += rtn_fp4(gt); acc_sr += sr_fp4(gt, gen)
   out.setdefault('deadzone', {})[str(noise)] = {
```

```
        'true_mean_small': float(mu[1:].mean()),
        'rtn_mean_small': float((acc_rtn / steps)[1:].mean()),
        'sr_mean_small': float((acc_sr / steps)[1:].mean()),
    }
    print('deadzone', noise, out['deadzone'][str(noise)], flush=True)

# ---- (c) GEMM error vs inner dimension K (random Gaussian operands)
torch.manual_seed(9)
M = N = 128
rows = []
for K in [64, 256, 1024, 4096, 16384]:
    A = torch.randn(M, K); Bm = torch.randn(K, N)
    C = A @ Bm
    rec = {'K': K}
    for name, qa, qb in [
        ('NVFP4 RTN', lambda a: quant_nvfp4(a, 16), lambda b: quant_nvfp4(b.T.contiguous(), 16).T),
        ('NVFP4 SR', lambda a: quant_nvfp4(a, 16, rounding='sr'), lambda b: quant_nvfp4(b.T.contiguous(), 16, rounding='sr').T),
        ('MXFP4 RTN', lambda a: quant_mxfp4(a, 32, 'ceil'), lambda b: quant_mxfp4(b.T.contiguous(), 32, 'ceil').T),
        ('INT4 B32', lambda a: quant_int(a, 4, 32), lambda b: quant_int(b.T.contiguous(), 4, 32).T)]:
        Cq = qa(A) @ qb(Bm)
        rec[name] = float(((Cq - C).norm() / C.norm()))
        eA = rel_mse(A, qa(A)); eB = rel_mse(Bm, qb(Bm))
        rec[name + ' pred'] = math.sqrt(eA + eB) # first-order prediction sqrt(eps_a+eps_b)
    rows.append(rec)
    print('GEMM', rec, flush=True)
out['gemm_vs_K'] = rows

# ---- (d) averaging many independent SR draws vs RTN for a fixed product (bias test)
torch.manual_seed(10)
K = 1024
A = torch.randn(64, K); Bm = torch.randn(K, 64)
# make the product have a deterministic structure: B = A^T-like to give signal
Bm = A.T[:, :64].clone() * 0.5 + 0.2 * torch.randn(K, 64)
C = A @ Bm
Crtn = quant_nvfp4(A, 16) @ quant_nvfp4(Bm.T.contiguous(), 16).T
res_avg = {'rtn_err': float((Crtn - C).norm() / C.norm()), 'sr': {}, 'sr_headroom': {}}
accum = torch.zeros_like(C); accum_h = torch.zeros_like(C)
gen = torch.Generator().manual_seed(2)
n_list = [1, 4, 16, 64, 256, 1024]
for n in range(1, 1025):
    Aq = quant_nvfp4(A, 16, rounding='sr', gen=gen)
    Bq = quant_nvfp4(Bm.T.contiguous(), 16, rounding='sr', gen=gen).T
    accum += Aq @ Bq
    Ah = quant_nvfp4(A, 16, rounding='sr', gen=gen, headroom=16 / 17)
    Bh = quant_nvfp4(Bm.T.contiguous(), 16, rounding='sr', gen=gen, headroom=16 / 17).T
    accum_h += Ah @ Bh
    if n in n_list:
        res_avg['sr'][n] = float(((accum / n) - C).norm() / C.norm())
        res_avg['sr_headroom'][n] = float(((accum_h / n) - C).norm() / C.norm())
print('SR averaging', res_avg, flush=True)
out['sr_averaging'] = res_avg

# ---- (e) accumulation precision: sequential FP16 accumulate vs FP32 accumulate
torch.manual_seed(12)
accum_rows = []
for K in [256, 1024, 4096, 16384, 65536]:
    a = torch.randn(256, K) * 0.5; b = torch.randn(256, K) * 0.5 + 0.1 # mean>0 -> growing partial sums
    p = (a * b).double()
    exact = p.sum(1)
    s32 = torch.zeros(256, dtype=torch.float32); s16 = torch.zeros(256, dtype=torch.float16)
    sbf = torch.zeros(256, dtype=torch.bfloat16)
    pf = (a * b)
    for k in range(K):
        s32 += pf[:, k]
```

```
        s16 = (s16.float() + pf[:, k].half().float()).half()
        sbf = (sbf.float() + pf[:, k].bfloat16().float()).bfloat16()
    rel = lambda s: float(((s.double() - exact).abs() / exact.abs()).median())
    accum_rows.append({'K': K, 'fp32': rel(s32), 'fp16': rel(s16), 'bf16': rel(sbf)})
    print('accum', accum_rows[-1], flush=True)
out['accumulation'] = accum_rows
json.dump(out, open('/home/claude/paper1_nvfp4/results/exp4_sr_gemm.json', 'w'), indent=1)
```

B.3.3 Exp. 5b: SR dot-product variance with and without RHT (`exp5b_sr_variance_rht.py`)

```
"""Experiment 5b: std of an SR-quantised dot product <x, y> with and without RHT,
for vectors that contain sparse block-level outliers (cf. Tseng et al. 2025)."""
import sys, json, math
sys.path.insert(0, '/home/claude/paper1_nvfp4/code')
import torch
from fp4lib import *
torch.manual_seed(31)
n = 4096; nrep = 400
res = {}
for dens, mag in [(0.0, 1), (1 / 256, 20), (1 / 64, 20), (1 / 16, 20), (1 / 256, 100)]:
   x = torch.randn(n)
   mask = (torch.rand(n) < dens)
   x = torch.where(mask, x * mag, x)
   y = x + 0.3 * torch.randn(n) # correlated so that <x,y> ~ ||x||^2
   true = float(x @ y)
   for fmt, kw in [('NVFP4 SR', dict(B=16)), ('MXFP4 SR', dict(B=32))]:
      for d in [0, 16, 32, 64, 128]:
         gen = torch.Generator().manual_seed(0)
         ests = []
         for r in range(nrep):
            xv, yv = x[None, :], y[None, :]
            if d:
               sv = (torch.randint(0, 2, (d,), generator=gen).float() * 2 - 1)
               xv, _ = rht_blocks(xv, d, signs=sv); yv, _ = rht_blocks(yv, d, signs=sv)
            if fmt.startswith('NV'):
               xq = quant_nvfp4(xv, 16, rounding='sr', gen=gen, headroom=16 / 17)
               yq = quant_nvfp4(yv, 16, rounding='sr', gen=gen, headroom=16 / 17)
            else:
               # unbiased MXFP4 SR needs 3/4 headroom (Tseng et al.) so nothing clips
               xq = quant_mxfp4(xv * 0.75, 32, 'ceil', 'sr', gen) / 0.75
               yq = quant_mxfp4(yv * 0.75, 32, 'ceil', 'sr', gen) / 0.75
            ests.append(float((xq * yq).sum()))
         e = torch.tensor(ests, dtype=torch.float64)
         res[f'dens={dens:.4f},mag={mag},{fmt},rht={d}'] = {
            'rel_bias': float((e.mean() - true) / true),
            'rel_std': float(e.std() / abs(true)),
         }
   print('done', dens, mag, flush=True)
json.dump(res, open('/home/claude/paper1_nvfp4/results/exp5b_sr_variance.json', 'w'),
    indent=1)
```

B.3.4 Exp. 6: real weights and perplexity (`exp6_real_weights.py`)

```
"""Experiment 6: format error on *real* LLM weights and WikiText-2 perplexity under
    simulated
(fake) quantisation. Usage: python exp6_real_weights.py MODEL_DIR [--ppl]"""
import sys, json, math, time, os
sys.path.insert(0, '/home/claude/paper1_nvfp4/code')
import torch, pandas as pd
from safetensors.torch import load_file
from fp4lib import *
torch.set_num_threads(2)
```

```

model_dir = sys.argv[1]
name = os.path.basename(model_dir.rstrip('/'))
do_ppl = '--ppl' in sys.argv

def rot_wrap(f, d):
    def g(w):
        wr, M = rht_blocks(w, d, signs=torch.ones(d)) # deterministic Hadamard (sign vector all +1)
        q = f(wr)
        return (q.reshape(-1, d) @ M).reshape(w.shape) # M orthogonal: inverse = M^T^T
    return g

WFORMATS = {
    'NVFP4 B16': lambda w: quant_nvfp4(w, 16),
    'NVFP4 B16 + 4/6': lambda w: quant_nvfp4_46(w, 16),
    'NVFP4 16x16 (2D)': lambda w: quant_nvfp4_2d(w, 16),
    'MXFP4 B32 (ceil)': lambda w: quant_mxfp4(w, 32, 'ceil'),
    'MXFP4 B32 (floor/OCP)':lambda w: quant_mxfp4(w, 32, 'floor'),
    'INT4 B32 (FP32 scale)':lambda w: quant_int(w, 4, 32),
    'NVINT4 B16': lambda w: quant_int(w, 4, 16, scale='e4m3'),
    'NVFP4 B16 + RHT16': rot_wrap(lambda w: quant_nvfp4(w, 16), 16),
    'MXFP4 B32 + RHT32': rot_wrap(lambda w: quant_mxfp4(w, 32, 'ceil'), 32),
    'INT4 B32 + RHT32': rot_wrap(lambda w: quant_int(w, 4, 32), 32),
    'NVINT4 B16 + RHT16': rot_wrap(lambda w: quant_int(w, 4, 16, scale='e4m3'), 16),
}

def kurt(x):
    x = x.reshape(-1).double(); return (((x - x.mean()) ** 4).mean() / x.var() ** 2 - 3).item()

sd = load_file(os.path.join(model_dir, 'model.safetensors'))
targets = [k for k, v in sd.items() if v.ndim == 2 and 'layers.' in k and k.endswith('.weight')
           and 'norm' not in k]
print(name, len(targets), 'linear weights', flush=True)
agg = {f: {} for f in WFORMATS}
kurts = {}
t0 = time.time()
for k in targets:
    w = sd[k].float()
    mod = k.split('.')[-2]
    kurts.setdefault(mod, []).append(kurt(w))
    for f, fn in WFORMATS.items():
        agg[f].setdefault(mod, []).append(rel_mse(w, fn(w)))
print('weights done', time.time() - t0, flush=True)
summary = {'kurtosis': {m: sum(v) / len(v) for m, v in kurts.items()}, 'rel_mse': {}}
for f in WFORMATS:
    summary['rel_mse'][f] = {m: sum(v) / len(v) for m, v in agg[f].items()}
    allv = [x for v in agg[f].values() for x in v]
    summary['rel_mse'][f]['ALL'] = sum(allv) / len(allv)
json.dump(summary, open(f'/home/claude/paper1_nvfp4/results/exp6_weights_{name}.json', 'w'), indent=1)
for f in WFORMATS:
    print('%-26s mean relMSE %.5f' % (f, summary['rel_mse'][f]['ALL']))

if do_ppl:
    from transformers import AutoModelForCausalLM, AutoTokenizer
    tok = AutoTokenizer.from_pretrained(model_dir)
    text = "\n\n".join(pd.read_parquet('/home/claude/paper1_nvfp4/data/wt2_test.parquet').text.tolist())
    ids = tok(text, return_tensors='pt').input_ids[0]
    L, NWIN = 512, 64
    ids = ids[: L * NWIN]
    windows = ids.reshape(NWIN, L)

    def load():
        m = AutoModelForCausalLM.from_pretrained(model_dir, torch_dtype=torch.float32)
        m.eval(); return m
```

```

    def linears(m):
        out = []
        for n, mod in m.named_modules():
            if isinstance(mod, torch.nn.Linear) and 'layers.' in n:
                out.append((n, mod))
        return out

    @torch.no_grad()
    def ppl(m):
        tot, cnt = 0.0, 0
        for i in range(0, NWIN, 4):
            b = windows[i:i + 4]
            logits = m(b).logits
            loss = torch.nn.functional.cross_entropy(logits[:, :-1].reshape(-1, logits.shape
                [-1]), b[:, 1:].reshape(-1), reduction='sum')
            tot += loss.item(); cnt += b[:, 1:].numel()
        return math.exp(tot / cnt)

    AFORMATS = {
        'NVFP4': lambda a: quant_nvfp4(a, 16),
        'NVFP4 + 4/6': lambda a: quant_nvfp4_46(a, 16),
        'MXFP4': lambda a: quant_mxfp4(a, 32, 'ceil'),
    }
    configs = [('BF16-equivalent (FP32)', None, None)]
    for wn in ['NVFP4 B16', 'NVFP4 B16 + 4/6', 'MXFP4 B32 (ceil)', 'INT4 B32 (FP32 scale)', '
        NVINT4 B16']:
        configs.append((f'W4A16 {wn}', wn, None))
    for wn, an in [('NVFP4 B16', 'NVFP4'), ('NVFP4 B16 + 4/6', 'NVFP4 + 4/6'), ('MXFP4 B32 (
        ceil)', 'MXFP4')]:
        configs.append((f'W4A4 {wn}', wn, an))
    results = {}
    for cname, wn, an in configs:
        t1 = time.time()
        m = load()
        hooks = []
        if wn is not None:
            for n, mod in linears(m):
                mod.weight.data = WFORMATS[wn](mod.weight.data.float())
        if an is not None:
            af = AFORMATS[an]
            def mk(af):
                def hook(mod, args):
                    x = args[0]; sh = x.shape
                    return (af(x.reshape(-1, sh[-1])).reshape(sh),)
                return hook
            for n, mod in linears(m):
                hooks.append(mod.register_forward_pre_hook(mk(af)))
        p = ppl(m)
        results[cname] = p
        print('%-40s ppl %.3f (%.0fs)' % (cname, p, time.time() - t1), flush=True)
        json.dump({'tokens_evaluated': int(L * NWIN - NWIN), 'ppl': results},
                  open(f'/home/claude/paper1_nvfp4/results/exp6_ppl_{name}.json', 'w'), indent
                      =1)
        del m
```

B.3.5 Exp. 7: toy training (`exp7_toy_train.py`)

```
"""Experiment 7: toy-scale (byte-level GPT, ~0.8M linear params) training with simulated
NVFP4/MXFP4 recipes. Usage: python exp7_toy_train.py RECIPE SEED STEPS"""
import sys, json, math, time, os
sys.path.insert(0, '/home/claude/paper1_nvfp4/code')
import numpy as np, torch, pandas as pd
import torch.nn as nn, torch.nn.functional as F
from fp4lib import *
from qlinear import QLinear, Recipe
torch.set_num_threads(1)
```

```

RECIPES = {
 'fp32': dict(),
 'naive': dict(fmt='nvfp4'),
 'sr': dict(fmt='nvfp4', sr_grad=True),
 'rht': dict(fmt='nvfp4', rht_wgrad=16),
 '2d': dict(fmt='nvfp4', w_2d=True),
 'nvidia': dict(fmt='nvfp4', w_2d=True, sr_grad=True, rht_wgrad=16),
 'nvidia_lastbf16': dict(fmt='nvfp4', w_2d=True, sr_grad=True, rht_wgrad=16), # last block kept FP32 (see below)
 'nvidia_46': dict(fmt='nvfp4', w_2d=True, sr_grad=True, rht_wgrad=16, four_over_six=True),
 'oned_requant': dict(fmt='nvfp4', w_2d=False, sr_grad=True, rht_wgrad=16, rht_dgrad=16),
 'oned_46': dict(fmt='nvfp4', w_2d=False, sr_grad=True, rht_wgrad=16, rht_dgrad=16, four_over_six=True),
 'mxfp4': dict(fmt='mxfp4', w_2d=False, sr_grad=True, rht_wgrad=32),
 'mxfp4_naive': dict(fmt='mxfp4'),
}
rname, seed, steps = sys.argv[1], int(sys.argv[2]), int(sys.argv[3])
D, H, NL, T, BS, V = 128, 4, 4, 128, 16, 256

class Block(nn.Module):
   def __init__(self, rec_attn, rec_mlp):
      super().__init__()
      self.n1 = nn.LayerNorm(D); self.n2 = nn.LayerNorm(D)
      self.qkv = QLinear(D, 3 * D, rec_attn); self.o = QLinear(D, D, rec_attn)
      self.fc1 = QLinear(D, 4 * D, rec_mlp); self.fc2 = QLinear(4 * D, D, rec_mlp)
   def forward(self, x):
      B, T_, _ = x.shape
      q, k, v = self.qkv(self.n1(x)).chunk(3, -1)
      q, k, v = [t.reshape(B, T_, H, D // H).transpose(1, 2) for t in (q, k, v)]
      a = F.scaled_dot_product_attention(q, k, v, is_causal=True).transpose(1, 2).reshape(B, T_, D)
      x = x + self.o(a)
      return x + self.fc2(F.gelu(self.fc1(self.n2(x))))

class GPT(nn.Module):
   def __init__(self, recipe_kw, last_hp):
      super().__init__()
      self.emb = nn.Embedding(V, D); self.pos = nn.Embedding(T, D)
      rec = Recipe(**recipe_kw); hp = Recipe()
      self.blocks = nn.ModuleList([Block(rec, rec) if not (last_hp and i == NL - 1) else Block(hp, hp) for i in range(NL)])
      self.nf = nn.LayerNorm(D); self.head = nn.Linear(D, V, bias=False)
   def forward(self, idx):
      x = self.emb(idx) + self.pos(torch.arange(idx.shape[1]))
      for b in self.blocks: x = b(x)
      return self.head(self.nf(x))

# ---- data: byte-level WikiText-2 (raw)
tr = "".join(pd.read_parquet('/home/claude/paper1_nvfp4/data/wt2_train.parquet').text.tolist()).encode('utf-8', 'ignore')
va = "".join(pd.read_parquet('/home/claude/paper1_nvfp4/data/wt2_test.parquet').text.tolist()).encode('utf-8', 'ignore')
tr = torch.tensor(np.frombuffer(tr, dtype=np.uint8).astype(np.int64)); va = torch.tensor(np.frombuffer(va, dtype=np.uint8).astype(np.int64))

torch.manual_seed(seed)
model = GPT(RECIPES[rname], last_hp=(rname == 'nvidia_lastbf16'))
# same initial weights for every recipe at a given seed
opt = torch.optim.AdamW(model.parameters(), lr=3e-3, betas=(0.9, 0.95), weight_decay=0.05)
gdata = torch.Generator().manual_seed(1000 + seed)
def batch(src):
   ix = torch.randint(0, len(src) - T - 1, (BS,), generator=gdata)
   x = torch.stack([src[i:i + T] for i in ix]); y = torch.stack([src[i + 1:i + T + 1] for i in ix])
   return x, y
def lr_at(s): # WSD: warmup 5%, stable to 70%, linear decay to 5% of peak
   w = int(0.05 * steps); d0 = int(0.7 * steps)
   if s < w: return 3e-3 * (s + 1) / w
```

```
    if s < d0: return 3e-3
    return 3e-3 * (1 - 0.95 * (s - d0) / (steps - d0))

@torch.no_grad()
def evaluate():
    model.eval(); g = torch.Generator().manual_seed(99); tot = 0.0; n = 0
    for _ in range(20):
        ix = torch.randint(0, len(va) - T - 1, (32,), generator=g)
        x = torch.stack([va[i:i + T] for i in ix]); y = torch.stack([va[i + 1:i + T + 1] for
            i in ix])
        tot += F.cross_entropy(model(x).reshape(-1, V), y.reshape(-1)).item(); n += 1
    model.train(); return tot / n

log = {'recipe': rname, 'seed': seed, 'steps': steps, 'train': [], 'val': []}
t0 = time.time(); ema = None
for s in range(steps):
    for gp in opt.param_groups: gp['lr'] = lr_at(s)
    x, y = batch(tr)
    loss = F.cross_entropy(model(x).reshape(-1, V), y.reshape(-1))
    opt.zero_grad(set_to_none=True); loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0); opt.step()
    ema = loss.item() if ema is None else 0.98 * ema + 0.02 * loss.item()
    if (s + 1) % 50 == 0: log['train'].append((s + 1, ema))
    if (s + 1) % 250 == 0 or s + 1 == steps:
        log['val'].append((s + 1, evaluate()))
        print(rname, seed, s + 1, 'train_ema %.4f val %.4f %.0fs' % (ema, log['val'][-1][1],
            time.time() - t0), flush=True)
log['final_val'] = log['val'][-1][1]; log['time'] = time.time() - t0
json.dump(log, open(f'/home/claude/paper1_nvfp4/results/toy_{rname}_s{seed}.json', 'w'))
```

B.3.6 Exp. 12: low-norm channels in 2D tiles (`exp12_lowchannel.py`)

```
"""Experiment 12: low-norm channels inside 16x16 weight tiles (mechanism reported for zero
    weight-gradients in Nemotron 3 Super).
Weight W (R x C) = diag(c) Z, rows (output channels) have norm ratio r for a fraction f of
    rows, random placement.
Compare 1D (1x16 along input dim) vs 2D (16x16) NVFP4 RTN: zero fraction and relative error
     restricted to low-norm rows."""
import sys, json, torch
sys.path.insert(0, '/home/claude/paper1_nvfp4/code')
from fp4lib import *
torch.manual_seed(0)
R = C = 1024
out = {}
for f in [0.1, 0.5]:
    for r in [1.0, 0.3, 0.1, 0.03, 0.01, 0.001]:
        low = torch.rand(R) < f
        c = torch.where(low, torch.full((R,), r), torch.ones(R))
        W = torch.randn(R, C) * c[:, None]
        q1 = quant_nvfp4(W.reshape(-1, 16), 16).reshape(R, C) if False else quant_nvfp4(W, 16)

        q2 = quant_nvfp4_2d(W, 16)
        res = {}
        for nm, q in [('1D', q1), ('2D', q2)]:
            Wl, ql = W[low], q[low]
            res[nm] = dict(zero_low=float((ql == 0).float().mean()), relmse_low=float(((ql -
                Wl) ** 2).sum() / (Wl ** 2).sum()),
                           relmse_all=float(((q - W) ** 2).sum() / (W ** 2).sum()),
                           zero_high=float((q[~low] == 0).float().mean()))
        out[f'f{f}_r{r}'] = res
        print(f, r, {k: {a: round(b, 4) for a, b in v.items()} for k, v in res.items()},
            flush=True)
json.dump(out, open('/home/claude/paper1_nvfp4/results/exp12_lowchannel.json', 'w'), indent
    =1)
```

B.3.7 Exp. 14: SGD stationary loss and Monte Carlo errors (`exp14_ou.py`)

```
"""Experiment 14: check the stationary SGD excess loss eta*tr(C)/4 on a noisy quadratic,
    and the MC standard errors of Exp. 1."""
import sys, json, torch, math
sys.path.insert(0,'/home/claude/paper1_nvfp4/code')
from fp4lib import *
torch.manual_seed(0)
d=64
h=torch.logspace(-1,1,d) # Hessian eigenvalues 0.1..10
sig=1.0 # gradient-noise std per coordinate
out={'sgd':[]}
for eta in [0.2,0.1,0.05,0.025,0.0125]:
   th=torch.zeros(d); T=200000; burn=20000; acc=0.0; n=0
   # vectorise across 256 independent chains
   th=torch.zeros(256,d)
   steps=int(40/ (eta*0.1)) # relax time 1/(eta*h_min)=10/eta -> run 4 relaxation times
   steps=min(steps,40000)
   for t in range(steps):
      g=th*h+sig*torch.randn(256,d)
      th=th-eta*g
      if t>steps//2:
         acc+=float(0.5*(h*th**2).sum(1).mean()); n+=1
   emp=acc/n; pred=eta*d*sig**2/4
   out['sgd'].append(dict(eta=eta,empirical=emp,predicted=pred,ratio=emp/pred))
   print(eta,emp,pred,emp/pred,flush=True)
# MC standard error of relative MSE (NVFP4 G=16 Gaussian etc): 32 chunks of 2^16 blocks-
    aligned samples
res={}
for nm,gen in [('gaussian',lambda n: torch.randn(n)),('laplace',lambda n: torch.
    distributions.Laplace(0.,1.).sample((n,)))]:
   for fn,f in [('NVFP4 G16',lambda x: quant_nvfp4(x,16)),('MXFP4 floor G32',lambda x:
       quant_mxfp4(x,32,'floor')),('NVFP4+4/6',lambda x: quant_nvfp4_46(x,16))]:
      vals=[]
      for i in range(32):
         x=gen(1<<16); vals.append(rel_mse(x,f(x)))
      v=torch.tensor(vals); res[f'{nm} {fn}']=dict(mean=float(v.mean()),se=float(v.std()/
          math.sqrt(32)))
      print(nm,fn,res[f'{nm} {fn}'],flush=True)
out['mc_se']=res
json.dump(out,open('/home/claude/paper1_nvfp4/results/exp14_ou.json','w'),indent=1)
```

B.4 Other files

File	Purpose
`exp1_synthetic_mse.py`	relative MSE of all formats on six synthetic distributions; block-size sweep
`exp3_hadamard.py`	effect of the Hadamard transform on RTN error versus rotation size
`exp5_wgrad_rht.py`	one-step weight-gradient error with and without RHT
`exp8_decomp.py`	error decomposition by magnitude bin; E4M3 penalty-model check
`exp9_spike_block.py`	one-spike blocks with and without rotation
`exp10_gain.py`	gain and energy ratio of quantisers
`exp11_bench_noise.py`	binomial noise audit of the 12B downstream table
`exp13_lowbits.py`	block INT-b error for $b = 2, \ldots, 8$
`exp6b_w4a4.py`	W4A4 and rotation perplexity variants for the small model
`run_exp7.sh`	job queue for toy training
`make_figs1.py`, `maketables.py`, `gen_tables2.py`	figures and LaTeX tables from the JSON results

Appendix C

Supplementary Result Tables

This appendix holds result tables that support the main text but would interrupt it: the full block-size sweeps for the distributions not shown in Chapter 4, the signal-to-noise form of the main format comparison, the E4M3 variant of the outlier experiment, and a catalogue of all experiments with their sample sizes. All entries are own experiments ([E]) unless stated.

C.1 Catalogue of experiments

Table C.1: All numerical experiments of this report. "Seeds" counts independent random draws of the data or model initialisation; most synthetic experiments use one very large sample instead (Appendix A.5). Runtime on two CPU cores.

No.	Question	Design	Sample / seeds	Reported in
1	Format MSE on six distributions; block-size sweeps	15 quantisers, $G = 4$–128, RTN/SR	2^{21} values	Chapter 4
2	Exact quadrature vs MC; scale-range anomaly	Prop. 4.2; one outlier of 10^2–$10^6\sigma$	2^{22} (quadrature check), 2^{20} (anomaly)	Chapter 4
3	Does a Hadamard transform lower RTN error?	rotation size $d = 0$–256, 4 formats, 6 distributions	2048×1024 values	Chapter 6
4	SR bias and variance in GEMM	K sweep, averaging over draws	128×128 matrices; up to 1024 SR draws	Chapter 5
5, 5b	Wgrad error and SR variance with/without RHT	$T = 4096$ tokens (5); $n = 4096$, five outlier densities (5b)	40 (5) and 400 (5b) SR draws	Chapter 6
6	Format error on real weights; perplexity	SmolLM2-135M, Qwen2.5-0.5B; 64 windows of 512 tokens	1 model each	Chapter 13
7	Toy training under recipes	byte-level GPT, 500 steps	1–2 seeds	Chapter 13
8	Error by magnitude bin; E4M3 model check	Gaussian, $G = 16$	2^{20}	Chapter 4
9	One-spike blocks with RHT	$G = 16$, NVFP4/NVINT4	2×10^5 blocks	Chapter 6
10	Gain and energy ratio	6 quantisers	2^{21}	Chapter 7
11	Binomial noise audit	Table of the 12B report	derived	Chapter 12
12	Low-norm channels in 2D tiles	1024×1024, r and f sweep	1 draw	Chapter 15
13	INT-b error for $b = 2$–8	$G = 16$, E4M3 scale	2^{21}	Chapter 18
14	SGD stationary loss; MC standard errors	noisy quadratic; 32 chunks	256 chains	Appendix A

C.2 Signal-to-noise form of the format comparison

Format	Gauss.	Laplace	t_5	t_3	Row-scaled	Outl. chan.
FP4, G=16, ideal FP32 scale	20.53	20.80	20.82	21.01	20.51	22.19
FP4, G=32, ideal FP32 scale	19.91	19.78	19.83	19.46	19.90	19.41
INT4, G=16, FP32 scale	21.39	19.90	19.94	18.90	21.38	20.49
INT4, G=32, FP32 scale	20.26	18.33	18.24	16.76	20.27	17.51
NVINT4 (G=16, E4M3 scale)	21.28	19.78	19.82	18.79	21.26	20.19
MXINT4 (G=32, E8M0 up)	16.83	14.98	14.82	13.63	16.91	15.94
MXFP4 floor (OCP rule)	18.80	17.99	17.99	16.95	18.79	15.55
MXFP4 ceil (round scale up)	18.77	17.56	17.57	16.42	18.64	15.93
MXFP4 nearest-log scale	18.32	17.59	17.58	16.52	18.42	14.87
NVFP4 (G=16, E4M3)	20.44	20.65	20.68	20.81	20.42	21.75
NVFP4 G=32 (E4M3)	19.86	19.71	19.75	19.35	19.84	19.19
NVFP4 + Four Over Six (MSE)	21.22	21.19	21.24	21.17	21.21	21.98
NVFP4 with SR	17.23	17.38	17.40	17.44	17.26	17.56
NVFP4 with SR, 16/17 headroom	16.28	16.05	16.12	15.74	16.29	14.99

Table C.2: Quantisation SNR ($-10\log_{10}$ of relative MSE, in dB) for the formats of Table 4.5. Higher is better. Each $+3$ dB halves the MSE.

C.3 Block-size sweeps for the remaining distributions

G	FP4 ideal	INT4 ideal	NVFP4 (E4M3)	MXFP4 ceil	MXFP4 floor
4	0.00390	0.00354	0.00439	0.01194	0.01589
8	0.00611	0.00637	0.00649	0.01313	0.01546
16	0.00827	0.01007	0.00855	0.01487	0.01542
32	0.01038	0.01492	0.01058	0.01742	0.01582
64	0.01261	0.02117	0.01275	0.02120	0.01697
128	0.01516	0.02935	0.01525	0.02648	0.01926

Table C.3: Student-t with 5 degrees of freedom (unit variance): relative MSE versus block size, as in Table 4.7.

C.4 Real-weight results

G	FP4 ideal	INT4 ideal	NVFP4 (E4M3)	MXFP4 ceil	MXFP4 floor
4	0.00472	0.00316	0.00515	0.01149	0.01515
8	0.00699	0.00516	0.00729	0.01209	0.01427
16	0.00883	0.00726	0.00905	0.01282	0.01364
32	0.01016	0.00941	0.01030	0.01365	0.01331
64	0.01112	0.01164	0.01122	0.01462	0.01323
128	0.01188	0.01372	0.01191	0.01558	0.01334

Table C.4: Row-scaled Gaussian (rows of length 1024 with log-normal row scale): relative MSE versus block size.

A/σ	G=4	G=8	G=16	G=32	G=64
10^2	0.0051	0.0073	0.0090	0.0103	0.0112
10^3	0.0051	0.0073	0.0090	0.0103	0.0112
10^4	0.0051	0.0073	0.0090	0.0103	0.0112
10^5	0.0086	0.0090	0.0098	0.0107	0.0114
10^6	0.7440	0.6630	0.5280	0.3355	0.1417

Table C.5: Rest-of-tensor relative MSE with one outlier, by block size, for E4M3 block scales (the data of Figure 4.3 in tabular form).

Format	down_proj	gate_proj	up_proj	k_proj	o_proj	q_proj	v_proj
excess kurtosis	1.24	2.12	1.15	2.06	2.44	2.38	0.86
NVFP4 B16	0.00895	0.00899	0.00899	0.00893	0.00900	0.00894	0.00889
NVFP4 B16 + 4/6	0.00755	0.00755	0.00755	0.00756	0.00756	0.00756	0.00754
NVFP4 16x16 (2D)	0.01360	0.01405	0.01315	0.01394	0.01392	0.01405	0.01335
MXFP4 B32 (ceil)	0.01419	0.01416	0.01399	0.01437	0.01401	0.01444	0.01462

Table C.6: SmolLM2-135M: relative MSE by projection type for four formats, and the excess kurtosis of the weights of each type (layer averages).

Appendix D

Glossary

Terms are defined as used in this report. Where a term has several meanings in the literature the one used here is given.

Absmax scaling
Choosing a block scale so that the largest magnitude in the block maps to the largest representable element value (here 6 for E2M1).

Accumulator
The register in which partial sums of a dot product are held; its width determines the rounding error of long contractions (Chapter 7).

Amdahl's law
The overall speed-up from accelerating a fraction ϕ of the run time by a factor s is $1/[(1-\phi)+\phi/s]$.

AutoQuantize
NVIDIA Model Optimizer search that assigns a precision (NVFP4, FP8, BF16) per operator under an average-bits budget (Chapter 10).

Binade
An interval $[2^k, 2^{k+1})$ of floating-point magnitudes; the dynamic range of a format is counted in binades.

Block (micro-block)
The group of G consecutive elements along the contraction dimension that share one scale: 16 for NVFP4, 32 for MXFP4.

Block scale
The scale factor shared by a block. NVFP4: E4M3. MXFP4: E8M0.

Chain-rule violation
The mismatch between the quantised weight used in the forward pass and the (differently blocked) quantised weight used in the backward pass; the motivation for 2D weight scaling.

Crest factor
Ratio of the maximum magnitude in a block to its root-mean-square value.

Dgrad, Fprop, Wgrad
The three GEMMs of a linear layer: gradient with respect to the input, forward product, and gradient with respect to the weights.

E2M1
4-bit floating-point element: 1 sign, 2 exponent, 1 mantissa bit; magnitudes $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$.

E4M3 / E5M3 / E8M0
8-bit scale formats with the given exponent and mantissa widths. E4M3 has maximum finite value 448; E8M0 holds only powers of two; UE5M3 is an unsigned E5M3.

EDEN, MS-EDEN
Unbiased quantisation estimators based on random rotations with a correction, used by Quartet II as an alternative to stochastic rounding.

effN, effD
Parameter and data efficiency in the Quartet scaling law: the fraction of parameters or data that is "effectively" present in a low-precision run.

Fake quantisation
Simulating a low-precision format by quantising and immediately de-quantising a tensor, then computing in higher precision. All experiments in this report use it.

Four Over Six (4/6)
Per-block choice between scaling the block maximum to 6 or to 4, whichever gives the smaller error (Section 4.9).

GEMM
General matrix–matrix multiplication.

Hadamard transform (HT, RHT)
Orthogonal transform built from ± 1 entries; RHT is the randomised variant with a random sign vector (Chapter 6).

HiF4, HiF8
Huawei HiFloat formats with hierarchical scaling (Chapter 14).

Healing
Finishing the last part of a low-precision training run in higher precision to recover the loss gap.

KL divergence
$D_{\mathrm{KL}}(p\|q) = \sum p \log(p/q)$; the loss of quantisation-aware distillation.

MR-GPTQ
GPTQ with block-wise rotations tailored to MX and NV formats (Chapter 10).

MXFP4, MXFP8
OCP microscaling formats with E8M0 scales over 32 elements; E2M1 or FP8 elements.

NVFP4
NVIDIA's 4-bit format: E2M1 elements, one E4M3 scale per 16 elements, plus an FP32 per-tensor scale; 4.5 bits per element.

NVINT4
Our name (after Chen et al. [11]) for INT4 elements with an E4M3 scale per 16 elements.

OCP
Open Compute Project, which published the MX specification.

PTQ, QAT, QAD
Post-training quantisation; quantisation-aware training (task loss with a quantised forward pass); quantisation-aware distillation (KL loss to a full-precision teacher).

Quartet, Quartet II
MXFP4 and NVFP4 training recipes of Castro et al. and Panferov et al.

QSNR
Quantisation signal-to-noise ratio, $-10\log_{10}$ of relative MSE, in dB.

Relative MSE
$\mathbb{E}\|\hat{x} - x\|^2 / \mathbb{E}\|x\|^2$.

RTN
Round to nearest (ties to even).

SR Stochastic rounding: round up with probability equal to the fractional position between neighbouring grid points, so that the expectation is exact.

SSM cache
The recurrent state of a state-space (Mamba) layer kept between decode steps.

STE
Straight-through estimator: passing gradients through a quantiser as if it were the identity.

Tensor scale (per-tensor scale)
The FP32 factor that maps the tensor maximum to $6 \cdot 448$ so that E4M3 block scales are in range.

TN layout
Matrix layout (transpose of first operand, no transpose of second) supported by the NVFP4 GEMM.

Transformer Engine (TE)
NVIDIA library implementing low-precision training layers.

Wgrad
See Dgrad.

WSD
Warmup–stable–decay learning-rate schedule.

2D scaling
Using one scale per 16×16 tile (for weights) so that row-wise and column-wise quantisations coincide.

Evidence tags

[S] Source: reported in a retrieved document; numbers and statements are the authors'.

[D] Derived: our own derivation or arithmetic on stated inputs.

[E] Own experiment: CPU simulation run for this report.

[V] Vendor claim: from a manufacturer's marketing or technical blog, not independently verified.

[U] Uncertain: a secondary source, a recollection, an inference or an extrapolation.

Bibliography

[1] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433*, 2022. URL https://arxiv.org/abs/2209.05433.

[2] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023. URL https://arxiv.org/abs/2310.10537.

[3] NVIDIA. Pretraining large language models with NVFP4. *arXiv preprint arXiv:2509.25149 (v2, 4 Mar 2026)*, 2025. URL https://arxiv.org/abs/2509.25149.

[4] Eduardo Alvarez, Omri Almog, Eric Chung, Simon Layton, Dusan Stosic, Ronny Krashinsky, and Kyle Aubrey. Introducing NVFP4 for efficient and accurate low-precision inference. NVIDIA Technical Blog, 24 June 2025, 2025. URL https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/.

[5] Andrei Panferov, Erik Schultheis, Soroush Tabesh, and Dan Alistarh. Quartet II: Accurate LLM pre-training in NVFP4 by improved unbiased gradient estimation. In *Proc. 43rd International Conference on Machine Learning (ICML), PMLR 306*, 2026. URL https://arxiv.org/abs/2601.22813. arXiv:2601.22813.

[6] Yuxiang Chen, Yifan Liu, Xiaoming Xu, Pengle Zhang, Michael Beyer, Martin Rapp, Jun Zhu, and Jianfei Chen. TetraJet-v2: Accurate NVFP4 training for large language models with oscillation suppression and outlier control. In *Proc. 43rd International Conference on Machine Learning (ICML), PMLR 306*, 2026. URL https://arxiv.org/abs/2510.27527. arXiv:2510.27527.

[7] Jack Cook, Junxian Guo, Guangxuan Xiao, Yujun Lin, Keith Wyss, Mahdi Nazemi, Asit Mishra, Carlo del Mundo, Tijmen Blankevoort, and Song Han. Four over six: More accurate NVFP4 quantization with adaptive block scaling. *arXiv preprint arXiv:2512.02010 (v5, 9 May 2026)*, 2025. URL https://arxiv.org/abs/2512.02010.

[8] Mehdi Rahimifar, Amin Darabi, Xing Huang, Zhijun Tu, Yunke Peng, Mehran Taghian Jazi, Yao Wang, Yufei Cui, and Hongliang Li. Stable FP4 training via transposition-invariant block quantization. *arXiv preprint arXiv:2607.24953*, 2026. URL https://arxiv.org/abs/2607.24953.

[9] Siyu Ding, Mingchuan Ma, Xingrun Xing, Ziming Wang, Jiabo Tong, and Guoqi Li. Full-stack FP4: Stable LLM pretraining with quantized projections, optimizers, and attention. *arXiv preprint arXiv:2607.04422 (v2, 7 Aug 2026)*, 2026. URL https://arxiv.org/abs/2607.04422.

[10] Anjulie Agrusa, Andrei Panferov, Elizabeth Wei, Keith Wyss, Paul Gibbons, Erik Schultheis, Jinhang Choi, Frank Sun, Tijmen Blankevoort, and Dan Alistarh. What matters for NVFP4 training? a comparative study of low-precision pretraining recipes. *arXiv preprint arXiv:2610.24343v1 (17 Oct 2026); ICML 2026 AdaptFM workshop*, 2026. URL https://arxiv.org/abs/2610.24343.

[11] Mengzhao Chen, Meng Wu, Hui Jin, Zhihang Yuan, Jing Liu, Chaoyi Zhang, Yunshui Li, Jie Huang, Jin Ma, Zeyue Xue, Zhiheng Liu, Xingyan Bin, and Ping Luo. INT v.s. FP: A comprehensive study of fine-grained low-bit quantization formats. *arXiv preprint arXiv:2510.25602*, 2025. URL https://arxiv.org/abs/2510.25602.

[12] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Roberto L. Castro, Shubhra Pandit, Alexandre Marques, Mark Kurtz, Saleh Ashkboos, Torsten Hoefler, Eldar Kurtić, and Dan Alistarh. Bridging the gap between promise and performance for microscaling FP4 quantization. In *International Conference on Learning Representations (ICLR)*, 2026. URL https://arxiv.org/abs/2509.23202. arXiv:2509.23202.

[13] Andrea Fasoli, Monodeep Kar, Chi-Chun Liu, Viji Srinivasan, Leland Chang, Naigang Wang, and Swagath Venkataramani. Is finer better? the limits of microscaling formats in large language models. *arXiv preprint arXiv:2601.19026*, 2026. URL https://arxiv.org/abs/2601.19026.

[14] Robert Hu, Carlo Luschi, and Paul Balanca. UE5M3 FP4 block scaling for stable language model pretraining. *Graphcore Research technical report, arXiv:2609.02846*, 2026. URL https://arxiv.org/abs/2609.02846.

[15] NVIDIA. NVIDIA Nemotron 3 Super: Open, efficient mixture-of-experts hybrid Mamba-Transformer model for agentic reasoning. Technical report, NVIDIA Research, 2026. URL https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Super-Technical-Report.pdf.

[16] DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026. URL https://arxiv.org/abs/2606.19348.

[17] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025. URL https://arxiv.org/abs/2508.10925.

[18] Moonshot AI. Kimi K2 thinking (model card). Hugging Face model card, 2025. URL https://huggingface.co/moonshotai/Kimi-K2-Thinking.

[19] Mehran Taghian, Yunke Peng, Xing Huang, Yao Wang, et al. HiFloat4 format for language model pre-training on ascend NPUs. *arXiv preprint arXiv:2604.08826 (v2, 25 Sep 2026)*, 2026. URL https://arxiv.org/abs/2604.08826.

[20] Red Hat. Accelerating large language models with NVFP4 quantization. Red Hat Developer article, 4 February 2026, 2026. URL https://developers.redhat.com/articles/2026/02/04/accelerating-large-language-models-nvfp4-quantization.

[21] umans.ai. GLM-5.2 NVFP4: fast, cheap, and not worth serving. Company blog, summer 2026, 2026. URL https://blog.pr-602.umans.ai/blog/glm-5-2-nvfp4-not-worth-serving/.

[22] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018. arXiv:1710.03740.

[23] Asit Mishra, Dusan Stosic, Simon Layton, and Paulius Micikevicius. Recipes for pre-training LLMs with MXFP8. *arXiv preprint arXiv:2506.08027*, 2025. URL https://arxiv.org/abs/2506.08027.

[24] Haocheng Xi, Changhao Li, Jianfei Chen, and Jun Zhu. Training transformers with 4-bit integers. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[25] Brian Chmiel, Ron Banner, Elad Hoffer, Hilla Ben-Yaacov, and Daniel Soudry. Accurate neural training with 4-bit matrix multiplications at standard formats. In *International Conference on Learning Representations*, 2023.

[26] Ruizhe Wang, Yeyun Gong, Xiao Liu, Guoshuai Zhao, Ziyue Yang, Baining Guo, Zhengjun Zha, and Peng Cheng. Optimizing large language model training using FP4 quantization. *arXiv preprint arXiv:2501.17116*, 2025. URL https://arxiv.org/abs/2501.17116.

[27] Albert Tseng, Tao Yu, and Youngsuk Park. Training LLMs with MXFP4. In *Proc. AISTATS (PMLR 258)*, 2025. URL https://arxiv.org/abs/2502.20586. arXiv:2502.20586.

[28] Jiecheng Zhou, Ding Tang, Rong Fu, Boni Hu, Haoran Xu, Yi Wang, Zhilin Pei, Zhongling Su, Liang Liu, Xingcheng Zhang, and Weiming Zhang. Towards efficient pre-training: Exploring FP4 precision in large language models. *arXiv preprint arXiv:2502.11458*, 2025.

[29] Yuxiang Chen, Haocheng Xi, Jun Zhu, and Jianfei Chen. Oscillation-reduced MXFP4 training for vision transformers. In *Proc. ICML (PMLR 267)*, 2025. URL https://arxiv.org/abs/2502.20853. arXiv:2502.20853.

[30] Roberto L. Castro, Andrei Panferov, Soroush Tabesh, Oliver Sieberling, Jiale Chen, Mahdi Nikdan, Saleh Ashkboos, and Dan Alistarh. Quartet: Native FP4 training can be optimal for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, 2025. URL https://arxiv.org/abs/2505.14669. arXiv:2505.14669.

[31] Brian Chmiel, Maxim Fishman, Ron Banner, and Daniel Soudry. FP4 all the way: Fully quantized training of LLMs. *arXiv preprint arXiv:2505.19115*, 2025. URL https://arxiv.org/abs/2505.19115.

[32] Kirthi Devleker and Farshad Ghodsian. NVFP4 trains with precision of 16-bit and speed and efficiency of 4-bit. NVIDIA Technical Blog, 25 August 2025, 2025. URL https://developer.nvidia.com/blog/nvfp4-trains-with-precision-of-16-bit-and-speed-and-efficiency-of-4-bit/.

[33] NVIDIA. Inside NVIDIA Blackwell Ultra: The chip powering the AI factory era. NVIDIA Technical Blog, 2025. URL https://developer.nvidia.com/blog/inside-nvidia-blackwell-ultra-the-chip-powering-the-ai-factory-era.

[34] NVIDIA. NVIDIA Blackwell architecture sweeps MLPerf training v5.1 benchmarks. NVIDIA Technical Blog, November 2025, 2025. URL https://developer.nvidia.com/blog/nvidia-blackwell-architecture-sweeps-mlperf-training-v5-1-benchmarks/.

[35] NVIDIA. NVFP4 – Transformer Engine user guide (low-precision training). NVIDIA Transformer Engine documentation, 2026. URL https://nvidia.github.io/TransformerEngine/features/low_precision_training/nvfp4/nvfp4.html.

[36] InsiderLLM. FP4 just landed in llama.cpp: NVFP4 vs MXFP4 explained (2026). Web article (with later corrections), 2026. URL https://insiderllm.com/guides/fp4-inference-llamacpp-nvfp4-mxfp4/.

[37] Shay Vargaftik, Ran Ben Basat, Amit Portnoy, Gal Mendelson, Yaniv Ben-Itzhak, and Michael Mitzenmacher. EDEN: Communication-efficient and robust distributed mean estimation for federated learning. In *Proc. ICML (PMLR 162)*, 2022.

[38] Albert Tseng, Jerry Chee, Qingyao Sun, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. *arXiv preprint arXiv:2402.04396*, 2024.

[39] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. QuaRot: Outlier-free 4-bit inference in rotated LLMs. *arXiv preprint arXiv:2404.00456*, 2024.

[40] DeepSeek-AI, Aixin Liu, et al. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[41] NVIDIA. Nemotron-H: A family of accurate and efficient hybrid Mamba-Transformer models. *arXiv preprint arXiv:2504.03624*, 2025.

[42] Peijie Dong, Ruibo Fan, Yuechen Tao, Di Mou, Wenhu Hu, Zhenheng Tang, Yinghao Yu, Jiamang Wang, Wenbo Su, Guodong Yang, Liping Zhang, Xiaowen Chu, Baochun Li, and Bo Li. Dissecting outlier dynamics in LLM NVFP4 pretraining. *arXiv preprint arXiv:2602.02047*, 2026. URL https://arxiv.org/abs/2602.02047.

[43] Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. Overcoming oscillations in quantization-aware training. In *Proc. ICML (PMLR 162)*, 2022.

[44] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*, 2022.

[45] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.

[46] Musa Cim, Sarthak Arora, Poovaiah Palangappa, Miro Hodak, Ravi Dwivedula, Meena Arunachalam, and Mahmut Taylan Kandemir. Pretraining large language models with MXFP4 on native FP4 hardware. *arXiv preprint arXiv:2605.09825 (v4, 12 Aug 2026)*, 2026. URL https://arxiv.org/abs/2605.09825.

[47] Meng Xin, Sweta Priyadarshi, Jingyu Xin, Bilal Kartal, Aditya Vavre, et al. Quantization-aware distillation for NVFP4 inference accuracy recovery. *NVIDIA technical report, arXiv:2601.20088 (v3, 3 Mar 2026)*, 2026. URL https://arxiv.org/abs/2601.20088.

[48] Wei Huang, Yukang Chen, Weian Mao, Luozhou Wang, Shuai Yang, and Song Han. Pushing intelligence to 4-bit. NVIDIA Efficient AI research blog, 29 June 2026, 2026. URL https://research.nvidia.com/labs/eai/blogs/pushing-intelligence-to-4-bit/.

[49] AI Weekly. Deep learning: 4-bit pretraining just matched FP8 at scale. Newsletter issue, 26 May 2026. URL https://aiweekly.co/issues/deep-learning-4-bit-pretraining-just-matched-fp8-at-scale.

[50] NVIDIA. Nemotron 3 super pretraining (stage 0) documentation. NVIDIA Nemotron docs v0.1.0. URL https://docs.nvidia.com/nemotron/0.1.0/nemotron/super3/pretrain.html.

[51] Ben Dickson. NVIDIA researchers unlock 4-bit LLM training that matches 8-bit performance. VentureBeat, 28 October 2025, . URL https://venturebeat.com/ai/nvidia-researchers-unlock-4-bit-llm-training-that-matches-8-bit-performance.

[52] Ben Dickson. NVIDIA's NVFP4 enables 4-bit LLM training without the accuracy trade-off. TechTalks, 10 November 2025, . URL https://bdtechtalks.com/2025/11/10/nvidia-nvfp4-llm-quantization/.

[53] Pengxiang Zhao, Hui-Ling Zhen, Xing Li, Han Bao, Weizhe Lin, Zhiyuan Yang, Manyi Zhang, Yuanyong Luo, Ziwei Yu, Xin Wang, Mingxuan Yuan, Xianzhi Yu, and Zhenhua Dong. Unleashing low-bit inference on ascend NPUs: A comprehensive evaluation of HiFloat formats. *arXiv preprint arXiv:2602.12635*, 2026. URL https://arxiv.org/abs/2602.12635.

[54] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[55] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *arXiv preprint arXiv:2306.00978*, 2023.

[56] Reddit users. r/machinelearningnews post on NVFP4 pretraining (archived snapshot). Reddit, 18 May 2026 (snapshot 20 May 2026). URL https://reddit.sentinel-team.org/view/post/11343186.